

② 座・Navi 全ファイル解説書（コードリーディング教材）

この資料は、座・Navi の **全40ファイル**（Javaソース21 + JSP/JSPF 19）を、プログラミング初学者でも上から順に読んでいけば理解できるように解説したものです。

「①プログラム解説書」が全体の地図だとすると、この資料は **その地図の上を実際に1行ずつ歩いていく** ための教材です。チームでの勉強会・コードリーディング会のベース資料として使うことを想定しています。

0. 本書の使い方

0-1. 全体の進め方

1. まず ①**プログラム解説書**（01_プログラム解説書.pdf）に目を通し、「座・Naviとは何か」「画面はどう繋がっているか」「DBはどうなっているか」の大枠を把握する。
2. 本書「1. Java/JSPの基礎知識集（用語集）」をいったん流し読みする。最初は全部わからなくてOK。「あとで戻ってくる索引」として使う。
3. 本書「2. 推奨読書プラン」の Day1 から順にファイルを読み進める。
4. 各ファイルの解説の中で **はじめて出てくる構文・メソッド** には **■用語集 → ○○** という形で印をつけている。わからなければ「1. 用語集」の該当項目に戻って確認する。
5. 1週間（Day1～Day7）で全ファイルを一通り読み終えるのが目標。2周目は「①解説書」の機能解説（7章・8章）と合わせて読むと理解が深まる。

0-2. コードの読み方の凡例

- コードブロックは実際のソースコードそのまま（行頭の `package` や `import` も含む）。
 - コードの直後に「**ここで新しく出てくるもの**」という小見出しで、初出の構文・メソッドをまとめている。
 - 2回目以降に同じ構文が出てきたときは、いちいち説明を繰り返さず **■用語集 → ○○（○○.javaで既出）** のように参照する形にしている。
 - `entity` `dao` `model` `control` といった**パッケージ名**は、フォルダ名と完全に一致している（`src/main/java/entity/` フォルダ = `entity` パッケージ）。
-

1. Java/JSPの基礎知識集（用語集）

ここでは、座・Naviのコードに出てくる構文・クラス・メソッドのうち、初学者がつまずきやすいものをまとめている。最初から全部理解する必要はない。本編を読んでいて「これ何だっけ？」となったなら、ここに戻ってくる。

1-1. クラスの基本

用語	説明	初出ファイル
<code>package</code> 文	そのファイルがどのフォルダ（パッケージ）に属するかを示す宣言。 <code>package entity;</code> なら <code>src/main/java/entity/</code> フォルダにある。	<code>entity/Account.java</code>
<code>import</code> 文	他のパッケージのクラスを使うための宣言。 <code>import java.util.*;</code> は <code>java.util</code> パッケージの全クラスを使えるようにする。	<code>entity/Account.java</code>
フィールド（インスタンス変数）	クラスの中に直接書く変数。そのクラスのオブジェクト1つにつき1つの値を持つ「データの入れ物」。 <code>private</code> を付けるとクラスの外から直接アクセスできない。	<code>entity/Account.java</code>
<code>getter</code> / <code>setter</code>	フィールドが <code>private</code> のとき、外部から読み書きするための専用メソッド。 <code>getName()</code> で読む、 <code>setName(値)</code> で書く。	<code>entity/Account.java</code>
コンストラクタ	<code>new クラス名(...)</code> で呼ばれる、オブジェクト生成時の初期化処理。クラス名と同じ名前で戻り値の型がない。	<code>entity/Account.java</code>
コンストラクタのオーバーロード	同じ名前でも引数の数・型が違うコンストラクタを複数定義できること。 <code>Account()</code> （空）と <code>Account(userId, password, ...)</code> （全項目）の2種類がある。	<code>entity/Account.java</code>
<code>int</code> / <code>String</code> などの型	<code>int</code> は整数、 <code>String</code> は文字列。Javaは「変数の型を最初に決める」言語（C#・Javaなど静的型付け言語の特徴）。	<code>entity/Account.java</code>

1-2. インターフェースと例外処理

用語	説明	初出ファイル
<code>interface</code>	「このメソッドを必ず持っていること」を約束する型。実際の処理は書かず、メソッドの名前・引数・戻り値の型だけを定義する。	<code>model/Logic.java</code>
<code>implements</code>	あるクラスが、指定した <code>interface</code> の約束を実装することを示す。 <code>class LoginLogic implements Logic</code> なら、 <code>LoginLogic</code> は <code>Logic</code> が要求する <code>execute()</code> メソッドを必ず持つ。	<code>model/LoginLogic.java</code>
<code>throws Exception</code>	このメソッドの中で例外（エラー）が発生する可能性があり、呼び出し元で対処してもらう、という宣言。	<code>model/Logic.java</code>
<code>try</code> / <code>catch</code>	<code>try</code> の中で例外が起きたら <code>catch</code> の中の処理に飛ぶ、というエラー処理の基本構文。	<code>dao/DBUtil.java</code>
<code>throw new RuntimeException(...)</code>	自分で例外を発生させる文。「ここで想定外のエラーが起きた」ことを呼び出し元に伝える。	<code>dao/AccountsDAO.java</code>
<code>try-with-resources</code>	<code>try (Connection conn = ...; PreparedStatement ps = ...) { ... }</code> の形。 <code>{ }</code> を抜けるときに <code>conn</code> や <code>ps</code> を自動的に <code>close()</code> してくれる（後始末忘れを防ぐ）。	<code>dao/AccountsDAO.java</code>

1-3. JDBC（データベース操作）関連

用語	説明	初出ファイル
<code>static</code> 変数	クラス全体で1つだけ共有される変数（インスタンスごとに増えない）。 <code>private static final String URL = ...</code> のように、設定値を入れることが多い。	<code>dao/DBUtil.java</code>
<code>static { ... }</code> (静的初期化ブロック)	そのクラスが最初に使われるタイミングで一度だけ実行される処理。 <code>DBUtil</code> ではJDBCドライバの読み込みに使っている。	<code>dao/DBUtil.java</code>
<code>Class.forName(...)</code>	指定した名前のクラス（ここではMySQL用のJDBCドライバ）をロードする。	<code>dao/DBUtil.java</code>
<code>Connection</code>	データベースへの接続そのものを表すオブジェクト。 <code>DriverManager.getConnection(URL, USER, PASS)</code> で取得する。	<code>dao/DBUtil.java</code>
<code>PreparedStatement</code>	SQL文を実行するためのオブジェクト。 <code>?</code> というプレースホルダにあとから値をセットできる（SQLインジェクション対策にもなる）。	<code>dao/AccountsDAO.java</code>
<code>ResultSet</code>	SQLの <code>SELECT</code> 結果（複数行）を1行ずつ取り出すためのオブジェクト。 <code>rs.next()</code> で次の行に進み、 <code>rs.getString("列名")</code> など値を取る。	<code>dao/AccountsDAO.java</code>
<code>ps.setInt(1, id)</code> / <code>ps.setString(2, ...)</code>	<code>PreparedStatement</code> の <code>?</code> に値をセットする。第1引数は「何番目の <code>?</code> か」（1始まり）。	<code>dao/AccountsDAO.java</code>
<code>Statement.RETURN_GENERATED_KEYS</code>	<code>INSERT</code> 実行後、DBが自動で振った連番（オートインクリメントID）を取得したいときに指定するオプション。	<code>dao/ReservationsDAO.java</code>

1-4. コレクション（List・Map）とループ

用語	説明	初出ファイル
<code>List<T></code> / <code>ArrayList<T></code>	複数のデータを「順番付きで」入れておく箱。 <code>List<Account></code> なら <code>Account</code> がたくさん入っている。 <code>ArrayList</code> はその実装の1つ。	<code>dao/AccountsDAO.java</code>
<code>for (Type x : collection)</code> （拡張for文 / for-each）	<code>List</code> や配列の中身を1つずつ取り出して処理するループ。 「 <code>collection</code> の中の <code>x</code> ひとつひとつについて」という意味。	<code>dao/AccountsDAO.java</code>
<code>while (rs.next())</code>	<code>ResultSet</code> の行がある限りループする、SELECT結果を全部読むときの定番パターン。	<code>dao/AccountsDAO.java</code>
<code>Map<K, V></code> / <code>HashMap<K, V></code>	「キーと値」のペアを保存する箱。例: <code>Map<Integer, String></code> なら「座席ID→ユーザーID」のような対応表になる。	<code>model/LocationRegistLogic.java</code>
<code>Map.put(key, value)</code> / <code>Map.get(key)</code> / <code>Map.containsKey(key)</code>	<code>Map</code> に値を追加する／キーから値を取り出す／キーが存在するか確認する、それぞれの基本操作。	<code>model/LocationRegistLogic.java</code>
<code>LinkedHashMap<K, V></code>	<code>HashMap</code> と同じだが、 追加した順番を保持する <code>Map</code> 。座席利用状況の日付一覧などで「順番が大事」なときに使う。	<code>control/ControlServlet.java</code>
<code>Map.Entry<K, V></code> / <code>entrySet()</code>	<code>Map</code> の「キーと値のペア」を1つずつ取り出すための型。 <code>for (Map.Entry<K,V> e : map.entrySet())</code> で全ペアをループできる。	<code>control/ControlServlet.java</code>

1-5. 日付・文字列操作

用語	説明	初出ファイル
<code>LocalDate</code>	日付（年月日のみ、時刻なし）を表すクラス。 <code>LocalDate.now()</code> で今日の日付、 <code>.plusDays(n)</code> で n 日後の日付を取得できる。	<code>model/LocationRegistLogic.java</code>
<code>String.equals(...)</code>	文字列の内容が同じかどうかを比較する。Javaでは文字列の比較に <code>==</code> ではなく <code>.equals()</code> を使う（ <code>==</code> だと「同じオブジェクトかどうか」になってしまうため）。	<code>model/LocationRegistLogic.java</code>
<code>String.trim()</code>	文字列の前後にある余分な空白を取り除く。検索キーワードの入力チェックなどで使う。	<code>model/NameSearchLogic.java</code>
<code>String.isEmpty()</code>	文字列が空文字 <code>""</code> かどうかを判定する。	<code>model/NameSearchLogic.java</code>
<code>StringBuilder</code>	文字列を効率よく組み立てるためのクラス。 <code>.append(...)</code> でどんどん文字列を追加していく。SQL文を条件によって組み立てるときによく使う。	<code>model/LocationRegistLogic.java</code>
<code>URLEncoder.encode(文字列, "UTF-8")</code>	日本語などをURLに埋め込めるように変換する（例: 「三田」 → <code>%E4%B8%89%E7%80%9...</code> ）。リンクのパラメータに日本語の拠点名を入れるときに必要。	<code>model/ReserveLogic.java</code>

1-6. Stream API（やや難しい・MAP/Listが得意な人向け）

「①プログラム解説書」でも触れているが、ここでは実際のコードに即してもう少し詳しく説明する。

用語	説明	初出ファイル
<code>list.stream()</code>	<code>List</code> （や <code>Map</code> ）を「流れ作業（ストリーム）」で処理するための入り口。これだけでは何も起きない。	<code>model/ReserveLogic.java</code>
<code>.filter(条件)</code>	ストリームの中から「条件に合うものだけ」を残す。条件部分はラムダ式（後述）で書く。	<code>model/ReserveLogic.java</code>
<code>.count()</code>	ストリームに残っている要素の数を数える（ <code>long</code> 型で返る）。 <code>.stream().filter(...).count()</code> で「条件に合う件数」を一発で数えられる。	<code>model/ReserveLogic.java</code>
<code>.collect(Collectors.toList())</code>	ストリームの処理結果を、もう一度 <code>List</code> に戻す。 <code>.filter()</code> だけだとストリームのままなので、 <code>List</code> として使いたいときに必要。	<code>model/MasterUpdateLogic.java</code>
ラムダ式 <code>s -> 条件式</code>	「 <code>s</code> を受け取って、右側の式を計算して返す小さな関数」をその場で書く記法。 <code>.filter(r -> "出社".equals(r.getGyosaki()))</code> は「 <code>r</code> の行先が"出社"のものだけ残す」という意味。	<code>model/ReserveLogic.java</code>

1-7. Servlet / JSP関連

用語	説明	初出ファイル
<code>HttpServletRequest req</code> / <code>HttpServletResponse res</code>	ブラウザからのリクエスト情報（パラメータなど）と、ブラウザへ返すレスポンスを表すオブジェクト。すべての <code>Logic</code> クラスの <code>execute()</code> で受け取る。	<code>model/Logic.java</code>
<code>req.getParameter("名前")</code>	フォームやURLから送られてきたパラメータの値を取得する（戻り値は常に <code>String</code> 、無ければ <code>null</code> ）。	<code>model/LoginLogic.java</code>
<code>req.setAttribute("名前", 値)</code> / <code>req.getAttribute("名前")</code>	JSP側に値を渡すための入れ物に値を入れる／取り出す。1回のリクエストの中だけ有効。	<code>model/LoginLogic.java</code>
<code>HttpSession session</code>	ログイン中のユーザーごとに保持される情報（ログインしている間ずっと使える）。 <code>session.setAttribute("userId", ...)</code> のように使う。	<code>model/LoginLogic.java</code>
<code>session.invalidate()</code>	セッションを破棄する＝ログアウト処理の本体。	<code>model/LogoutLogic.java</code>
<code>res.sendRedirect(URL)</code> と <code>forward</code> の違い	<code>forward</code> はサーバー内部で別のJSPに処理を引き渡す（URLは変わらない）。 <code>sendRedirect</code> はブラウザに「このURLに行き直して」と指示する（URLが変わり、ブラウザが再度リクエストし直す）。座・Naviではログイン直後やメニュー更新時に <code>sendRedirect</code> を使い、メニューの集計を最新化させている。	<code>model/LoginLogic.java</code>
<code><%@ page import="..." %></code>	JSPファイルの先頭で、その中で使うJavaクラスを <code>import</code> する宣言。	<code>webapp/.../reserve.jsp</code>
<code><% ... %></code> （スクリプトレット）	JSPファイルの中に直接Javaコードを書く部分。	<code>webapp/.../reserve.jsp</code>
<code><%= ... %></code> （式）	Javaの式の結果をそのままHTMLに出力する部分。 <code><%= a.getName() %></code> ならその人の名前が表示される。	<code>webapp/.../reserve.jsp</code>
三項演算子 <code>条件 ? 値A : 値B</code>	「条件が真なら値A、偽なら値Bになる」という、 <code>if</code> を1行で書ける書き方。	<code>model/LoginLogic.java</code>

2. 推奨読書プラン（Day1～Day7）

全40ファイルを7日間に分けて読む。1日あたり 5～8ファイル、新しく覚える構文は3～5個程度になるように構成している。

Day	テーマ	読むファイル
Day1	全体の土台：データの入れ物（entity）とインターフェース	model/Logic.java、entity/Account.java、entity/Seat.java、entity/Location.java、entity/Reservation.java、dao/DBUtil.java
Day2	ログイン・ログアウトの一往復	model/LoginLogic.java、model/LogoutLogic.java、dao/AccountsDAO.java、webapp/.../login.jsp、webapp/.../loginNG.jsp、webapp/.../logout.jsp、webapp/.../common/header.jspf、webapp/.../common/headerAdmin.jspf
Day3	行先登録（出社・在宅・出張）	model/LocationRegistLogic.java、dao/LocationsDAO.java、dao/ReservationsDAO.java、webapp/.../locationRegist.jsp、webapp/.../businessTrip.jsp、webapp/.../remoteWork.jsp
Day4	座席予約（4段階ウィザード）	model/ReserveLogic.java、dao/SeatsDAO.java、webapp/.../reserve.jsp、webapp/.../reserveOK.jsp、webapp/.../reserveNG.jsp
Day5	予約確認・氏名検索・部門検索	model/ReserveConfirmLogic.java、webapp/.../reserveConfirm.jsp、model/NameSearchLogic.java、webapp/.../nameSearch.jsp、model/DepartmentSearchLogic.java、webapp/.../departmentSearch.jsp
Day6	メニュー集計・座席利用状況・代理予約（管理者機能）	control/ControlServlet.java、webapp/.../menu.jsp、webapp/.../adminMenu.jsp、webapp/.../seatStatus.jsp、model/AdminReserveLogic.java、webapp/.../adminReserve.jsp
Day7	座席マスタ管理 + 総まとめ	model/MasterUpdateLogic.java、dao/MastersDAO.java、webapp/.../masterUpdate.jsp、「3-終.まとめと索引」

💡 各Dayの最後に、その日読んだファイルが①解説書のどの章とつながっているかを「関連」として示している。気になったら①解説書も合わせて確認するとよい。

3. ファイル別解説

Day1: 全体の土台 (entity・Logicインターフェース・DB接続)

Day1では、座・Navi全体で繰り返し出てくる「データの入れ物 (entity)」と、すべての画面処理が実装する「Logicインターフェース」、そしてDBへの接続方法 (DBUtil) を確認する。 **新しい構文が一番多い日**なので、慌てず一つずつ確認していく。

3-1-1. model/Logic.java (全画面処理の"共通の型")

```
package model;

import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

/**
 * 各機能（ログイン・予約・検索など）の処理を表すインターフェース。
 * ControlServlet は action の値に応じて、対応するLogicの execute() を呼び出すだけ。
 *
 * execute() の戻り値（forward先のJSPパス）について：
 *   - JSPのパス（例: "/WEB-INF/jsp/menu.jsp"）を返すと、ControlServletがそのJSPへforwardする。
 *   - null を返すと、ControlServlet は何もしない。
 *   その代わり、Logic側で既に res.sendRedirect(...) を呼び終えている、という意味になる。
 *   （リダイレクト後のリクエストで、メニューの集計処理などを再実行させたい場合に使う）
 */
public interface Logic {
    String execute(HttpServletRequest req, HttpServletResponse res) throws Exception;
}
```

ここで新しく出てくるもの

- package model; ... このファイルは src/main/java/model/ フォルダに属する。■用語集 → package文
- import javax.servlet.http.HttpServletRequest; ... Servlet APIが提供する「リクエスト」を表す型を読み込む。■用語集 → import文
- public interface Logic { ... } ... 「execute という名前のメソッドを持つこと」を約束する型。■用語集 → interface
- String execute(HttpServletRequest req, HttpServletResponse res) throws Exception; ... メソッドのシグネチャ（名前・引数・戻り値の型）だけを定義し、処理本体（{ }）は書かない。実装は implements Logic するクラス側で書く。

読み方のポイント

座・Naviの全ロジッククラス（LoginLogic、ReserveLogic、MasterUpdateLogic ...）は、すべてこの Logic を implements している。つまり「execute(req, res) を呼べば、その機能の処理結果（次に表示するJSPのパス）が返ってくる」という**共通のルール**がある。ControlServlet はこのルールのおかげで、action の値ごとに new ○○Logic().execute(req, res) を呼ぶだけでよくなっている（Day6で詳しく見る）。

3-1-2. entity/Account.java (ユーザー1人分のデータ)

```
package entity;

/**
 * users テーブルの1行分を表すentity。
 *
 *   userId   : 社員ID (DBでは数値だが、画面・ロジック側では String として扱う)
 *   password : パスワード (平文。学習用のため暗号化はしていない)
 *   name      : 氏名
 *   bId       : 所属部署ID (departments.b_id と対応)
 *   admin     : 権限 (0=一般ユーザー, 1=管理者)
 */
public class Account {
    private String userId;
    private String password;
    private String name;
    private String bId;
    private int admin;

    public Account() {}

    public Account(String userId, String password, String name, String bId, int admin) {
        this.userId = userId;
        this.password = password;
        this.name = name;
        this.bId = bId;
        this.admin = admin;
    }

    public String getUserId() { return userId; }
    public void setUserId(String userId) { this.userId = userId; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getBId() { return bId; }
    public void setBId(String bId) { this.bId = bId; }
    public int getAdmin() { return admin; }
    public void setAdmin(int admin) { this.admin = admin; }
}
```

ここで新しく出てくるもの

- `private String userId;` など… **フィールド**。 `Account` のオブジェクト1つにつき、これらの値を1組持つ。 ■ 用語集 → フィールド
- `public Account() {}` … 引数なしのコンストラクタ (空っぽ)。 `new Account()` で「空のAccount」を作れる。
- `public Account(String userId, ...) { this.userId = userId; ... }` … 引数ありのコンストラクタ。 `this.userId` は「このオブジェクト自身のuserIdフィールド」を指す。 ■ 用語集 → コンストラクタ/オーバーロード
- `public String getUserId() { return userId; }` / `public void setUserId(String userId) { this.userId = userId; }` … getter/setter。 ■ 用語集 → getter/setter

読み方のポイント

`Account` クラスには「処理 (ロジック)」が一切書かれていない。**データを持つだけのクラス**であることに注目する。「ログインできるか」「どの座席を予約したか」といった処理は、このあと出てくる `dao` や `model` パッケージのクラスが行う。このように「データ」と「処理」を分けて書くのが、座・Naviのようなアプリでの基本的な設計パターン。

3-1-3. entity/Seat.java (座席1つ分のデータ)

```
package entity;

/**
 * seats テーブルの1行分を表すentity (座席マスタ1件 = 1つの座席)。
 *
 * seatId   : 座席ID (一意の番号)
 * baseName : 拠点名 (例: 三田・芝浦・中野)
 * fName    : フロア名 (例: 1F・2F)
 * areaName : エリア名 (例: A・B・C)
 * seatName : 座席番号 (例: A-01)
 */
public class Seat {
    private int seatId;
    private String baseName;
    private String fName;
    private String areaName;
    private String seatName;

    public Seat() {}

    public Seat(int seatId, String baseName, String fName, String areaName, String seatName) {
        this.seatId = seatId;
        this.baseName = baseName;
        this.fName = fName;
        this.areaName = areaName;
        this.seatName = seatName;
    }

    public int getSeatId() { return seatId; }
    public void setSeatId(int seatId) { this.seatId = seatId; }
    public String getBaseName() { return baseName; }
    public void setBaseName(String baseName) { this.baseName = baseName; }
    public String getFName() { return fName; }
    public void setFName(String fName) { this.fName = fName; }
    public String getAreaName() { return areaName; }
    public void setAreaName(String areaName) { this.areaName = areaName; }
    public String getSeatName() { return seatName; }
    public void setSeatName(String seatName) { this.seatName = seatName; }
}
```

ここで新しく出てくるもの

- `private int seatId;` … `seatId` だけ `int` (数値) 型。他は `String`。これは `Account` と同じ形のクラスなので、新しい構文はない。

読み方のポイント

座席の「場所」は 拠点(baseName) → フロア(fName) → エリア(areaName) → 座席(seatName) の4階層で表現されている。この4階層構造は、Day4の `ReserveLogic` (座席予約の4段階ウィザード) で再び登場する、座・Navi全体を通じて最重要の構造の一つ。

3-1-4. entity/Location.java (部署1つ分のデータ)

```
package entity;

/**
 * departments テーブルの1行分を表すentity (部署マスタ1件)。
 *
 *   bId   : 部署ID
 *   bName : 部署名
 *
 * 注意：クラス名は "Location" だが、内容は「部署」であり、
 * Seat の「拠点 (baseName)」とは別物 (紛らわしいので注意)。
 */
public class Location {
    private String bId;
    private String bName;

    public Location() {}

    public Location(String bId, String bName) {
        this.bId = bId;
        this.bName = bName;
    }

    public String getBId() { return bId; }
    public void setBId(String bId) { this.bId = bId; }
    public String getBName() { return bName; }
    public void setBName(String bName) { this.bName = bName; }
}
```

ここで新しく出てくるもの

新しい構文はなし。Account ・ Seat と同じ形。

読み方のポイント

クラス名 Location (場所) と内容 (部署) が一致していない点に注意。コード中で「Location 型の変数」が出てきたら、「拠点 (オフィスの場所)」ではなく「部署」のことだと読み替える必要がある。これは設計上の命名ミスとして①解説書でも触れている。

3-1-5. entity/Reservation.java (予約・行先登録1件分のデータ)

```
package entity;

/**
 * 予約・行先登録1件分のデータを表すentity (DBの reservations テーブル1行に相当)。
 * 「出社」「在宅」「出張」のいずれであっても、すべてこの1つのクラスで表す
 * (行き先の種類によって座席ID(seatId)があったり0だったりする、という違いだけ)。
 *
 * reserveId : 予約ID (一意の連番)
 * userId    : 予約の本人 (誰の予定か)
 * registerId : 予約を登録した人 (自分で登録ならuserIdと同じ。管理者の代理予約なら管理者のID)
 * reserveDate : 対象日 (YYYY-MM-DD形式の文字列)
 * gyosaki    : 行先の種類 ("出社" / "在宅" / "出張" のいずれか)
 * memo       : 自由記入のメモ
 * seatId     : 予約した座席ID (0の場合は座席なし=在宅・出張など)
 * cancelFlag : 取消状態 (0=有効な予約, 1=取消済み)
 */
public class Reservation {
    private int reserveId;
    private String userId;
    private String registerId;
    private String reserveDate;
    private String gyosaki;
    private String memo;
    private int seatId;
    private int cancelFlag;

    public Reservation() {}

    public Reservation(int reserveId, String userId, String registerId,
        String reserveDate, String gyosaki, String memo,
        int seatId, int cancelFlag) {
        this.reserveId = reserveId;
        this.userId = userId;
        this.registerId = registerId;
        this.reserveDate = reserveDate;
        this.gyosaki = gyosaki;
        this.memo = memo;
        this.seatId = seatId;
        this.cancelFlag = cancelFlag;
    }

    public int getReserveId() { return reserveId; }
    public void setReserveId(int reserveId) { this.reserveId = reserveId; }
    public String getUserId() { return userId; }
    public void setUserId(String userId) { this.userId = userId; }
    public String getRegisterId() { return registerId; }
    public void setRegisterId(String registerId) { this.registerId = registerId; }
    public String getReserveDate() { return reserveDate; }
    public void setReserveDate(String reserveDate) { this.reserveDate = reserveDate; }
    public String getGyosaki() { return gyosaki; }
    public void setGyosaki(String gyosaki) { this.gyosaki = gyosaki; }
    public String getMemo() { return memo; }
    public void setMemo(String memo) { this.memo = memo; }
    public int getSeatId() { return seatId; }
    public void setSeatId(int seatId) { this.seatId = seatId; }
    public int getCancelFlag() { return cancelFlag; }
    public void setCancelFlag(int cancelFlag) { this.cancelFlag = cancelFlag; }
}
```

ここで新しく出てくるもの

新しい構文はなし。ここまでの4つのentityクラスはすべて「フィールド+コンストラクタ2種+getter/setter」という同じパターンで書かれている。このパターンに慣れることが、Day1のいちばんの目的。

読み方のポイント

Reservation は座・Naviで最も重要なentity。「出社」「在宅」「出張」というまったく違う3種類の予定を、1つのテーブル・1つのクラスで表現している点に注目する。

- 出社 → **gyosaki**="出社"、**seatId** に座席番号が入る

- 在宅・出張 → `gyosaki="在宅"または"出張"`、`seatId=0`（座席なし）

この「種類によって一部の項目の意味・使い方が変わる」という設計は、Day3（`LocationRegistLogic` / `ReserveLogic`）で実際の使われ方を見るとよくわかる。

3-1-6. `dao/DBUtil.java`（MySQL接続の共通クラス）

```
package dao;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/**
 * MySQL接続の共通クラス。
 * 各DAOはここから Connection を取得してJDBC操作を行う（接続情報を一箇所にまとめるため）。
 *
 * 接続先DB: zaseki1 (docs/db/za_navi_schema.sql で作成)
 */
public class DBUtil {
    private static final String URL = "jdbc:mysql://localhost:3306/zaseki1?useSSL=false&serverTimezone=Asia/Tokyo&characterEncoding=UTF-8";
    private static final String USER = "root";
    private static final String PASS = "your_password";

    static {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            throw new RuntimeException("MySQL JDBCドライバが見つかりません。jarファイルを追加したか確認してください。", e);
        }
    }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASS);
    }
}
```

ここで新しく出てくるもの

- `private static final String URL = "...";` … `static` は「クラス全体で共有」、`final` は「あとから変更できない（再代入できない）」という意味。接続先・ユーザー名・パスワードのような**設定値**の定番の書き方。■用語集 → static変数
- `static { ... }` … **静的初期化ブロック**。このクラスが最初に使われたとき（≒最初に `DBUtil.getConnection()` が呼ばれる前）に一度だけ実行される。■用語集 → static初期化ブロック
- `try { ... } catch (ClassNotFoundException e) { ... }` … `Class.forName(...)` がドライバを見つけられなかった場合のエラー処理。■用語集 → try/catch
- `throw new RuntimeException(..., e)` … 「このエラーは回復できない致命的なエラーなので、ここで処理を止めて呼び出し元に伝える」という意味。■用語集 → throw new RuntimeException
- `public static Connection getConnection() throws SQLException { return DriverManager.getConnection(URL, USER, PASS); }` … 引数なしでDB接続を1つ作って返すだけの、`static` な（`= new DBUtil()` せずに `DBUtil.getConnection()` の形で直接呼べる）メソッド。

読み方のポイント

- 「**接続情報（URL・ユーザー名・パスワード）を1か所に集約する**」のがこのクラスの役割。各DAO（`AccountsDAO` など）は `DBUtil.getConnection()` を呼ぶだけで、接続文字列を意識しなくてよい。
- URL の中の `zaseki1` がデータベース名。`docs/db/za_navi_schema.sql` を実行して作成したDBと対応している（①解説書 4章 参照）。
- もしDBのパスワードを変更した場合は、**このファイルの PASS を書き換えるだけ**でアプリ全体に反映される。「①解説書」8章のQ&Aでも、DB接続周りのトラブルはまずこのファイルを確認するよう案内している。

Day1のまとめ・①解説書との関連

- Day1で見た4つのentity（`Account` / `Seat` / `Location` / `Reservation`）は、①解説書「4. データベース構成（MySQL zaseki1）」の4テーブルとそれぞれ1対1に対応している。
- `Logic` インターフェースは、①解説書「3. フォルダ構成とファイル一覧」のmodel層、「5. 各レイヤーの詳しい説明」で説明している「Logicクラスの役割」と対応している。
- `DBUtil` は①解説書「5. 各レイヤーの詳しい説明」のJDBC基本パターン（①接続取得～⑦例外処理）の「①接続取得」部分にあたる。

次回（Day2）は、この4つのentityと `Logic` インターフェースを実際に「動かす」最初の例として、ログイン処理（`LoginLogic` ・ `AccountsDAO`）を読んでいく。

Day2: ログイン・ログアウトの一往復

Day2では「ログイン画面を表示する → IDとパスワードを送る → 認証する → メニューへ進む／失敗する」という **一番シンプルな一往復**を、JSP・Logic・DAOの3層を通して読む。Day1で見た `Logic` と `Account` が、ここで初めて実際に動く。

3-2-1. webapp/WEB-INF/jsp/login.jsp (ログイン画面)

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>ログイン - 座・Navi</title>
<link rel="stylesheet" href="%= request.getContextPath() %>/css/style.css">
</head>
<body>
<div class="login-page">
  <div class="login-card">
    <div class="login-title">座・Navi</div>
    <div class="login-subtitle">座席管理システム</div>

    <% if (request.getAttribute("errorMsg") != null) { %>
    <div class="alert alert-danger"><%= request.getAttribute("errorMsg") %></div>
    <% } %>

    <form action="%= request.getContextPath() %>/ControlServlet" method="post">
      <input type="hidden" name="action" value="doLogin">

      <div class="form-group">
        <label>ユーザーID</label>
        <input type="text" name="userId" placeholder="例: user001" required autofocus>
      </div>
      <div class="form-group">
        <label>パスワード</label>
        <input type="password" name="password" placeholder="パスワードを入力" required>
      </div>

      <div style="display:flex; gap:10px; margin-top:24px;">
        <button type="submit" class="btn btn-primary btn-lg" style="flex:1;">
          ログイン
        </button>
      </div>
    </form>

    <div style="margin-top:20px; padding-top:16px; border-top:1px solid #ECEFF1; text-align:center;">
      <p style="font-size:12px; color:#757575; margin-bottom:10px;">管理者の方はこちら</p>
      <form action="%= request.getContextPath() %>/ControlServlet" method="post">
        <input type="hidden" name="action" value="doLogin">
        <input type="hidden" name="loginType" value="admin">
        <div class="form-group" style="margin-bottom:8px;">
          <input type="text" name="userId" placeholder="管理者ID (例: 1001)" required>
        </div>
        <div class="form-group" style="margin-bottom:12px;">
          <input type="password" name="password" placeholder="パスワード" required>
        </div>
        <button type="submit" class="btn btn-outline" style="width:100%;">
          管理者ログイン
        </button>
      </form>
    </div>

    <!-- ▼▼ 差し替え注意: ここは「動作確認用のテストID/パスワードを画面に表示している」サンプル表示です。
    本番環境 (実際のデータベースに繋いだ後) では、ログイン情報を画面に表示するのはセキュリティ上NGなので、
    この <div> ごと削除すること。 ▼▼ -->
    <div style="margin-top:16px; font-size:11px; color:#aaa; text-align:center;">
      ※ デモ: 1002/pass1002 (一般) ・ 1001/pass1001 (管理者)
    </div>
    <!-- ▲▲ 差し替え注意ここまで ▲▲ -->
  </div>
</div>
</body>
</html>
```

ここで新しく出てくるもの

- `<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>` … JSPファイルの先頭に必ず書く宣言。「Javaで書く」「文字コードはUTF-8」という指定。
- `request.getContextPath()` … アプリの公開URLの先頭部分（例: `/za-navi`）。リンクやフォームの送信先を組み立てるときに必ず使う。これを使うことで、アプリ名が変わってもリンクを書き直さずに済む。
- `<% if (request.getAttribute("errorMsg") != null) { %> ... <% } %>` … JSPの中にJavaの `if` 文を書いている。■用語集 → スクリプトレット `<% %>`
- `<input type="hidden" name="action" value="doLogin">` … フォーム送信時に `action=doLogin` という情報も一緒に送る。ControlServletはこの `action` の値で処理を振り分ける（Day6で詳しく）。
- `<input type="hidden" name="loginType" value="admin">` … 「管理者ログイン」フォームから送るときだけ付く目印。`LoginLogic` 側でこれを見て、管理者権限があるかチェックする。

読み方のポイント（要対応の注意点）

ファイル末尾の「デモ: 1002/pass1002（一般）・1001/pass1001（管理者）」という表示は、コード中のコメントにもある通り、**動作確認用の一時的な表示**。実際に社内で使う場合は、この `<div>` を削除する必要がある（①解説書にも記載済みの既知の残作業）。

3-2-2. model/LoginLogic.java (ログイン処理本体)

```
package model;

import dao.AccountsDAO;
import dao.ReservationsDAO;
import entity.Account;
import entity.Reservation;
import java.io.IOException;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.util.List;

/**
 * ログイン処理 (action=login で画面表示／action=doLogin で認証)。
 *
 * ★ここで session にセットする userId / userName / isAdmin / bId の4つは、
 * ログイン後すべての画面・Logicから session.getAttribute(...) で参照される
 * 「誰がログインしているか」の基本情報。新しい画面を作るときもこの4つを使えばよい。
 */
public class LoginLogic implements Logic {

    @Override
    public String execute(HttpServletRequest req, HttpServletResponse res) throws Exception {
        String action = req.getParameter("action");

        if ("doLogin".equals(action)) {
            return doLogin(req, res);
        }
        // action=login: ログイン画面を表示するだけ
        return "/WEB-INF/jsp/login.jsp";
    }

    private String doLogin(HttpServletRequest req, HttpServletResponse res) throws IOException {
        String userId = req.getParameter("userId");
        String password = req.getParameter("password");

        Account account = new AccountsDAO().authenticate(userId, password);
        if (account == null) {
            req.setAttribute("errorMsg", "ユーザーIDまたはパスワードが正しくありません。");
            return "/WEB-INF/jsp/loginNG.jsp";
        }

        // 「管理者ログイン」フォームから入力された場合は、管理者権限を持つアカウントのみ許可する
        // (一般社員が管理者ログインから一般メニューに入れてしまわないようにするため)
        if ("admin".equals(req.getParameter("loginType"))) && account.getAdmin() != 1 {
            req.setAttribute("errorMsg", "管理者権限がありません。");
            return "/WEB-INF/jsp/loginNG.jsp";
        }

        // セッションを新規発行してユーザー情報を格納
        HttpSession session = req.getSession(true);
        session.setAttribute("userId", account.getUserId());
        session.setAttribute("userName", account.getName());
        session.setAttribute("isAdmin", account.getAdmin() == 1);
        session.setAttribute("bId", account.getBId());

        // 本日の予約を取得してセッションに保持
        List<Reservation> todayRes = new ReservationsDAO().findTodayByUserId(account.getUserId());
        session.setAttribute("todayReservations", todayRes);

        // メニューJSPへ直接forwardすると、ControlServlet#showMenu()が用意する
        // 「同じ部署の出社状況」「本日の予約」「今週の予約状況」などの表示用データが
        // セットされないまま画面が表示されてしまう。必ず action=menu / adminMenu を
        // 経由させて showMenu() を実行させるため、リダイレクトする。
        if (account.getAdmin() == 1) {
            res.sendRedirect(req.getContextPath() + "/ControlServlet?action=adminMenu");
        } else {
            res.sendRedirect(req.getContextPath() + "/ControlServlet?action=menu");
        }
    }
}
```

```
        return null;
    }
}
```

ここで新しく出てくるもの

- `public class LoginLogic implements Logic { ... }` … Day1で見た `Logic` インターフェースの**最初の実装例**。■用語集 → implements
- `if ("doLogin".equals(action)) { ... }` … 文字列の比較は `==` ではなく `.equals()` を使う。■用語集 → String.equals
- `req.getSession(true)` … `true` を渡すと「セッションが無ければ新しく作る」。ログイン成功時は必ず新しいセッションを発行する。
- `session.setAttribute("userId", account.getUserId())` など … ログイン中のユーザー情報をセッションに保存する。■用語集 → session.setAttribute
- `account.getAdmin() == 1` … `int` 型の `admin` フィールドが `1`（管理者）かどうかを判定。 `==` は数値の比較なのでOK（文字列の `.equals()` と混同しないよう注意）。
- `res.sendRedirect(req.getContextPath() + "/ControlServlet?action=adminMenu")` と `return null` … ■用語集 → sendRedirectとforwardの違い。ここでは「メニューの集計をやり直してもらうために、わざとリダイレクトしている」。

読み方のポイント

- `execute()` は最初に `action` を見て、`doLogin`（認証する）か `login`（画面を出すだけ）かを振り分ける。この「**actionで分岐する**」構造は、これから読むすべてのLogicクラスで繰り返し出てくる基本パターン。
 - 認証失敗時（`account == null`、または管理者ログインなのに権限がない）は、いずれも `loginNG.jsp` に `errorMsg` をセットして表示する。
 - 認証成功時は、`return "/WEB-INF/jsp/menu.jsp"` のように直接JSPを返すのではなく、`res.sendRedirect(...)` して `return null` している。これは「メニュー画面の表示には `ControlServlet.showMenu()` が用意する追加データが必要」という事情によるもの（Day6で `showMenu()` を読むときに、もう一度ここに戻ってくるとよい）。
-

3-2-3. dao/AccountsDAO.java (ユーザーアカウントのDB操作)

```
package dao;

import entity.Account;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

/**
 * ユーザーアカウントDAO (JDBC版)
 *
 * DB側の user_id は INT だが、画面・ロジック側では従来どおり String として扱う
 * (影響範囲を抑えるため、JDBCの境界でだけ int ⇄ String を変換する)。
 */
public class AccountsDAO {

    private Account map(ResultSet rs) throws SQLException {
        return new Account(
            String.valueOf(rs.getInt("user_id")),
            rs.getString("password"),
            rs.getString("name"),
            rs.getString("b_id"),
            rs.getInt("admin")
        );
    }

    /** "1001" のような数字文字列を user_id として解釈する (数値でなければ null=該当なし扱い) */
    private Integer parseUserId(String userId) {
        try {
            return Integer.valueOf(userId);
        } catch (NumberFormatException | NullPointerException e) {
            return null;
        }
    }

    public Account findById(String userId) {
        Integer id = parseUserId(userId);
        if (id == null) return null;

        String sql = "SELECT user_id, password, b_id, name, admin FROM users WHERE user_id = ?";
        try (Connection conn = DBUtil.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) return map(rs);
            }
        } catch (SQLException e) {
            throw new RuntimeException("アカウントの取得に失敗しました", e);
        }
        return null;
    }

    public Account authenticate(String userId, String password) {
        Integer id = parseUserId(userId);
        if (id == null) return null;

        String sql = "SELECT user_id, password, b_id, name, admin FROM users WHERE user_id = ? AND password = ?";
        try (Connection conn = DBUtil.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, id);
            ps.setString(2, password);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) return map(rs);
            }
        } catch (SQLException e) {
            throw new RuntimeException("ログイン認証に失敗しました", e);
        }
        return null;
    }
}
```

```

public List<Account> findAll() {
    String sql = "SELECT user_id, password, b_id, name, admin FROM users ORDER BY user_id";
    List<Account> list = new ArrayList<>();
    try (Connection conn = DBUtil.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {
        while (rs.next()) list.add(map(rs));
    } catch (SQLException e) {
        throw new RuntimeException("アカウント一覧の取得に失敗しました", e);
    }
    return list;
}

public List<Account> findByName(String keyword) {
    String sql = (keyword == null || keyword.isEmpty())
        ? "SELECT user_id, password, b_id, name, admin FROM users ORDER BY user_id"
        : "SELECT user_id, password, b_id, name, admin FROM users WHERE name LIKE ? ORDER BY user_id";
    List<Account> list = new ArrayList<>();
    try (Connection conn = DBUtil.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        if (keyword != null && !keyword.isEmpty()) {
            ps.setString(1, "%" + keyword + "%");
        }
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) list.add(map(rs));
        }
    } catch (SQLException e) {
        throw new RuntimeException("氏名検索に失敗しました", e);
    }
    return list;
}

/**
 * 部署 (bId) と社員ID (部分一致) の両方、またはどちらか一方で社員を絞り込む。
 * 条件が空のものは無視する (両方空ならWHERE 1=1のみ=全件)。
 */
public List<Account> search(String bId, String userIdKeyword) {
    StringBuilder sql = new StringBuilder(
        "SELECT user_id, password, b_id, name, admin FROM users WHERE 1=1");
    List<String> params = new ArrayList<>();
    if (bId != null && !bId.isEmpty()) {
        sql.append(" AND b_id = ?");
        params.add(bId);
    }
    if (userIdKeyword != null && !userIdKeyword.isEmpty()) {
        sql.append(" AND CAST(user_id AS CHAR) LIKE ?");
        params.add("%" + userIdKeyword + "%");
    }
    sql.append(" ORDER BY user_id");

    List<Account> list = new ArrayList<>();
    try (Connection conn = DBUtil.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql.toString())) {
        for (int i = 0; i < params.size(); i++) ps.setString(i + 1, params.get(i));
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) list.add(map(rs));
        }
    } catch (SQLException e) {
        throw new RuntimeException("社員検索に失敗しました", e);
    }
    return list;
}

public List<Account> findByBId(String bId) {
    String sql = "SELECT user_id, password, b_id, name, admin FROM users WHERE b_id = ? ORDER BY user_id";
    List<Account> list = new ArrayList<>();
    try (Connection conn = DBUtil.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, bId);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) list.add(map(rs));
        }
    }
}

```

```

    } catch (SQLException e) {
        throw new RuntimeException("部署メンバーの取得に失敗しました", e);
    }
    return list;
}
}

```

ここで新しく出てくるもの

- `private Account map(ResultSet rs) throws SQLException { ... }` … `ResultSet` (DBから取得した1行) を `Account` オブジェクトに変換する**専用メソッド**。同じ変換処理を何度も書かなくて済むように、最初にこれだけ用意しておく。■用語集 → `ResultSet`
- `String.valueOf(rs.getInt("user_id"))` … `int` を `String` に変換する。DBの `user_id` (INT型) を、Java側で扱う `String` の `userId` に変換している (クラスのJavadocで説明されている「境界での型変換」)。
- `try (Connection conn = DBUtil.getConnection(); PreparedStatement ps = conn.prepareStatement(sql)) { ... }` … ■用語集 → `try-with-resources`。 `{ }` の中の処理が終わったら、`conn` と `ps` が自動的に閉じられる。
- `ps.setInt(1, id) / ps.setString(2, password)` … ? に値をセットする。■用語集 → `ps.setInt/setString`
- `try (ResultSet rs = ps.executeQuery()) { if (rs.next()) return map(rs); }` … `SELECT` を実行し、最初の1行があれば `map()` で変換して返す。0件なら `rs.next()` が `false` になり、メソッドの最後の `return null;` に進む。
- `catch (NumberFormatException | NullPointerException e) … |` で2種類の例外をまとめてキャッチできる (Java 7以降の書き方)。`userId` が数字でない・`null` のいずれでも同じ扱い (該当なし) にする。
- `while (rs.next()) list.add(map(rs));` … ■用語集 → `while(rs.next())`。検索結果が複数行ある場合 (`findAll / findByName / search / findByBId`) は、行が無くなるまでループしてリストに追加する。
- `StringBuilder sql = new StringBuilder("...") + sql.append(" AND b_id = ?")` … ■用語集 → `StringBuilder`。検索条件 (部署・社員ID) が指定されているときだけ `AND` 句を追加し、SQL文を動的に組み立てる。
- `for (int i = 0; i < params.size(); i++) ps.setString(i + 1, params.get(i));` … 動的に追加した ? の数だけ、対応するパラメータを順番にセットするループ。

読み方のポイント

- `authenticate(userId, password)` が**ログイン認証の本体**。`WHERE user_id = ? AND password = ?` が両方一致する行が1件でもあれば、その `Account` を返す。1件も無ければ `null` (= ログイン失敗)。
- `search(bId, userIdKeyword)` は、Day5の部門検索で使われる。「部署だけで絞り込む」「社員IDだけで絞り込む」「両方で絞り込む」「両方指定なし (全件)」の**4パターンすべてに対応するため**、`StringBuilder` で動的にSQLを組み立てている。

3-2-4. webapp/WEB-INF/jsp/loginNG.jsp (ログイン失敗画面)

```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>ログイン失敗 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<div class="login-page">
  <div class="login-card">
    <div class="login-title">座・Navi</div>
    <div class="alert alert-danger" style="margin:16px 0;">
      <%= request.getAttribute("errorMsg") != null ? request.getAttribute("errorMsg") : "ログインに失敗しました。" %>
    </div>
    <a href="<%= request.getContextPath() %>/ControlServlet?action=login" class="btn btn-primary" style="width:100%; justify-content:center;">
      ログイン画面に戻る
    </a>
  </div>
</div>
</body>
</html>

```

ここで新しく出てくるもの

- `<%= 条件 ? 値A : 値B %>` … ■用語集 → 三項演算子。「`errorMsg` があればその内容、無ければ"ログインに失敗しました。"」を表示する。`LoginLogic` が `errorMsg` をセットしなかった場合でも、画面が空欄にならないようにする保険。

読み方のポイント

ごく短いJSP。LoginLogic.doLogin() が errorMsg をセットして return "/WEB-INF/jsp/loginNG.jsp"; した先がここ。「ログイン画面に戻る」リンクは action=login に戻るだけのシンプルなリンク。

3-2-5. model/LogoutLogic.java (ログアウト処理)

```
package model;

import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

/**
 * ログアウト処理。
 * セッション（ログイン情報を保持している入れ物）を破棄して、ログアウト完了画面を表示する。
 * これ以降、ログインし直さない限り action=login 以外のページへはアクセスできない
 * （ControlServlet.process() のセッションチェックで弾かれる）。
 */
public class LogoutLogic implements Logic {

    @Override
    public String execute(HttpServletRequest req, HttpServletResponse res) throws Exception {
        // false を渡しているのが、セッションが無ければ新規作成しない（無ければ何もしなくてよい）
        HttpSession session = req.getSession(false);
        if (session != null) {
            session.invalidate(); // セッション内の userId / isAdmin などすべて破棄する
        }
        return "/WEB-INF/jsp/logout.jsp";
    }
}
```

ここで新しく出てくるもの

- req.getSession(false) … LoginLogic の getSession(true) との違いに注目。false は「セッションが無ければ作らず null を返す」。すでにログアウト済み（セッションが無い）状態で誤って logout を呼んでもエラーにならないようにしている。
- session.invalidate() … 用語集 → session.invalidate()。これで userId ・ isAdmin などセッションに入っていた情報がすべて消える。

読み方のポイント

LoginLogic（ログイン＝セッション作成）と対になる、最も短いLogicクラス。「Logicを実装する」という型に沿っていれば、処理が3行でも10行でもControlServlet側からは同じように呼び出せる、という点を確認しておく。

3-2-6. webapp/WEB-INF/jsp/logout.jsp (ログアウト完了画面)

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>ログアウト - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<div class="login-page">
  <div class="login-card" style="text-align:center;">
    <div class="login-title">座・Navi</div>
    <div style="margin:24px 0; font-size:48px;">&#128075;</div>
    <p style="font-size:16px; font-weight:600; margin-bottom:8px;">ログアウトしました</p>
    <p style="font-size:13px; color:#757575; margin-bottom:28px;">ご利用ありがとうございました。</p>
    <a href="<%= request.getContextPath() %>/ControlServlet?action=login" class="btn btn-primary btn-lg" style="width:100%; justify-content:center;">
      ログイン画面へ
    </a>
  </div>
</div>
</body>
</html>
```

ここで新しく出てくるもの

新しい構文はなし。👋 は絵文字 (👉) をHTMLの文字実体参照で表したものの。

読み方のポイント

login.jsp ・ loginNG.jsp ・ logout.jsp の3つは、いずれも <%@ include %> でヘッダー (header.jspf) を読み込んでいない。これは「ログインしていない状態の画面には、ログイン中専用のヘッダー (メニューやログアウトボタン) を表示する必要がない」ため。次に見る header.jspf ・ headerAdmin.jspf は、ログイン後の画面 (menu.jsp 以降) でのみ使われる。

3-2-7. webapp/WEB-INF/jsp/common/header.jspf (一般ユーザー用ヘッダー)

```
<%@ page pageEncoding="UTF-8" %>
<!-- 一般ユーザー用ヘッダー (常にteal系の配色のまま。管理者がここに来た場合も「一般ユーザーとして見ている」と分かるように、わざと配色を切り替えない。管理者用の配色変更は headerAdmin.jspf 側だけで行う) -->
<header class="header">
  <div class="logo">座・Navi <span>座席管理システム</span></div>
  <div class="user-info">
    <details class="menu-dropdown">
      <summary>
        
        <%= session.getAttribute("userName") %> さん
        <span class="caret">&#9662;</span>
      </summary>
      <div class="dropdown-panel">
        <a href="<%= request.getContextPath() %>/ControlServlet?action=menu">メニュー</a>
        <% if (Boolean.TRUE.equals(session.getAttribute("isAdmin"))) { %>
          <a href="<%= request.getContextPath() %>/ControlServlet?action=adminMenu">管理メニューへ戻る</a>
        <% } %>
        <a href="<%= request.getContextPath() %>/ControlServlet?action=logout">ログアウト</a>
      </div>
    </details>
  </div>
</header>
```

ここで新しく出てくるもの

- <%@ include file="common/header.jspf" %> (このファイル自体ではなく、これを読み込む側に書かれる) … 他のJSPファイルの内容をそのまま挿入する命令。ヘッダーのような「共通部分」を1か所にまとめて、各画面から include する。 .jspf は「JSP Fragment (断片)」の意

味で、単体では完全なHTMLになっていない部品ファイル。

- `<details class="menu-dropdown"><summary>...</summary> ... </details>` … HTML標準の「クリックで開閉するメニュー」要素。JavaScriptを使わずにドロップダウンメニューを実現している。
- `Boolean.TRUE.equals(session.getAttribute("isAdmin"))` … `session.getAttribute(...)` の戻り値の型は `Object`（何が入っているかわからない型）なので、`Boolean.TRUE.equals(...)` のように比較する。`isAdmin` が `null`（未設定）のときも安全に `false` 扱いになる。

読み方のポイント

一般ユーザー用ヘッダーは常に同じ配色（ティール系）。ただし、管理者がいったん「一般メニュー」を見ているときは「管理メニューへ戻る」リンクが追加で表示される（`isAdmin` が `true` の場合のみ）。

3-2-8. webapp/WEB-INF/jsp/common/headerAdmin.jspf（管理者用ヘッダー）

```
<%@ page pageEncoding="UTF-8" %>
<!-- 管理者用ヘッダー -->
<style>
/* 管理者画面は常にインディゴ/ネイビー系の基調色にする（一般ユーザー画面のティール系と区別するため）。
   style.css の :root で定義された --teal 系の変数をここで上書きするだけで、
   ヘッダー・ボタン・バッジなど teal 変数を使っている箇所がまとめて変わる。 */
:root { --teal:#3949AB; --teal-light:#7986CB; --teal-dark:#283593; }
</style>
<header class="header">
  <div class="logo">座・Navi <span>座席管理システム</span><span class="admin-badge">管理者</span></div>
  <div class="user-info">
    <details class="menu-dropdown">
      <summary>
        
        <%= session.getAttribute("userName") %> さん
        <span class="caret">&#9662;</span>
      </summary>
      <div class="dropdown-panel">
        <a href="<%= request.getContextPath() %>/ControlServlet?action=adminMenu">管理メニュー</a>
        <a href="<%= request.getContextPath() %>/ControlServlet?action=menu">一般メニュー</a>
        <a href="<%= request.getContextPath() %>/ControlServlet?action=logout">ログアウト</a>
      </div>
    </details>
  </div>
</header>
```

ここで新しく出てくるもの

- `<style> :root { --teal:#3949AB; ... } </style>` … CSS変数（カスタムプロパティ）の**上書き**。`style.css` 本体で定義されている `--teal` 系の色を、このヘッダーを読み込んだページだけインディゴ系に変える。JavaやJSPの構文ではないが、「管理者画面だけ配色が違う」仕組みの正体がここにある。

読み方のポイント

`header.jspf` と `headerAdmin.jspf` はほぼ同じ構造（ロゴ+ドロップダウンメニュー）だが、

- `headerAdmin.jspf` には「管理者」バッジと、CSS変数を上書きする `<style>` がある
- メニューのリンク順が「管理メニュー → 一般メニュー → ログアウト」になっている

という2点が違う。どちらを `include` するかで、画面が一般ユーザー向けか管理者向けかが決まる。（`adminMenu.jsp` ・ `seatStatus.jsp` ・ `adminReserve.jsp` ・ `masterUpdate.jsp` は `headerAdmin.jspf` を使う）

Day2のまとめ・①解説書との関連

- Day2で読んだ「`login.jsp` → `LoginLogic` → `AccountsDAO` → セッションへ保存 → リダイレクト」という流れは、①解説書「6.『画面が動くまで』の具体例」で図解している内容そのもの。
- `header.jspf` / `headerAdmin.jspf` の配色切り替えは①解説書 7-6「CSS変数による管理者テーマ切り替え」と対応している。

- session に保存される `userId` / `userName` / `isAdmin` / `bId` の4つは、Day3以降のほぼ全てのLogicクラスで使われる。Day3に進む前に、もう一度この4つの役割を意識しておくといよい。

Day3: 行先登録（出社・在宅・出張）

Day3では「今日の行先（出社・在宅・出張）を登録する」機能を読む。ここで初めて `Map` ・ `LocalDate` ・ `switch` （複数case）といった、座・Navi全体で多用される構文が登場する。

3-3-1. webapp/WEB-INF/jsp/locationRegist.jsp (行先選択画面)

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>行先登録 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/header.jspf" %>

<%
String menuAction = Boolean.TRUE.equals(session.getAttribute("isAdmin")) ? "adminMenu" : "menu";
%>

<div class="container">
<div class="card">
<h2>行先登録</h2>
<p style="color:#757575; margin-bottom:24px;">本日の行先を選択してください。</p>

<div class="gyosaki-group">
<!-- 出社 -->
<form action="<%= request.getContextPath() %>/ControlServlet" method="post" style="flex:1; min-width:120px;">
<input type="hidden" name="action" value="doLocationRegist">
<input type="hidden" name="gyosaki" value="出社">
<button type="submit" class="gyosaki-btn office" style="width:100%;">

出社
</button>
</form>

<!-- 在宅（次の画面で任意のメモを入力） -->
<form action="<%= request.getContextPath() %>/ControlServlet" method="post" style="flex:1; min-width:120px;">
<input type="hidden" name="action" value="doLocationRegist">
<input type="hidden" name="gyosaki" value="在宅">
<button type="submit" class="gyosaki-btn remote" style="width:100%;">

在宅
</button>
</form>

<!-- 出張 -->
<form action="<%= request.getContextPath() %>/ControlServlet" method="post" style="flex:1; min-width:120px;">
<input type="hidden" name="action" value="doLocationRegist">
<input type="hidden" name="gyosaki" value="出張">
<button type="submit" class="gyosaki-btn trip" style="width:100%;">

出張
</button>
</form>
</div>

<div style="margin-top:24px;">
<a href="<%= request.getContextPath() %>/ControlServlet?action=<%= menuAction %>" class="btn btn-secondary">
&larr; メニューへ戻る
</a>
</div>
</div>
</div>
</body>
</html>
```

ここで新しく出てくるもの

新しい構文はなし。3つのボタンはそれぞれ別の `<form>` で、`gyosaki`（出社/在宅/出張）の値だけが違う。すべて `action=doLocationRegist` に送信される。

読み方のポイント

「出社」「在宅」「出張」という3つの選択肢が、**すべて同じ `action=doLocationRegist` に送られる**ことに注目する。分岐は画面側ではなく、これから読む `LocationRegistLogic` の中で `gyosaki` パラメータの値によって行われる。

3-3-2. model/LocationRegistLogic.java (行先登録の処理本体)

```
package model;

import dao.ReservationsDAO;
import dao.SeatsDAO;
import entity.Reservation;
import entity.Seat;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * 行先登録（出社・在宅・出張のいずれかを指定日の行先として登録する）。
 *
 * 「在宅」「出張」はそれぞれ専用のメモ入力画面を経由して登録する
 * （出張先とメモが混在して分かりにくならないよう、表示時にも区別できる形式で保存する）。
 * 対象日は座席予約と合わせて「今日から2週間先まで」選択できる。
 * 「出社」だけは特別で、ここでは登録せず座席予約画面（ReserveLogic）へリダイレクトし、
 * 座席が決まった時点で「出社＋座席」をまとめて1件登録する
 * （そうしないと「出社（座席なし）」と「出社（座席あり）」の2件ができてしまうため）。
 */
public class LocationRegistLogic implements Logic {

    /** 選択可能日数（今日を含めて2週間先まで＝15日分） */
    private static final int SELECTABLE_DAYS = 15;

    @Override
    public String execute(HttpServletRequest req, HttpServletResponse res) throws Exception {
        String action = req.getParameter("action");

        switch (action) {
            case "locationRegist":
                return "/WEB-INF/jsp/locationRegist.jsp";

            case "doLocationRegist":
                return doRegist(req, res);

            case "businessTrip":
                return showDateForm(req, "/WEB-INF/jsp/businessTrip.jsp");

            case "doBusinessTrip":
                return doBusinessTrip(req, res);

            case "remoteWork":
                return showDateForm(req, "/WEB-INF/jsp/remoteWork.jsp");

            case "doRemoteWork":
                return doRemoteWork(req, res);

            default:
                return "/WEB-INF/jsp/locationRegist.jsp";
        }
    }

    /** 今日から2週間先まで（座席予約の選択可能期間と合わせる） */
    private List<String> buildSelectableDates() {
        List<String> dates = new ArrayList<>();
        LocalDate d = LocalDate.now();
        for (int i = 0; i < SELECTABLE_DAYS; i++) {
            dates.add(d.plusDays(i).toString());
        }
        return dates;
    }
}
```

```

* 在宅・出張の対象日選択画面を表示する。
* 「すでに別の予定（出社の座席予約や、別の在宅・出張）が入っている日」が
* 選択肢の中で一目で分かるよう、日付ごとの予定状況をあわせて渡す
* （二重登録に気づかず選んでエラーになるのを防ぐため）。
*/
private String showDateForm(HttpServletRequest req, String jspPath) {
    HttpSession session = req.getSession(false);
    String userId = (String) session.getAttribute("userId");
    List<String> dates = buildSelectableDates();

    req.setAttribute("dates", dates);
    req.setAttribute("dateStatusLabels", buildDateStatusLabels(userId, dates));
    return jspPath;
}

/** 選択可能な各日付について、その人がすでに持っている有効な予定をラベル化して返す（予定がない日は含めない） */
private Map<String, String> buildDateStatusLabels(String userId, List<String> dates) {
    Map<String, String> labels = new LinkedHashMap<>();
    ReservationsDAO resDAO = new ReservationsDAO();
    SeatsDAO seatsDAO = new SeatsDAO();

    for (String date : dates) {
        Reservation r = resDAO.findActiveByUserAndDate(userId, date);
        if (r == null) continue;

        String label;
        if ("出社".equals(r.getGyosaki()) && r.getSeatId() > 0) {
            Seat seat = seatsDAO.findById(r.getSeatId());
            label = seat != null
                ? "予定あり：出社 (" + seat.getBaseName() + " " + seat.getSeatName() + ") "
                : "予定あり：出社";
        } else {
            label = "予定あり：" + r.getGyosaki();
        }
        labels.put(date, label);
    }
    return labels;
}

/** 画面から渡された日付が選択可能期間内であればそのまま使い、それ以外は今日にする */
private String resolveDate(String dateParam) {
    if (dateParam == null || dateParam.isEmpty()) return LocalDate.now().toString();
    return buildSelectableDates().contains(dateParam) ? dateParam : LocalDate.now().toString();
}

private String doRegist(HttpServletRequest req, HttpServletResponse res) throws Exception {
    String gyosaki = req.getParameter("gyosaki"); // 出社 / 在宅 / 出張
    if ("出張".equals(gyosaki)) {
        return showDateForm(req, "/WEB-INF/jsp/businessTrip.jsp");
    }
    if ("在宅".equals(gyosaki)) {
        return showDateForm(req, "/WEB-INF/jsp/remoteWork.jsp");
    }
    // 出社：ここでは登録せず座席予約画面（ReserveLogic）へリダイレクトし、
    // 座席が決まった時点で「出社＋座席」をまとめて1件登録する
    // （そうしないと「出社（座席なし）」と「出社（座席あり）」の2件ができてしまうため）
    res.sendRedirect(req.getContextPath() + "/ControlServlet?action=reserve");
    return null;
}

private String doRemoteWork(HttpServletRequest req, HttpServletResponse res) throws Exception {
    String date = resolveDate(req.getParameter("date"));
    String memo = req.getParameter("memo");
    return saveAndGoMenu(req, res, date, "在宅", memo != null ? memo : "");
}

private String doBusinessTrip(HttpServletRequest req, HttpServletResponse res) throws Exception {
    String date = resolveDate(req.getParameter("date"));
    String destination = req.getParameter("destination");
    String memo = req.getParameter("memo");
    // 出張先とメモを混ぜると後で見たとときにどちらか分からなくなるため、
    // 「出張先:oo/メモ:oo」の形式で保存し、表示時にも一目で区別できるようにする

```

```

        StringBuilder fullMemo = new StringBuilder("出張先:").append(destination != null ? destination : "");
        if (memo != null && !memo.isEmpty()) {
            fullMemo.append("/メモ:").append(memo);
        }
        return saveAndGoMenu(req, res, date, "出張", fullMemo.toString());
    }

    /** 登録を試みて、失敗した場合はエラーメッセージを返す（成功時はnull） */
    private String saveReservation(HttpServletRequest req, String date, String gyosaki, String memo) {
        HttpSession session = req.getSession(false);
        String userId = (String) session.getAttribute("userId");

        // すでにその日の有効な予約（出社・在宅・出張のいずれか）を持っている場合はDB制約違反になるため、
        // ここで先にチェックして分かりやすいメッセージを返す
        // （DB側のUNIQUE制約 uq_reservations_user_active が1人1日1件に制限している）
        if (new ReservationsDAO().findActiveByUserAndDate(userId, date) != null) {
            return date + " はすでに予約（出社・在宅・出張のいずれか）が登録されています。"
                + "先に「予約確認」から既存の予約を取り消してください。";
        }

        Reservation r = new Reservation();
        r.setUserId(userId);
        r.setRegisterId(userId);
        r.setReserveDate(date);
        r.setGyosaki(gyosaki);
        r.setMemo(memo);
        r.setSeatId(0);
        r.setCancelFlag(0);

        new ReservationsDAO().add(r);

        // メニューへ戻る前に本日の予約を更新
        session.setAttribute("todayReservations",
            new ReservationsDAO().findTodayByUserId(userId));
        return null;
    }

    private String saveAndGoMenu(HttpServletRequest req, HttpServletResponse res, String date, String gyosaki, String memo) throws Exception {
        String error = saveReservation(req, date, gyosaki, memo);
        if (error != null) {
            req.setAttribute("errorMsg", error);
            return "/WEB-INF/jsp/reserveNG.jsp";
        }

        HttpSession session = req.getSession(false);
        // リダイレクト後の画面で表示するため、リクエスト属性ではなくセッションに積んでおく
        session.setAttribute("successMsg", date + " の" + gyosaki + "登録が完了しました。");

        boolean isAdmin = Boolean.TRUE.equals(session.getAttribute("isAdmin"));
        res.sendRedirect(req.getContextPath() + "/ControlServlet?action=" + (isAdmin ? "adminMenu" : "menu"));
        return null;
    }
}

```

ここで新しく出てくるもの

- `switch (action) { case "locationRegist": return ...; case "doLocationRegist": return ...; ... default: return ...; } ...` ■ 新しい構文: **switch文**。 `if/else` をたくさん書く代わりに、「`action` の値ごとに処理を分ける」 ことを見やすく書ける。 `case` に当てはまらなければ `default` が実行される。 **ControlServletを含め、このあと出てくる全Logicクラスの `execute()` はほぼすべてこの形。**
- `private static final int SELECTABLE_DAYS = 15;` ... クラスの中で繰り返し使う定数。 `15` という数値が複数箇所に出てくるのを防ぐため、最初に名前を付けている。
- `List<String> dates = new ArrayList<>(); LocalDate d = LocalDate.now(); for (int i = 0; i < SELECTABLE_DAYS; i++) { dates.add(d.plusDays(i).toString()); } ...` ■ 用語集 → `LocalDate`。「今日から14日後まで」の日付文字列リストを作る、座・Navi全体で繰り返し使われる処理パターン。
- `Map<String, String> labels = new LinkedHashMap<>(); ...` ■ 用語集 → `Map / LinkedHashMap`。「日付文字列 → 予定の説明文」の対応表。`LinkedHashMap`なので、`dates` に追加した順序（日付の昇順）がそのまま保たれる。
- `for (String date : dates) { ... }` ... ■ 用語集 → `for-each`。`dates` リストの中身を1件ずつ取り出して処理する。

- `if (r == null) continue; ... continue` は「このループの残りをスキップして次のループへ進む」。予定が無い日（`r == null`）はラベルを作らずスキップする。
- `labels.put(date, label); ...` ■用語集 → `Map.put`。 `Map` にキーと値のペアを追加する。
- `buildSelectableDates().contains(dateParam) ... List.contains(値)` は「リストにその値が含まれているか」を `true / false` で返す。不正な日付（選択可能期間外）が送られてきても、ここで弾いて今日の日付にフォールバックする。
- `StringBuilder fullMemo = new StringBuilder("出張先:").append(...)` ... ■用語集 → `StringBuilder`。「出張先:○○/メモ:○○」という1つの文字列を組み立てている。

読み方のポイント（座・Navi最重要パターンの1つ）

- `execute()` の `switch` には、`gyosaki`（出社・在宅・出張）ごとに見える「画面の数」より多くの `case` がある。これは「1つの機能に対して、画面表示用のaction（例: `businessTrip`）と、フォーム送信用のaction（例: `doBusinessTrip`）がペアで存在する」という座・Navi全体の命名規則によるもの。`do` が付いているかどうかで「表示するだけ」か「処理を実行する」かを読み分けられる。
 - `doRegist()` での「出社」の扱いに注目。クラスのJavadocに書かれている通り、出社だけは**ここで登録せず**、`reserve`（座席予約）へリダイレクトする。これは、Day1で見た `Reservation` の `seatId` フィールドが「出社の場合は座席が決まらなと値が確定しない」ためで、Day4の `ReserveLogic` を読むとつながりがわかる。
 - `saveReservation()` の「先に `findActiveByUserAndDate()` でチェックしてからINSERTする」という流れは、①解説書のDB構成（UNIQUE制約 `uq_reservations_user_active`）と対応している。「**アプリ側でのチェック**」と「**DB側の制約**」が**二重に存在する**のは、エラーをユーザーにわかりやすく伝えるための工夫であることを意識する。
-

3-3-3. webapp/WEB-INF/jsp/businessTrip.jsp (出張先入力画面)

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.List" %>
<%@ page import="java.util.Map" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>出張先入力 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/header.jspf" %>

<div class="container">
  <div class="card" style="max-width:500px; margin:0 auto;">
    <h2>出張先を入力してください</h2>

    <%
      List<String> dates = (List<String>) request.getAttribute("dates");
      Map<String, String> dateStatusLabels = (Map<String, String>) request.getAttribute("dateStatusLabels");
    %>
    <form action="<%= request.getContextPath() %>/ControlServlet" method="post">
      <input type="hidden" name="action" value="doBusinessTrip">

      <div class="form-group">
        <label>対象日</label>
        <select name="date" style="width:100%;">
          <% if (dates != null) for (String d : dates) {
            String status = dateStatusLabels != null ? dateStatusLabels.get(d) : null;
          %>
          <option value="<%= d %>"><%= d %><%= status != null ? " - " + status : "" %></option>
          <% } %>
        </select>
        <p style="color:#757575; font-size:0.85em; margin-top:4px;">
          「予定あり」の日は、すでに会社・在宅・出張のいずれかの予定が登録されています（重複登録するとエラーになります）。
        </p>
      </div>

      <div class="form-group">
        <label>出張先</label>
        <input type="text" name="destination" placeholder="例: 大阪支社" required autofocus>
      </div>

      <div class="form-group">
        <label>メモ (任意) </label>
        <textarea name="memo" rows="2" placeholder="例: 午後から客先訪問のため帰社は夕方 など" style="width:100%;"></textarea>
      </div>

      <div style="display:flex; gap:10px; margin-top:8px;">
        <button type="submit" class="btn btn-primary" style="flex:1;">
          登録する
        </button>
        <a href="<%= request.getContextPath() %>/ControlServlet?action=locationRegist" class="btn btn-secondary">
          戻る
        </a>
      </div>
    </form>
  </div>
</div>
</body>
</html>
```

ここで新しく出てくるもの

- `(List<String>) request.getAttribute("dates")` … ■ 新しい構文：キャスト（型変換）。`getAttribute()` の戻り値は `Object` 型なので、`(List<String>)` を付けて「これは `List<String>` だ」と明示してから使う。Logic側で `req.setAttribute("dates", dates)` としていた `dates`（`LocationRegistLogic.buildSelectableDates()` の戻り値）がここで受け取られている。

- `<% if (dates != null) for (String d : dates) { ... } %>` … `if` と `for` を1つの `<% %>` の中に書き、複数行のHTML (`<option>`) をループで出力している。JSPでは、`<% %>` の「開き」と `<% } %>` の「閉じ」の間にHTMLを挟むことで、HTML部分を繰り返し出力できる。

読み方のポイント

`dateStatusLabels` (`LocationRegistLogic.buildDateStatusLabels()` で作られたMap) が、`<option>` の表示テキストに「 — 予定あり：出社 (三田 A-01)」のような形で反映されている。**Logic側で作った Map が、JSP側でどう使われるかの対応を確認する。**

3-3-4. webapp/WEB-INF/jsp/remotework.jsp (在宅メモ入力画面)

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.List" %>
<%@ page import="java.util.Map" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>在宅メモ入力 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/header.jspf" %>

<div class="container">
  <div class="card" style="max-width:500px; margin:0 auto;">
    <h2>在宅勤務として登録します</h2>
    <p style="color:#757575; margin-bottom:24px;">必要であれば、業務内容などのメモを残せます（任意）。</p>

    <%
      List<String> dates = (List<String>) request.getAttribute("dates");
      Map<String, String> dateStatusLabels = (Map<String, String>) request.getAttribute("dateStatusLabels");
    %>
    <form action="<%= request.getContextPath() %>/ControlServlet" method="post">
      <input type="hidden" name="action" value="doRemoteWork">

      <div class="form-group">
        <label>対象日</label>
        <select name="date" style="width:100%;">
          <% if (dates != null) for (String d : dates) {
            String status = dateStatusLabels != null ? dateStatusLabels.get(d) : null;
          %>
            <option value="<%= d %>"><%= d %><%= status != null ? " - " + status : "" %></option>
          <% } %>
        </select>
        <p style="color:#757575; font-size:0.85em; margin-top:4px;">
          「予定あり」の日は、すでに出社・在宅・出張のいずれかの予定が登録されています（重複登録するとエラーになります）。
        </p>
      </div>

      <div class="form-group">
        <label>メモ（任意）</label>
        <textarea name="memo" rows="2" placeholder="例：午後から商談対応のため在宅 など" style="width:100%;"></textarea>
      </div>

      <div style="display:flex; gap:10px; margin-top:8px;">
        <button type="submit" class="btn btn-primary" style="flex:1;">
          登録する
        </button>
        <a href="<%= request.getContextPath() %>/ControlServlet?action=locationRegist" class="btn btn-secondary">
          戻る
        </a>
      </div>
    </form>
  </div>
</div>
</body>
</html>
```

ここで新しく出てくるもの

新しい構文はなし。 `businessTrip.jsp` から「出張先」入力欄を取り除いただけの、ほぼ同じ構造。

読み方のポイント

`businessTrip.jsp` と `remoteWork.jsp` は、ほぼ同じ構造の画面を別ファイルとして用意している（共通化されていない）。これは「将来、在宅と出張で項目数が変わっても、互いに影響しないようにする」という考え方もできるし、「共通化できる余地がある」とも言える。チームで「ここは共通化すべきか？」を話し合ってみるのも良い学習になる。

3-3-5. `dao/LocationsDAO.java`（部署マスタのDB操作）

```
package dao;

import entity.Location;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

/**
 * 部署マスタDAO（JDBC版）。
 * ※クラス名・entity名は Location（場所）だが、扱っているのは「部署（基幹システム1部・営業システム本部など）」であり、
 * 座席の「拠点（三田・芝浦など）」とは別物なので注意（entity.Location のコメントも参照）。
 */
public class LocationsDAO {

    public List<Location> findAll() {
        String sql = "SELECT b_id, b_name FROM departments ORDER BY b_id";
        List<Location> list = new ArrayList<>();
        try (Connection conn = DBUtil.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                list.add(new Location(rs.getString("b_id"), rs.getString("b_name")));
            }
        } catch (SQLException e) {
            throw new RuntimeException("部署一覧の取得に失敗しました", e);
        }
        return list;
    }

    public Location findById(String bId) {
        String sql = "SELECT b_id, b_name FROM departments WHERE b_id = ?";
        try (Connection conn = DBUtil.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, bId);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return new Location(rs.getString("b_id"), rs.getString("b_name"));
                }
            }
        } catch (SQLException e) {
            throw new RuntimeException("部署の取得に失敗しました", e);
        }
        return null;
    }

    public String getBNameById(String bId) {
        Location loc = findById(bId);
        return loc != null ? loc.getBName() : "不明";
    }
}
```

ここで新しく出てくるもの

新しい構文はなし。Day2の `AccountsDAO` と同じパターン（`try-with-resources` + `PreparedStatement` + `ResultSet`）。

読み方のポイント

`getBNameById(bId)` は「部署IDから部署名を取得する」便利メソッド。内部では `findById(bId)` を呼び、結果が `null` なら "不明" という文字列を返す。**「DBに無いデータでも、画面の表示が崩れないようにする」**という工夫であり、Day6の `ControlServlet.showSeatStatus()` など

多用されている。

3-3-6. dao/ReservationsDAO.java (予約データのDB操作・座・Navi最重要DAO)

```
package dao;

import entity.Reservation;
import java.sql.Date;
import java.sql.*;
import java.time.LocalDate;
import java.util.*;

/**
 * 予約データDAO (JDBC版)
 *
 * DB側の user_id / register_id は INT だが、画面・ロジック側では従来どおり String として扱う
 * (影響範囲を抑えるため、JDBCの境界でだけ int ⇄ String を変換する)。
 * また、行先の種類 ("出社"/"在宅"/"出張") は entity 上は gyosaki だが、
 * DBのカラム名は place なので、ここでマッピングする。
 */
public class ReservationsDAO {

    private Reservation map(ResultSet rs) throws SQLException {
        return new Reservation(
            rs.getInt("reserve_id"),
            String.valueOf(rs.getInt("user_id")),
            String.valueOf(rs.getInt("register_id")),
            rs.getDate("reserve_date").toString(),
            rs.getString("place"),
            rs.getString("memo"),
            rs.getInt("seat_id"),
            rs.getInt("cancel_flag")
        );
    }

    private static final String SELECT_COLUMNS =
        "SELECT reserve_id, user_id, register_id, reserve_date, place, memo, seat_id, cancel_flag FROM reservations ";

    public List<Reservation> findByUserId(String userId) {
        String sql = SELECT_COLUMNS + "WHERE user_id = ? AND cancel_flag = 0 ORDER BY reserve_date";
        List<Reservation> list = new ArrayList<>();
        try (Connection conn = DBUtil.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, Integer.parseInt(userId));
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) list.add(map(rs));
            }
        } catch (SQLException e) {
            throw new RuntimeException("予約一覧の取得に失敗しました", e);
        }
        return list;
    }

    public List<Reservation> findTodayByUserId(String userId) {
        String sql = SELECT_COLUMNS + "WHERE user_id = ? AND reserve_date = ? AND cancel_flag = 0";
        List<Reservation> list = new ArrayList<>();
        try (Connection conn = DBUtil.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, Integer.parseInt(userId));
            ps.setDate(2, Date.valueOf(LocalDate.now()));
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) list.add(map(rs));
            }
        } catch (SQLException e) {
            throw new RuntimeException("本日の予約の取得に失敗しました", e);
        }
        return list;
    }

    public List<Reservation> findByDate(String date) {
        String sql = SELECT_COLUMNS + "WHERE reserve_date = ? AND cancel_flag = 0";
        List<Reservation> list = new ArrayList<>();
        try (Connection conn = DBUtil.getConnection();
```

```

        PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setDate(1, Date.valueOf(LocalDate.parse(date)));
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) list.add(map(rs));
            }
        } catch (SQLException e) {
            throw new RuntimeException("指定日の予約の取得に失敗しました", e);
        }
        return list;
    }
}

/**
 * 指定ユーザーが指定日に持っている有効な（取消されていない）予約を返す（なければnull）。
 * 1人が同じ日に複数の有効な予約を持てない（DB側のUNIQUE制約 uq_reservations_user_active）ため、
 * 登録前にここでチェックして、制約違反の生エラーではなく分かりやすいメッセージを出す。
 */
public Reservation findActiveByUserAndDate(String userId, String date) {
    String sql = SELECT_COLUMNS + "WHERE user_id = ? AND reserve_date = ? AND cancel_flag = 0";
    try (Connection conn = DBUtil.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, Integer.parseInt(userId));
        ps.setDate(2, Date.valueOf(LocalDate.parse(date)));
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return map(rs);
        }
    } catch (SQLException e) {
        throw new RuntimeException("予約状況の確認に失敗しました", e);
    }
    return null;
}

/**
 * 指定日のseatId→userId マップを返す（座席ステータス表示用）
 * key=seatId, value=userId
 */
public Map<Integer, String> getSeatUsageForDate(String date) {
    String sql = "SELECT seat_id, user_id FROM reservations "
        + "WHERE reserve_date = ? AND cancel_flag = 0 AND seat_id > 0";
    Map<Integer, String> map = new HashMap<>();
    try (Connection conn = DBUtil.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setDate(1, Date.valueOf(LocalDate.parse(date)));
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                map.put(rs.getInt("seat_id"), String.valueOf(rs.getInt("user_id")));
            }
        }
    } catch (SQLException e) {
        throw new RuntimeException("座席利用状況の取得に失敗しました", e);
    }
    return map;
}

public void add(Reservation r) {
    String sql = "INSERT INTO reservations (user_id, register_id, reserve_date, place, memo, seat_id, cancel_flag) "
        + "VALUES (?, ?, ?, ?, ?, ?, ?)";
    try (Connection conn = DBUtil.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
        ps.setInt(1, Integer.parseInt(r.getUserId()));
        ps.setInt(2, Integer.parseInt(r.getRegisterId()));
        ps.setDate(3, Date.valueOf(LocalDate.parse(r.getReserveDate())));
        ps.setString(4, r.getGyosaki());
        ps.setString(5, r.getMemo());
        ps.setInt(6, r.getSeatId());
        ps.setInt(7, r.getCancelFlag());
        ps.executeUpdate();
        try (ResultSet keys = ps.getGeneratedKeys()) {
            if (keys.next()) r.setReserveId(keys.getInt(1));
        }
    } catch (SQLException e) {
        throw new RuntimeException("予約の登録に失敗しました", e);
    }
}

```

```

    }

    public void cancel(int reserveId) {
        String sql = "UPDATE reservations SET cancel_flag = 1 WHERE reserve_id = ?";
        try (Connection conn = DBUtil.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, reserveId);
            ps.executeUpdate();
        } catch (SQLException e) {
            throw new RuntimeException("予約の取消に失敗しました", e);
        }
    }

    public Reservation findById(int reserveId) {
        String sql = SELECT_COLUMNS + "WHERE reserve_id = ?";
        try (Connection conn = DBUtil.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, reserveId);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) return map(rs);
            }
        } catch (SQLException e) {
            throw new RuntimeException("予約の取得に失敗しました", e);
        }
        return null;
    }
}

```

ここで新しく出てくるもの

- `private static final String SELECT_COLUMNS = "SELECT ... FROM reservations ";` … SQLの「SELECT 句」部分を**定数として共通化**している。`findById` / `findTodayByUserId` / `findByDate` / `findActiveByUserAndDate` / `findById` の5つのメソッドすべてが、この `SELECT_COLUMNS` の後ろに `WHERE ...` を付けて使っている。同じSQLを5回書くより、列名の変更が1か所で済む。
- `rs.getDate("reserve_date").toString()` … DBの `DATE` 型を `java.sql.Date` として取得し、`.toString()` で `"2026-06-13"` のような文字列に変換する。
- `ps.setDate(2, Date.valueOf(LocalDate.now())) / Date.valueOf(LocalDate.parse(date))` … `java.time.LocalDate` (Java側の日付) と `java.sql.Date` (JDBC側の日付) を変換している。`LocalDate.parse("2026-06-13")` で文字列から `LocalDate` に変換、`Date.valueOf(...)` で `java.sql.Date` に変換、という2段階。
- `try (Connection conn = DBUtil.getConnection(); PreparedStatement ps = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) { ... }` … **用語集** → `Statement.RETURN_GENERATED_KEYS`。`INSERT` 実行後に自動採番された `reserve_id` を取得するためのオプション。
- `try (ResultSet keys = ps.getGeneratedKeys()) { if (keys.next()) r.setReserveId(keys.getInt(1)); }` … `INSERT` で自動採番された値を取得し、引数で受け取った `Reservation r` のフィールドに設定する。**呼び出し元の `Reservation` オブジェクトが、メソッドの中で書き換えられる** (Javaのオブジェクトは参照で渡されるため) という点に注目。
- `Map<Integer, String> map = new HashMap<>();` … **用語集** → `HashMap`。`getSeatUsageForDate()` は「座席ID → ユーザーID」の対応表を返す。Day4・Day6で「この席は誰が使っているか」を調べるときに使われる。

読み方のポイント (このファイルが最重要な理由)

- `ReservationsDAO` は、Day3～Day6で読むほぼ全てのLogicクラス (`LocationRegistLogic`、`ReserveLogic`、`ReserveConfirmLogic`、`AdminReserveLogic`、`ControlServlet`) から呼ばれる。
- 特に `findActiveByUserAndDate()` (二重登録防止) と `getSeatUsageForDate()` (座席の空き状況) の2つは、座・Naviの「予約の整合性」を支える中核メソッド。この2つの**使われ方**を意識しながらDay4以降を読むと理解が早まる。
- `add()` の `Statement.RETURN_GENERATED_KEYS` の仕組みは、Day7の `MastersDAO.add()` (座席マスタへの追加。こちらは `AUTO_INCREMENT` を使わず手動で連番を計算する別方式) と対比すると面白い。

Day3のまとめ・①解説書との関連

- `LocationRegistLogic` の「出社だけは登録せず `reserve` ヘリダイレクトする」という分岐は、①解説書「2. 画面一覧・画面遷移表」のアクション遷移表で確認できる。
- `ReservationsDAO` の `SELECT_COLUMNS`、`gyosaki` ↔ `place` のマッピングは、①解説書「4. データベース構成」の `reservations` テーブル定義と対応している。

- Day3で読んだ `Map<String,String>`（日付→ラベル）のパターンは、Day4のReserveLogicで `Map<Integer,String>`（座席ID→ユーザーID）として再び出てくる。「キーと値が何を表しているか」を意識する習慣をつけておくと、Day4がスムーズになる。

Day4: 座席予約（座・Naviの中心機能）

Day4では「拠点→フロア→エリア→座席」の4段階で座席を選んで予約する、座・Naviで最も画面数の多い機能を読む。`SeatsDAO`は座席に関する情報を一手に引き受ける重要なDAOで、Day6（座席利用状況）・Day7（座席マスタ管理）でも繰り返し登場する。

3-4-1. dao/SeatsDAO.java (座席データのDB操作・画像対応表)

```
package dao;

import entity.Seat;
import java.sql.*;
import java.util.*;

/**
 * 座席データアクセスオブジェクト (JDBC版)
 */
public class SeatsDAO {

    /**
     * 拠点 (オフィス) ・フロア → フロアマップ画像ファイル名の対応表。
     * 座席利用状況・代理予約・座席予約の各画面で、選んだ拠点 (とフロア) に応じた
     * 画像を切り替えて表示するために使う。
     *
     * キーは「拠点名」または「拠点名_フロア名」。
     * フロアごとに画像が異なる場合は "拠点名_フロア名" で個別に登録すると、
     * getOfficeImage(拠点, フロア) がそちらを優先して返す
     * (例: 芝浦の1階と2階で画像が違う → "芝浦_1F" "芝浦_2F" をそれぞれ登録)。
     * フロアで画像が変わらない拠点は「拠点名」のみのキーで登録すればよい。
     *
     * 画像が用意できていない拠点・フロアはここに追加しない
     * → getOfficeImage() が null を返し、JSP側は画像なしで表示する。
     *
     * 新しいオフィス (やフロア) の画像を追加したときは、ここにも対応を追記すること。
     */
    static final Map<String, String> OFFICE_IMAGES = new LinkedHashMap<>();
    static {
        // フロアごとの専用マップ (1Fと2Fでレイアウトが異なるため、フロアを選んだ時点で
        // 該当フロアの画像に切り替わる。代理予約・座席利用状況・座席予約・マスター更新の
        // どの画面でも、拠点とフロアの両方が決まった時点でこちらが優先表示される)
        OFFICE_IMAGES.put("三田_1F", "office_mita_1F.png");
        OFFICE_IMAGES.put("三田_2F", "office_mita_2F.png");
        OFFICE_IMAGES.put("芝浦_1F", "office_shibaura_1F.png");
        OFFICE_IMAGES.put("芝浦_2F", "office_shibaura_2F.png");
        OFFICE_IMAGES.put("中野_1F", "office_nakano_1F.png");
        OFFICE_IMAGES.put("中野_2F", "office_nakano_2F.png");

        // 拠点単位のフォールバック画像 (フロアが未選択、またはそのフロア専用の画像が
        // まだ無い場合に使われる。例: 新しいフロアを増設したがまだ画像を用意していない時)
        OFFICE_IMAGES.put("三田", "office_mita.png");
        OFFICE_IMAGES.put("芝浦", "office_shibaura.png");
        OFFICE_IMAGES.put("中野", "office_nakano.png");

        // エリア専用画像 (座席選択画面でエリアを選んだときに表示)
        // ファイルが images/ フォルダに存在しない場合は getAreaImage() が null を返し、
        // JSP 側は動的グリッド (緑/赤の空き状況マップ) にフォールバックする
        // ファイル名はフロア画像 (office_mita_1F.png 等) と同じくローマ字の命名規則で統一する。
        // 命名規則: office_{拠点ローマ字}_{フロア}_{エリア}.png (例: office_mita_1F_A.png)
        OFFICE_IMAGES.put("三田_1F_A", "office_mita_1F_A.png");
        OFFICE_IMAGES.put("三田_1F_B", "office_mita_1F_B.png");
        OFFICE_IMAGES.put("三田_1F_C", "office_mita_1F_C.png");
        OFFICE_IMAGES.put("三田_2F_A", "office_mita_2F_A.png");
        OFFICE_IMAGES.put("三田_2F_B", "office_mita_2F_B.png");
        OFFICE_IMAGES.put("三田_2F_C", "office_mita_2F_C.png");
        OFFICE_IMAGES.put("芝浦_1F_A", "office_shibaura_1F_A.png");
        OFFICE_IMAGES.put("芝浦_1F_B", "office_shibaura_1F_B.png");
        OFFICE_IMAGES.put("芝浦_1F_C", "office_shibaura_1F_C.png");
        OFFICE_IMAGES.put("芝浦_2F_A", "office_shibaura_2F_A.png");
        OFFICE_IMAGES.put("芝浦_2F_B", "office_shibaura_2F_B.png");
        OFFICE_IMAGES.put("芝浦_2F_C", "office_shibaura_2F_C.png");
        OFFICE_IMAGES.put("中野_1F_A", "office_nakano_1F_A.png");
        OFFICE_IMAGES.put("中野_1F_B", "office_nakano_1F_B.png");
        OFFICE_IMAGES.put("中野_1F_C", "office_nakano_1F_C.png");
        OFFICE_IMAGES.put("中野_2F_A", "office_nakano_2F_A.png");
        OFFICE_IMAGES.put("中野_2F_B", "office_nakano_2F_B.png");
        OFFICE_IMAGES.put("中野_2F_C", "office_nakano_2F_C.png");
    }
}
```



```

/** 拠点名に対応するフロアマップ画像ファイル名を返す（未登録の拠点はnull） */
public String getOfficeImage(String baseName) {
    return getOfficeImage(baseName, null);
}

/**
 * 拠点名・フロア名に対応するフロアマップ画像ファイル名を返す。
 * "拠点名_フロア名" の専用画像が登録されていればそれを優先し、
 * 無ければ「拠点名のみ」の画像にフォールバックする（どちらも無ければ null）。
 */
public String getOfficeImage(String baseName, String floorName) {
    if (baseName == null || baseName.isEmpty()) return null;
    if (floorName != null && !floorName.isEmpty()) {
        String perFloor = OFFICE_IMAGES.get(baseName + "_" + floorName);
        if (perFloor != null) return perFloor;
    }
    return OFFICE_IMAGES.get(baseName);
}

/**
 * エリア専用のフロアマップ画像ファイル名を返す。
 * "拠点名_フロア名_エリア名" キーが登録されていればそのファイル名を、
 * 無ければ null を返す（フロア/拠点へのフォールバックはしない）。
 * null の場合、JSP 側は動的グリッドにフォールバックする。
 */
public String getAreaImage(String baseName, String floorName, String areaName) {
    if (baseName == null || floorName == null || areaName == null) return null;
    return OFFICE_IMAGES.get(baseName + "_" + floorName + "_" + areaName);
}

/**
 * フロアマップの「期待されるファイル名」を返す。常に非null（baseName が非空のとき）。
 * OFFICE_IMAGES に登録済みならその名、未登録（新規追加オフィス等）なら
 * "office_{拠点}_{フロア}.png" を自動生成して返す。
 * JSP側は常にこのファイル名で <img> を生成し、ファイルが存在しない場合は
 * onerror でプレースホルダーを表示する。
 */
public String getExpectedFloorImage(String baseName, String floorName) {
    if (baseName == null || baseName.isEmpty()) return null;
    String registered = getOfficeImage(baseName, floorName);
    if (registered != null) return registered;
    if (floorName != null && !floorName.isEmpty()) {
        return "office_" + baseName + "_" + floorName + ".png";
    }
    return "office_" + baseName + ".png";
}

/**
 * エリアマップの「期待されるファイル名」を返す。常に非null（全引数が非空のとき）。
 * OFFICE_IMAGES に登録済みならその名、未登録なら "office_{拠点}_{フロア}_{エリア}.png" を自動生成。
 */
public String getExpectedAreaImage(String baseName, String floorName, String areaName) {
    if (baseName == null || floorName == null || areaName == null) return null;
    String registered = getAreaImage(baseName, floorName, areaName);
    if (registered != null) return registered;
    return "office_" + baseName + "_" + floorName + "_" + areaName + ".png";
}

private Seat map(ResultSet rs) throws SQLException {
    return new Seat(
        rs.getInt("seat_id"),
        rs.getString("base_name"),
        rs.getString("f_name"),
        rs.getString("area_name"),
        rs.getString("seat_name")
    );
}

public List<Seat> getAllSeats() {
    String sql = "SELECT seat_id, base_name, area_name, f_name, seat_name FROM seats ORDER BY seat_id";
    List<Seat> list = new ArrayList<>();

```

```

        try (Connection conn = DBUtil.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {
            while (rs.next()) list.add(map(rs));
        } catch (SQLException e) {
            throw new RuntimeException("座席一覧の取得に失敗しました", e);
        }
        return list;
    }

    public Seat findById(int seatId) {
        String sql = "SELECT seat_id, base_name, area_name, f_name, seat_name FROM seats WHERE seat_id = ?";
        try (Connection conn = DBUtil.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, seatId);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) return map(rs);
            }
        } catch (SQLException e) {
            throw new RuntimeException("座席の取得に失敗しました", e);
        }
        return null;
    }

    public List<Seat> findByBase(String baseName) {
        return filterSeats(baseName, null, null);
    }

    public List<Seat> findByFloor(String baseName, String fName) {
        return filterSeats(baseName, fName, null);
    }

    public List<Seat> findByArea(String baseName, String fName, String areaName) {
        return filterSeats(baseName, fName, areaName);
    }

    /** 絞り込み検索 (nullや空文字は条件なし扱い) */
    public List<Seat> filterSeats(String baseName, String fName, String areaName) {
        StringBuilder sql = new StringBuilder(
            "SELECT seat_id, base_name, area_name, f_name, seat_name FROM seats WHERE 1=1");
        List<String> params = new ArrayList<>();
        if (baseName != null && !baseName.isEmpty()) { sql.append(" AND base_name = ?"); params.add(baseName); }
        if (fName != null && !fName.isEmpty()) { sql.append(" AND f_name = ?"); params.add(fName); }
        if (areaName != null && !areaName.isEmpty()) { sql.append(" AND area_name = ?"); params.add(areaName); }
        sql.append(" ORDER BY seat_id");

        List<Seat> list = new ArrayList<>();
        try (Connection conn = DBUtil.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql.toString())) {
            for (int i = 0; i < params.size(); i++) ps.setString(i + 1, params.get(i));
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) list.add(map(rs));
            }
        } catch (SQLException e) {
            throw new RuntimeException("座席の絞り込みに失敗しました", e);
        }
        return list;
    }

    /** 拠点一覧 (重複除去) */
    public List<String> getBases() {
        return distinctColumn("base_name", null, null, null);
    }

    /** 指定拠点のフロア一覧 */
    public List<String> getFloors(String baseName) {
        return distinctColumn("f_name", baseName, null, null);
    }

    /** 指定拠点・フロアのエリア一覧 */
    public List<String> getAreas(String baseName, String fName) {
        return distinctColumn("area_name", baseName, fName, null);
    }

```

```

    }

    private List<String> distinctColumn(String column, String baseName, String fName, String areaName) {
        StringBuilder sql = new StringBuilder("SELECT DISTINCT ").append(column).append(" FROM seats WHERE 1=1");
        List<String> params = new ArrayList<>();
        if (baseName != null && !baseName.isEmpty()) { sql.append(" AND base_name = ?"); params.add(baseName); }
        if (fName != null && !fName.isEmpty()) { sql.append(" AND f_name = ?"); params.add(fName); }
        if (areaName != null && !areaName.isEmpty()) { sql.append(" AND area_name = ?"); params.add(areaName); }
        sql.append(" ORDER BY ").append(column);

        List<String> list = new ArrayList<>();
        try (Connection conn = DBUtil.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql.toString())) {
            for (int i = 0; i < params.size(); i++) ps.setString(i + 1, params.get(i));
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) list.add(rs.getString(1));
            }
        } catch (SQLException e) {
            throw new RuntimeException("座席候補一覧の取得に失敗しました", e);
        }
        return list;
    }
}

```

ここで新しく出てくるもの

- `static final Map<String, String> OFFICE_IMAGES = new LinkedHashMap<>(); static { OFFICE_IMAGES.put(...); ... }` … 用語集 → `static変数` / `static初期化ブロック` (DBUtilのDay1で既出)。ここでは「拠点・フロア・エリア名 → 画像ファイル名」の固定の対応表として使われている。
- `getOfficeImage()` → `getExpectedFloorImage()` のような「同じ系統で名前の似たメソッドが複数ある」パターン。`getooImage()` 系は「登録されていなければnullを返す」、`getExpectedooImage()` 系は「登録されていなくても規則的なファイル名を自動生成して返す (必ず非null)」という違いがある。JSPはこの違いを使い分けて、画像が無い場合のプレースホルダー表示を実現している。
- `String barClass = rate < 50 ? "" : rate < 80 ? "med" : rate < 100 ? "high" : "full";` (後述のJSP内) … 用語集 → 三項演算子のネスト。`a ? b : c ? d : e` のように複数連結すると、`if/else if/else` を1行で表現できる。読みにくくなりやすいので、慣れるまでは括弧を補って読むとよい：`rate < 50 ? "" : (rate < 80 ? "med" : (rate < 100 ? "high" : "full"))`。

読み方のポイント

- `filterSeats()` と `distinctColumn()` は、どちらも「`StringBuilder` でSQL文を組み立て、`params` リストに条件値を集めて後から `ps.setString()` でまとめてセットする」という同じパターン。`findByBase` / `findByFloor` / `findByArea` / `getBases` / `getFloors` / `getAreas` は全てこの2つのラッパー。
- `OFFICE_IMAGES` は「フロアマップ画像をどのファイル名で表示するか」というUI的な関心事を、DAO (DB層) に置いている点はやや珍しい設計。「画像のファイル名規則」を1か所に集約するため、という意図がコメントから読み取れる。

3-4-2. model/ReserveLogic.java (座席予約の処理本体・4段階ウィザード)

```
package model;

import dao.AccountsDAO;
import dao.LocationsDAO;
import dao.ReservationsDAO;
import dao.SeatsDAO;
import entity.Account;
import entity.Reservation;
import entity.Seat;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.time.LocalDate;
import java.util.*;

public class ReserveLogic implements Logic {

    @Override
    public String execute(HttpServletRequest req, HttpServletResponse res) throws Exception {
        String action = req.getParameter("action");

        if ("doReserve".equals(action)) {
            return doReserve(req);
        }
        // action=reserve: stage で段階表示を切り替える
        return showReserve(req);
    }

    // ----- 座席選択フロー -----

    private String showReserve(HttpServletRequest req) {
        HttpSession session = req.getSession(false);
        SeatsDAO seatsDAO = new SeatsDAO();
        ReservationsDAO resDAO = new ReservationsDAO();
        String today = LocalDate.now().toString();

        String stage = req.getParameter("stage");
        if (stage == null || stage.isEmpty()) stage = "base";

        // パラメータをセッションへ蓄積
        String base = req.getParameter("base");
        String floor = req.getParameter("floor");
        String area = req.getParameter("area");
        String date = req.getParameter("date");

        if (base != null && !base.isEmpty()) session.setAttribute("resBase", base);
        if (floor != null && !floor.isEmpty()) session.setAttribute("resFloor", floor);
        if (area != null && !area.isEmpty()) session.setAttribute("resArea", area);

        // セッションから取得
        String sBase = (String) session.getAttribute("resBase");
        String sFloor = (String) session.getAttribute("resFloor");
        String sArea = (String) session.getAttribute("resArea");

        req.setAttribute("stage", stage);
        req.setAttribute("selBase", sBase);
        req.setAttribute("selFloor", sFloor);
        req.setAttribute("selArea", sArea);

        // フロアマップ画像：拠点とフロアの両方が決まる「エリア選択」以降の画面で表示。
        // 登録済み画像は実ファイル名、未登録（新規追加オフィス等）は命名規則から自動生成。
        // ファイルが存在しない場合は JSP 側の onerror でプレースホルダーを表示する。
        if (("area".equals(stage) || "seat".equals(stage)) && sBase != null && sFloor != null) {
            req.setAttribute("officeImage", seatsDAO.getExpectedFloorImage(sBase, sFloor));
        }

        // 今日の座席利用マップ（埋有率計算用）
        Map<Integer, String> todayUsage = resDAO.getSeatUsageForDate(today);
    }
}
```

```

switch (stage) {
    case "base": {
        // 拠点ごとの埋有率
        // 分母：その拠点の全席数（全フロア・全エリア合計）
        List<Map<String, Object>> items = new ArrayList<>();
        for (String b : seatsDAO.getBases()) {
            List<Seat> baseSeats = seatsDAO.findByBase(b);
            long used = baseSeats.stream().filter(s -> todayUsage.containsKey(s.getSeatId())).count();
            int total = baseSeats.size();
            int rate = total > 0 ? (int)(used * 100 / total) : 0;
            Map<String, Object> m = new HashMap<>();
            m.put("name", b); m.put("total", total); m.put("used", used); m.put("rate", rate);
            items.add(m);
        }
        req.setAttribute("items", items);
        break;
    }
    case "floor": {
        // フロアごとの埋有率
        // 分母：そのフロアの全席数（全エリア合計）
        List<Map<String, Object>> items = new ArrayList<>();
        for (String f : seatsDAO.getFloors(sBase)) {
            List<Seat> floorSeats = seatsDAO.findByFloor(sBase, f);
            long used = floorSeats.stream().filter(s -> todayUsage.containsKey(s.getSeatId())).count();
            int total = floorSeats.size();
            int rate = total > 0 ? (int)(used * 100 / total) : 0;
            Map<String, Object> m = new HashMap<>();
            m.put("name", f); m.put("total", total); m.put("used", used); m.put("rate", rate);
            items.add(m);
        }
        req.setAttribute("items", items);
        break;
    }
    case "area": {
        // エリアごとの埋有率
        // 分母：そのエリアの席のみ
        List<Map<String, Object>> items = new ArrayList<>();
        for (String a : seatsDAO.getAreas(sBase, sFloor)) {
            List<Seat> areaSeats = seatsDAO.findByArea(sBase, sFloor, a);
            long used = areaSeats.stream().filter(s -> todayUsage.containsKey(s.getSeatId())).count();
            int total = areaSeats.size();
            int rate = total > 0 ? (int)(used * 100 / total) : 0;
            Map<String, Object> m = new HashMap<>();
            m.put("name", a); m.put("total", total); m.put("used", used); m.put("rate", rate);
            items.add(m);
        }
        req.setAttribute("items", items);
        break;
    }
    case "seat": {
        // 日付プルダウン（予約できるのは今日から2週間先まで）
        List<String> dates = new ArrayList<>();
        LocalDate d = LocalDate.now();
        for (int i = 0; i < 15; i++) {
            dates.add(d.plusDays(i).toString());
        }
        req.setAttribute("dates", dates);

        // 選択日の座席利用状況
        String selectedDate = (date != null && !date.isEmpty()) ? date : today;
        req.setAttribute("selectedDate", selectedDate);
        session.setAttribute("resDate", selectedDate);

        Map<Integer, String> usage = resDAO.getSeatUsageForDate(selectedDate);
        List<Seat> areaSeats = seatsDAO.findByArea(sBase, sFloor, sArea);

        // 使用中の座席にカーソルを合わせたときに、予約者の氏名・部署名が
        // ひと目で分かるように（座席ボタンには userId しか入っていないため変換する）
        AccountsDAO accDAO = new AccountsDAO();
        LocationsDAO locDAO = new LocationsDAO();
        Map<Integer, String> seatUserNameMap = new HashMap<>();
        Map<Integer, String> seatUserDeptMap = new HashMap<>();
    }
}

```

```

        for (Map.Entry<Integer, String> e : usage.entrySet()) {
            Account acc = accDAO.findById(e.getValue());
            seatUserNameMap.put(e.getKey(), acc != null ? acc.getName() : e.getValue());
            seatUserDeptMap.put(e.getKey(), acc != null ? locDAO.getBNameById(acc.getBId()) : "");
        }

        // エリア埋有率（選択日）
        long usedCount = areaSeats.stream().filter(s -> usage.containsKey(s.getSeatId())).count();
        int total = areaSeats.size();
        int rate = total > 0 ? (int)(usedCount * 100 / total) : 0;

        // エリアマップ画像（登録済みは実ファイル名、未登録は命名規則から自動生成）
        // ファイルが存在しない場合は JSP 側の onerror でブレースホルダーを表示する
        req.setAttribute("areaImage", seatsDAO.getExpectedAreaImage(sBase, sFloor, sArea));

        req.setAttribute("areaSeats", areaSeats);
        req.setAttribute("seatUsage", usage);
        req.setAttribute("seatUserNameMap", seatUserNameMap);
        req.setAttribute("seatUserDeptMap", seatUserDeptMap);
        req.setAttribute("areaRate", rate);
        req.setAttribute("areaUsed", usedCount);
        req.setAttribute("areaTotal", total);
        break;
    }
}
return "/WEB-INF/jsp/reserve.jsp";
}

// ----- 予約実行 -----

private String doReserve(HttpServletRequest req) {
    HttpSession session = req.getSession(false);
    String userId = (String) session.getAttribute("userId");
    String seatIdStr = req.getParameter("seatId");
    String date = req.getParameter("date");
    String memo = req.getParameter("memo");
    if (date == null || date.isEmpty()) date = LocalDate.now().toString();

    if (seatIdStr == null) {
        req.setAttribute("errorMsg", "座席が選択されていません。");
        return "/WEB-INF/jsp/reserveNG.jsp";
    }
    int seatId = Integer.parseInt(seatIdStr);

    ReservationsDAO resDAO = new ReservationsDAO();

    // 同じ日にすでに有効な予約（出社・在宅・出張のいずれか）を持っていないか確認
    // （DB側のUNIQUE制約 uq_reservations_user_active が1人1日1件に制限しているため、
    // ここでチェックせずINSERTすると制約違反の生エラーがそのまま「システムエラー」になってしまう）
    if (resDAO.findActiveByUserAndDate(userId, date) != null) {
        req.setAttribute("errorMsg", "その日はすでに予約（出社・在宅・出張のいずれか）が登録されています。"
            + "先に「予約確認」から既存の予約を取り消してください。");
        return "/WEB-INF/jsp/reserveNG.jsp";
    }

    // 同一日・同一席に既存予約がないか確認
    Map<Integer, String> usage = resDAO.getSeatUsageForDate(date);
    if (usage.containsKey(seatId)) {
        req.setAttribute("errorMsg", "選択した座席はすでに予約済みです。");
        return "/WEB-INF/jsp/reserveNG.jsp";
    }

    Reservation r = new Reservation();
    r.setUserId(userId);
    r.setRegisterId(userId);
    r.setReserveDate(date);
    r.setGyosaki("出社");
    r.setSeatId(seatId);
    r.setMemo(memo != null && !memo.isEmpty() ? memo : null);
    r.setCancelFlag(0);
    resDAO.add(r);
}

```

```

// 座席選択の中間値をクリア
session.removeAttribute("resBase");
session.removeAttribute("resFloor");
session.removeAttribute("resArea");
session.removeAttribute("resDate");

Seat seat = new SeatsDAO().findById(seatId);
req.setAttribute("reservation", r);
req.setAttribute("seat", seat);
return "/WEB-INF/jsp/reserveOK.jsp";
}
}

```

ここで新しく出てくるもの

- `for (String b : seatsDAO.getBases()) { ... }` の中で `baseSeats.stream().filter(s -> todayUsage.containsKey(s.getSeatId())).count()` … 用語集 → Stream API (`.stream()` / `.filter()` / `.count()`)。「`baseSeats` (その拠点の座席リスト) の中から、`todayUsage` (今日使われている座席ID→ユーザーID) に含まれるものだけを数える」=「今日その拠点で使用中の座席数」を1行で計算している。
 - `.filter(s -> 条件)` … リストの中から条件に合う要素だけを残す。
 - `.count()` … 残った要素の数を `long` 型で返す。
 - `s -> todayUsage.containsKey(s.getSeatId())` … 用語集 → ラムダ式。「`s` という座席を受け取って、`todayUsage` にその`seatId`が含まれているか `true` / `false` を返す」という小さな関数をその場で書いている。
- `int rate = total > 0 ? (int)(used * 100 / total) : 0;` … 三項演算子 + キャスト。`total` が0 (座席が1つも無い) 場合に「0で割る」エラーを避けつつ、それ以外は `使用数×100÷総数` で埋有率 (%) を計算している。`(int)` は `long` の計算結果を `int` 型に変換するキャスト。
- `for (Map.Entry<Integer, String> e : usage.entrySet()) { ... e.getKey() ... e.getValue() ... }` … 用語集 → `Map.Entry` / `entrySet()` (Day1のControlServletで既出)。`usage` (座席ID→ユーザーID) を1件ずつ取り出し、`AccountsDAO` で氏名・部署名に変換して別の `Map` に詰め替えている。「IDのままでは画面が分かりにくいので、表示用の情報に変換する」という、座・Navi全体で繰り返されるパターン。

読み方のポイント (4段階ウィザードの仕組み)

- `showReserve()` は `stage` パラメータ (`base` / `floor` / `area` / `seat`) によって1つのメソッドの中で4種類の画面データを作り分けている。`switch(stage)` の各 `case` が、それぞれ「次にどんな `items` (埋有率リスト) を画面に渡すか」を決めている。
- 選んだ拠点・フロア・エリアはリクエストパラメータではなくセッション (`session.setAttribute("resBase", ...)` 等) に保存される。これにより、「フロアを選んだ後にブラウザの戻るボタンを押しても、拠点の選択が消えない」という挙動になる。Day3で見た `session` の使い方の延長線。
- `doReserve()` の二重チェック (`findActiveByUserAndDate` → `getSeatUsageForDate`) は、Day3の `LocationRegistLogic.saveReservation()` と同じ「予約済みかどうかをDB制約に頼らず先にチェックする」設計。座・Navi全体で繰り返されるこの二段階チェックのパターンを覚えておくと、Day5以降のキャンセル処理や代理予約も理解しやすくなる。
- `doReserve()` 成功時、`r.setGyosaki("出社")` で行先を「出社」として保存している。Day3の `LocationRegistLogic` で「出社はここでは登録せず `reserve` にリダイレクトする」と説明したのは、まさにここで「出社+座席」がセットで1件のレコードになるため。

3-4-3. webapp/WEB-INF/jsp/reserve.jsp (4段階ウィザード画面・座・Navi最大のJSP)

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.*, entity.Seat, dao.SeatsDAO" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>座席予約 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/header.jspf" %>
<%
    String stage    = (String) request.getAttribute("stage");
    if (stage == null) stage = "base";
    String selBase  = (String) request.getAttribute("selBase");
    String selFloor = (String) request.getAttribute("selFloor");
    String selArea  = (String) request.getAttribute("selArea");
    String officeImage = (String) request.getAttribute("officeImage");
    String ctx      = request.getContextPath();
    String menuAction = Boolean.TRUE.equals(session.getAttribute("isAdmin")) ? "adminMenu" : "menu";

    // stage → 番号マッピング (ステージバー用)
    int stageNum = "base".equals(stage) ? 1 : "floor".equals(stage) ? 2 : "area".equals(stage) ? 3 : 4;
    SeatsDAO _sdaoRsv = new SeatsDAO();

    // 一つ前のステージへ戻るリンク (座席→エリア→フロア→拠点の順に一段階ずつ戻る)
    String backUrl = null;
    if ("floor".equals(stage)) {
        backUrl = ctx + "/ControlServlet?action=reserve";
    } else if ("area".equals(stage)) {
        backUrl = ctx + "/ControlServlet?action=reserve&stage=floor&base=" + java.net.URLEncoder.encode(selBase, "UTF-8");
    } else if ("seat".equals(stage)) {
        backUrl = ctx + "/ControlServlet?action=reserve&stage=area&base=" + java.net.URLEncoder.encode(selBase, "UTF-8")
            + "&floor=" + java.net.URLEncoder.encode(selFloor, "UTF-8");
    }
%>

<div class="container">
<div class="card">
    <h2>座席予約</h2>

    <!-- ステージバー -->
    <div class="stage-bar">
        <div class="stage-step <%= stageNum >= 1 ? (stageNum > 1 ? "done" : "active") : "" %>>① 拠点</div>
        <div class="stage-step <%= stageNum >= 2 ? (stageNum > 2 ? "done" : "active") : "" %>>② フロア</div>
        <div class="stage-step <%= stageNum >= 3 ? (stageNum > 3 ? "done" : "active") : "" %>>③ エリア</div>
        <div class="stage-step <%= stageNum >= 4 ? "active" : "" %>>④ 座席</div>
    </div>

    <!-- パンくず -->
    <div class="breadcrumb">
        <a href="<%= ctx %>/ControlServlet?action=reserve">拠点選択</a>
        <% if (selBase != null) { %>
            <span class="sep">&rsquo;</span>
            <a href="<%= ctx %>/ControlServlet?action=reserve&stage=floor&base=<%= java.net.URLEncoder.encode(selBase, "UTF-8") %>">
<%= selBase %></a>
        <% } %>
        <% if (selFloor != null && selBase != null) { %>
            <span class="sep">&rsquo;</span>
            <a href="<%= ctx %>/ControlServlet?action=reserve&stage=area&base=<%= java.net.URLEncoder.encode(selBase, "UTF-8") %>&flo
oor=<%= java.net.URLEncoder.encode(selFloor, "UTF-8") %>"><%= selFloor %></a>
        <% } %>
        <% if (selArea != null && selFloor != null) { %>
            <span class="sep">&rsquo;</span>
            <span><%= selArea %></span>
        <% } %>
    </div>

    <!-- ===== 拠点ステージ ===== -->
```



```

<% if ("base".equals(stage)) { %>
<p style="color:#757575; margin-bottom:16px;">予約したい拠点を選んでください。</p>
<div class="occ-grid">
<%
    List<Map<String, Object>> items = (List<Map<String, Object>>) request.getAttribute("items");
    if (items != null) for (Map<String, Object> item : items) {
        String name = (String) item.get("name");
        int rate = (Integer) item.get("rate");
        int total = (Integer) item.get("total");
        Object usedObj = item.get("used");
        long used = usedObj instanceof Long ? (Long)usedObj : ((Integer)usedObj).longValue();
        String barClass = rate < 50 ? "" : rate < 80 ? "med" : rate < 100 ? "high" : "full";
        String encoded = java.net.URLEncoder.encode(name, "UTF-8");
        String _rBase = _sdaoRsv.getOfficeImage(name);
        String _iBase = (_rBase != null) ? _rBase : ("office_" + name + ".png");
    %>
    <a class="occ-card"
        href="<%= ctx %>/ControlServlet?action=reserve&stage=floor&base=<%= encoded %>">
    <div style="border-radius:5px; overflow:hidden; margin-bottom:8px; height:72px; background:#ECEFF1;">
        のオフィス"
            style="width:100%; height:72px; object-fit:cover; display:block;"
            onerror="this.style.display='none'; this.nextElementSibling.style.display='flex';">
        <div style="display:none; height:72px; align-items:center; justify-content:center; flex-direction:column; gap:3px;
font-size:11px; color:#B0BEC5; border:2px dashed #CFD8DC; border-radius:5px;">
            <span>画像未設定</span>
            <code style="font-size:9px; color:#90A4AE; word-break:break-all;"><%= _iBase %></code>
        </div>
    </div>
    <div class="occ-name"><%= name %></div>
    <div class="occ-bar-bg">
        <div class="occ-bar <%= barClass %>" style="width:<%= rate %>%;"></div>
    </div>
    <div class="occ-pct"><%= rate %><span style="font-size:14px;"></span></div>
    <!-- 分母: その拠点の全席 (全フロア・全エリア合計) が分母 -->
    <div class="occ-label"><%= used %> / <%= total %> 席 使用中</div>
    </a>
<% } %>
</div>

<!-- ===== フロアステージ ===== -->
<% } else if ("floor".equals(stage)) { %>
<p style="color:#757575; margin-bottom:16px;">フロアを選んでください。</p>

<!-- 選択した拠点の画像 (管理者の代理予約で拠点選択時に表示されるものと同じ) -->
<%
    String _rSelBase = _sdaoRsv.getOfficeImage(selBase);
    String _iSelBase = (_rSelBase != null) ? _rSelBase : ("office_" + selBase + ".png");
%>
<div style="margin-bottom:16px;">
    のオフィス"
        style="max-width:100%; border:1px solid #ECEFF1; border-radius:8px; display:block;"
        onerror="this.style.display='none'; this.nextElementSibling.style.display='flex';">
    <div class="map-placeholder" style="display:none;">
        <p class="ph-title">拠点画像が未設定です</p>
        <code><%= _iSelBase %></code>
        <p class="ph-hint">このファイルを images/ フォルダに配置すると自動で表示されます</p>
    </div>
</div>

<div class="occ-grid">
<%
    List<Map<String, Object>> items = (List<Map<String, Object>>) request.getAttribute("items");
    if (items != null) for (Map<String, Object> item : items) {
        String name = (String) item.get("name");
        int rate = (Integer) item.get("rate");
        int total = (Integer) item.get("total");
        Object usedObj = item.get("used");
        long used = usedObj instanceof Long ? (Long)usedObj : ((Integer)usedObj).longValue();
        String barClass = rate < 50 ? "" : rate < 80 ? "med" : rate < 100 ? "high" : "full";
        String encodedBase = java.net.URLEncoder.encode(selBase, "UTF-8");
        String encodedFloor = java.net.URLEncoder.encode(name, "UTF-8");
        String _rFloor = _sdaoRsv.getOfficeImage(selBase, name);

```



```

<% } else if ("seat".equals(stage)) {
    List<String> dates      = (List<String>) request.getAttribute("dates");
    String selectedDate    = (String) request.getAttribute("selectedDate");
    List<Seat> areaSeats    = (List<Seat>) request.getAttribute("areaSeats");
    Map<Integer,String> usage= (Map<Integer,String>) request.getAttribute("seatUsage");
    Map<Integer,String> userNameMap = (Map<Integer,String>) request.getAttribute("seatUserNameMap");
    Map<Integer,String> userDeptMap = (Map<Integer,String>) request.getAttribute("seatUserDeptMap");
    int areaRate  = request.getAttribute("areaRate") != null ? (Integer) request.getAttribute("areaRate") : 0;
    Object usedObj2 = request.getAttribute("areaUsed");
    long areaUsed = usedObj2 instanceof Long ? (Long)usedObj2 : (usedObj2 != null ? ((Integer)usedObj2).longValue() : 0);
    int areaTotal = request.getAttribute("areaTotal") != null ? (Integer) request.getAttribute("areaTotal") : 0;
    String encBase = selBase != null ? java.net.URLEncoder.encode(selBase, "UTF-8") : "";
    String encFloor = selFloor != null ? java.net.URLEncoder.encode(selFloor, "UTF-8") : "";
    String encArea  = selArea != null ? java.net.URLEncoder.encode(selArea, "UTF-8") : "";
%>

```

<!-- ① 日付プルダウン（今日から2週間先まで：15日分） -->

```

<form method="get" action="<%= ctx %>/ControlServlet" style="margin-bottom:20px;">
    <input type="hidden" name="action" value="reserve">
    <input type="hidden" name="stage" value="seat">
    <input type="hidden" name="base" value="<%= selBase %%">
    <input type="hidden" name="floor" value="<%= selFloor %%">
    <input type="hidden" name="area" value="<%= selArea %%">
    <div style="display:flex; gap:10px; align-items:flex-end; flex-wrap:wrap;">
        <div class="form-group" style="margin:0; flex:1; min-width:180px;">
            <label>予約日を選択してください（今日から2週間先まで）</label>
            <select id="dateSelect" name="date" onchange="submitReserveDateForm(this)">
                <% if (dates != null) for (String d : dates) { %>
                    <option value="<%= d %%" <%= d.equals(selectedDate) ? "selected" : "" %%"><%= d %%"</option>
                <% } %>
            </select>
        </div>
    </div>
</form>

```

<!-- ② エリア埋有率（選択日） -->

```

<div style="margin-bottom:20px;">
    <% String barClass2 = areaRate < 50 ? "" : areaRate < 80 ? "med" : areaRate < 100 ? "high" : "full"; %>
    <div style="display:inline-block; min-width:220px;">
        <div style="font-size:13px; font-weight:700; color:#757575; margin-bottom:6px;">
            <%= selectedDate %> の埋有率 (<%= selArea %>)
        </div>
        <div class="occ-bar-bg">
            <div class="occ-bar <%= barClass2 %%" style="width:<%= areaRate %%"></div>
        </div>
    <!-- 分母：そのエリアの席のみ -->
    <div style="margin-top:4px; font-size:13px;">
        <span class="occ-pct" style="font-size:20px;"><%= areaRate %%"</span>
        <span class="occ-label"> (<%= areaUsed %> / <%= areaTotal %> 席使用中) </span>
    </div>
</div>

```

<!-- ③ エリアマップ画像（ファイルが未設定の場合はプレースホルダーを表示） -->

```

<% String areaImage = (String) request.getAttribute("areaImage"); %>
<% if (areaImage != null) { %>
<div style="margin-bottom:16px;">
    <%= selFloor %><%= selArea %>エリアのフロアマップ"
        style="max-width:100%; border:1px solid #ECEFF1; border-radius:8px; display:block;"
        onerror="this.style.display='none'; this.nextElementSibling.style.display='flex';">
    <div class="map-placeholder" style="display:none;">
        <p class="ph-title">エリアマップ画像が未設定です</p>
        <code><%= areaImage %%"</code>
        <p class="ph-hint">このファイルを images/ フォルダに配置すると自動で表示されます</p>
    </div>
</div>
<% } %>

```

<!-- ④ 座席ボタン（緑=空き、赤=使用中） -->

```

<p style="font-size:13px; font-weight:700; margin-bottom:8px;">座席を選択してください</p>
<div class="summary-row">
    <span class="badge badge-green">■ 空き席</span>

```

```

        <span class="badge badge-red">■ 使用中</span>
    </div>

<!-- 空き席ボタンを押すと、下のメモを添えて予約する -->
<form method="post" action="<%= ctx %>/ControlServlet">
    <input type="hidden" name="action" value="doReserve">
    <input type="hidden" name="date" value="<%= selectedDate %>">

    <div class="form-group">
        <label for="memo">メモ（任意）</label>
        <textarea id="memo" name="memo" rows="2" placeholder="例：午後から外出予定 など" style="width:100%; max-width:480px;">
    </textarea>
    </div>

    <div class="seat-grid">
    <% if (areaSeats != null) for (Seat s : areaSeats) {
        boolean occupied = usage != null && usage.containsKey(s.getSeatId());
        String uname = (occupied && userNameMap != null && userNameMap.containsKey(s.getSeatId()))
            ? userNameMap.get(s.getSeatId()) : "";
        String udept = (occupied && userDeptMap != null && userDeptMap.containsKey(s.getSeatId()))
            ? userDeptMap.get(s.getSeatId()) : "";
        String hoverInfo = (udept != null && !udept.isEmpty()) ? (udept + " " + uname) : uname;
    %>
    <% if (!occupied) { %>
        <!-- 緑ボタン：押すと最終確認ダイアログを表示し、OKで予約確定 -->
        <button type="submit" name="seatId" value="<%= s.getSeatId() %>" class="seat-btn available"
            onclick="return confirmReserve('<%= selBase %>', '<%= selFloor %>', '<%= selArea %>', '<%= s.getSeatName() %>', '<%= selectedDate %>');">
            <span class="seat-id"><%= s.getSeatName() %></span>
            <span class="seat-user">空き</span>
        </button>
    <% } else { %>
        <!-- 赤ボタン：使用中（クリック不可）。誰が使っているか分かるよう、
            氏名（部署名）を表示しつつ、カーソルを合わせると title 属性で
            「部署名 氏名」がツールチップ表示される -->
        <button type="button" class="seat-btn occupied" disabled title="<%= hoverInfo %>">
            <span class="seat-id"><%= s.getSeatName() %></span>
            <span class="seat-user"><%= uname %><% if (udept != null && !udept.isEmpty()) { %><span style="font-size:11px; opacity:.8;"> (<%= udept %>) </span><% } %></span>
        </button>
    <% } %>
    <% } %>
    </div>
</form>

<script>
    // 座席ボタン押下時の最終確認ダイアログ
    function confirmReserve(base, floor, area, seatName, date) {
        return confirm(
            date + " に\n" +
            base + " " + floor + " " + area + " の " + seatName + " の座席を予約します。\n" +
            "よろしいですか？"
        );
    }

    // 日付プルダウンを変更すると画面が再読み込みされてメモが消えてしまうため、
    // 切り替え前の入力内容を一時保存しておき、再読み込み後に書き戻す。
    // ※ select.form.submit() はブラウザ標準のメソッド呼び出しで "submit" イベントが
    // 発火しないため、addEventListener('submit', ...) では保存できない。
    // そのため、保存してから送信する関数を onchange から直接呼び出す形にしている。
    var RESERVE_MEMO_KEY = 'reserveMemoDraft';

    function submitReserveDateForm(sel) {
        var memo = document.getElementById('memo');
        if (memo) sessionStorage.setItem(RESERVE_MEMO_KEY, memo.value);
        sel.form.submit();
    }

    (function() {
        var memo = document.getElementById('memo');
        if (!memo) return;
        var saved = sessionStorage.getItem(RESERVE_MEMO_KEY);

```

```

        if (saved != null) {
            memo.value = saved;
            sessionStorage.removeItem(RESERVE_MEMO_KEY);
        }
    })();
</script>

<% } %>

<div style="margin-top:24px; display:flex; gap:10px;">
    <% if (backUrl != null) { %>
        <a href="<%= backUrl %>" class="btn btn-secondary">&larr; 前のステップへ戻る</a>
    <% } %>
    <a href="<%= ctx %>/ControlServlet?action=<%= menuAction %>" class="btn btn-secondary">&larr; メニューへ戻る</a>
</div>
</div>
</div>
</body>
</html>

```

ここで新しく出てくるもの

- Object usedObj = item.get("used"); long used = usedObj instanceof Long ? (Long)usedObj : ((Integer)usedObj).longValue(); …
新しい構文: **instanceof**。「usedObj が Long 型かどうか」を確認している。ReserveLogic 側で m.put("used", used) の used は long 型 (Stream の .count() の戻り値) なので Long になるはずだが、念のため Integer の場合にも対応するための防御的なコード。
- <select id="dateSelect" name="date" onchange="submitReserveDateForm(this)"> と <%= d.equals(selectedDate) ? "selected" : "" %> … <option> の selected 属性を条件付きで出力することで、「現在選択中の日付」をプルダウンの初期表示に反映している。
- JavaScriptの sessionStorage (submitReserveDateForm /即時実行関数 (function(){...})()) … ブラウザの「タブを閉じるまで保持されるデータ保存領域」。日付を変更して画面が再読み込みされても、入力中のメモを一時的に保持して書き戻すために使われている。Javaのコードではないが、「画面の使い勝手のための工夫」として把握しておくといよい。

読み方のポイント

- このJSPは「1つのファイルで4つの画面（拠点・フロア・エリア・座席）を切り替える」という、座・Naviの中で最も複雑なJSP。<% if ("base".equals(stage)) { %> ... <% } else if ("floor".equals(stage)) { %> ... <% } %> という大きな if-else if の連鎖で、ReserveLogic が stage ごとに用意した items (または areaSeats 等) を表示し分けている。
- 拠点・フロア・エリアの各カードで使われている occ-card / occ-bar / occ-pct / occ-label は、Day6の seatStatus.jsp ・Day7の masterUpdate.jsp でも同じ見た目のパーツとして再利用されている (CSSクラス名で検索すると関連箇所が見つかる)。
- 座席ボタンの onclick="return confirmReserve(...)" は、JavaScriptの confirm() が true / false を返し、return false の場合は <form> の送信 (予約確定) がキャンセルされる、という標準的なブラウザの確認ダイアログパターン。

3-4-4. webapp/WEB-INF/jsp/reserveOK.jsp (予約完了画面)

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.List, entity.Reservation, entity.Seat" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>予約完了 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/header.jspf" %>

<div class="container">
  <div class="card" style="max-width:600px; margin:0 auto; text-align:center;">
    <div style="font-size:56px; margin-bottom:16px;">&#10003;</div>
    <h2 style="color:var(--green); border:none; text-align:center; margin-bottom:20px;">予約が完了しました</h2>

    <%
      // 通常の単席予約
      Reservation r = (Reservation) request.getAttribute("reservation");
      Seat seat = (Seat) request.getAttribute("seat");
      // 代理予約 (複数)
      List<String> createdList = (List<String>) request.getAttribute("createdList");
      String reserveDate = (String) request.getAttribute("reserveDate");
      String menuAction = Boolean.TRUE.equals(session.getAttribute("isAdmin")) ? "adminMenu" : "menu";
    %>

    <% if (r != null && seat != null) { %>
      <table class="table" style="text-align:left; max-width:400px; margin:0 auto 24px;">
        <tr><th style="width:40%;">予約日</th><td><%= r.getReserveDate() %></td></tr>
        <tr><th>拠点</th> <td><%= seat.getBaseName() %></td></tr>
        <tr><th>フロア</th><td><%= seat.getFName() %></td></tr>
        <tr><th>エリア</th><td><%= seat.getAreaName() %></td></tr>
        <tr><th>座席</th> <td><strong><%= seat.getSeatName() %></strong></td></tr>
        <% if (r.getMemo() != null && !r.getMemo().isEmpty()) { %>
        <tr><th>メモ</th> <td><%= r.getMemo() %></td></tr>
        <% } %>
      </table>
    <% } %>

    <% if (createdList != null && !createdList.isEmpty()) { %>
    <p style="font-weight:700; margin-bottom:10px;">代理予約完了 (<%= reserveDate %>) </p>
    <ul style="list-style:none; text-align:left; display:inline-block; margin-bottom:20px;">
      <% for (String line : createdList) { %>
      <li style="padding:4px 0; border-bottom:1px solid #eee;">&#10003; <%= line %></li>
      <% } %>
    </ul>
    <% } %>

    <div style="display:flex; gap:12px; justify-content:center;">
      <a href="<%= request.getContextPath() %>/ControlServlet?action=reserveConfirm" class="btn btn-primary">
        予約一覧を確認する
      </a>
      <a href="<%= request.getContextPath() %>/ControlServlet?action=<%= menuAction %>" class="btn btn-secondary">
        メニューへ戻る
      </a>
    </div>
  </div>
</div>
</body>
</html>
```

ここで新しく出てくるもの

- `List<String> createdList = (List<String>) request.getAttribute("createdList");` … この画面は、Day4の通常予約（reservation + seat が入る）だけでなく、Day6の**代理予約（複数人をまとめて登録する機能）**でも共用される。代理予約のときは `createdList`（「氏名: 拠点 座席名」のような文字列のリスト）が渡される。

- `<% if (r != null && seat != null) { %> ... <% } %>` と `<% if (createdList != null && !createdList.isEmpty()) { %> ... <% } %>`
… どちらか一方だけが表示される、という条件分岐。1つのJSPが「呼び出し元によって表示内容が変わる」典型例。

読み方のポイント

`reserveOK.jsp` は今の時点（Day4）では「通常の単席予約」の表示だけを見ればよい。`createdList / reserveDate` に関する部分は、Day6で `AdminReserveLogic` を読むときに改めて戻ってくると理解しやすい。**「1つのJSPが複数のLogicから forward される」**という構造に、ここで初めて触れることになる。

3-4-5. webapp/WEB-INF/jsp/reserveNG.jsp （予約失敗画面）

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>予約失敗 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/header.jspf" %>

<%
String menuAction = Boolean.TRUE.equals(session.getAttribute("isAdmin")) ? "adminMenu" : "menu";
%>

<div class="container">
<div class="card" style="max-width:500px; margin:0 auto; text-align:center;">
<div style="font-size:56px; color:var(--red); margin-bottom:16px;">&#10007;</div>
<h2 style="color:var(--red); border:none; text-align:center; margin-bottom:16px;">予約できませんでした</h2>
<div class="alert alert-danger">
<%= request.getAttribute("errorMsg") != null ? request.getAttribute("errorMsg") : "予約処理中にエラーが発生しました。" %>
</div>
<div style="display:flex; gap:12px; justify-content:center; margin-top:16px;">
<a href="<%= request.getContextPath() %>/ControlServlet?action=reserve" class="btn btn-primary">
座席選択に戻る
</a>
<a href="<%= request.getContextPath() %>/ControlServlet?action=<%= menuAction %>" class="btn btn-secondary">
メニューへ
</a>
</div>
</div>
</div>
</body>
</html>
```

ここで新しく出てくるもの

新しい構文はなし。`<%= request.getAttribute("errorMsg") != null ? ... : "..." %>` は、`errorMsg` が設定されていない場合のデフォルト文言を三項演算子で出している。

読み方のポイント

`reserveNG.jsp` は、Day3で見た `LocationRegistLogic.saveAndGoMenu()` から `errorMsg` 付きでforwardされる**共通のエラー画面**。`ReserveLogic.doReserve()` ・ `LocationRegistLogic` のどちらも、同じ `reserveNG.jsp` に `errorMsg` という同じキー名でメッセージを渡している点に注目すると、「画面の共通化」が見えてくる。

Day4のまとめ・①解説書との関連

- `ReserveLogic` の4段階ウィザード（`stage=base/floor/area/seat`）は、①解説書「2. 画面一覧・画面遷移表」の「座席予約」フローと対応している。1つのアクション（`reserve`）が複数の画面状態を持つのは座・Naviの中で唯一この機能だけ。
- `SeatsDAO` の `OFFICE_IMAGES` ・ `getExpectedImage()` 系メソッドは、①解説書「7. 画面デザイン・スタイル」の「フロアマップ画像の命名規則」と対応している。新しい拠点・フロアを追加するときは、ここに画像の対応を追記する必要がある。

- Day4で2回目の登場となった「Stream の `.filter().count()` で埋有率を計算する」パターンは、Day6の `ControlServlet.showSeatStatus()` ・ `showMenu()` でも繰り返し使われる。「`stream().filter(条件).count()` = 条件に合う要素の数」という読み方に慣れておくと、Day6が格段に読みやすくなる。
-

Day5：予約確認・キャンセル／検索系

Day5では「自分の予約を見る・取り消す」「人を探す」という3つの機能、計6ファイルを読む。ここまでのDayで出てきた `Map` での名寄せ・`List<Map<String, Object>>` での行データ作成パターンが 3回目・4回目の登場となるため、「あ、またこのパターンか」と感じられるかどうかを確認する回でもある。

3-5-1. model/ReserveConfirmLogic.java (予約確認・キャンセル)

```
package model;

import dao.AccountsDAO;
import dao.ReservationsDAO;
import dao.SeatsDAO;
import entity.Account;
import entity.Reservation;
import entity.Seat;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.util.*;

/**
 * 予約確認・キャンセル (action=reserveConfirm で一覧表示／action=doCancel で取消)。
 * ログイン中の本人の予約一覧を表示し、各行の「取消」ボタンから
 * doCancel を呼ぶと該当予約に取消フラグを立てる (物理削除ではなく論理削除)。
 */
public class ReserveConfirmLogic implements Logic {

    @Override
    public String execute(HttpServletRequest req, HttpServletResponse res) throws Exception {
        String action = req.getParameter("action");
        if ("doCancel".equals(action)) {
            return doCancel(req);
        }
        return showConfirm(req);
    }

    private String showConfirm(HttpServletRequest req) {
        HttpSession session = req.getSession(false);
        String userId = (String) session.getAttribute("userId");
        loadReservations(req, userId);
        return "/WEB-INF/jsp/reserveConfirm.jsp";
    }

    private String doCancel(HttpServletRequest req) {
        HttpSession session = req.getSession(false);
        String userId = (String) session.getAttribute("userId");
        String reserveIdStr = req.getParameter("reserveId");

        if (reserveIdStr != null && !reserveIdStr.isEmpty()) {
            int reserveId = Integer.parseInt(reserveIdStr);
            ReservationsDAO resDAO = new ReservationsDAO();
            Reservation r = resDAO.findById(reserveId);
            // 自分の予約のみキャンセル可 (セキュリティチェック)
            if (r != null && r.getUserId().equals(userId)) {
                resDAO.cancel(reserveId);
                req.setAttribute("cancelMessage", "予約をキャンセルしました。");
            }
        }
        loadReservations(req, userId);
        return "/WEB-INF/jsp/reserveConfirm.jsp";
    }

    private void loadReservations(HttpServletRequest req, String userId) {
        ReservationsDAO resDAO = new ReservationsDAO();
        SeatsDAO seatsDAO = new SeatsDAO();
        AccountsDAO accDAO = new AccountsDAO();

        List<Reservation> list = resDAO.findById(userId);

        // 座席名マップ (seatId → 座席情報)
        Map<Integer, Seat> seatMap = new HashMap<>();
        for (Reservation r : list) {
            if (r.getSeatId() > 0 && !seatMap.containsKey(r.getSeatId())) {
                Seat s = seatsDAO.findById(r.getSeatId());
                if (s != null) seatMap.put(r.getSeatId(), s);
            }
        }
    }
}
```

```

    }

    req.setAttribute("reservations", list);
    req.setAttribute("seatMap", seatMap);
}
}

```

ここで新しく出てくるもの

- `Map<Integer, Seat> seatMap = new HashMap<>(); ... if (... && !seatMap.containsKey(r.getSeatId())) { seatMap.put(...); } ...` 
新しい考え方：キャッシュ用Map。1件ずつ予約を見ていく中で、「同じ座席を何度も `SeatsDAO.findById()` で問い合わせない」ために、「すでに調べた座席IDなら `seatMap` に入っているはずなので、入っていない時だけDBに聞きに行く」という書き方。 `!containsKey(...)` が「まだ調べていない」という意味になる。
 - 例えば1人が「3日分の予約」を持っていて、そのうち2日が同じ座席だった場合、`seatMap` が無ければDBに2回問い合わせるが、`seatMap` があれば1回で済む。
- `r.getUserId().equals(userId)` のチェック ...  セキュリティチェック。`doCancel` では `reserveId` (予約ID) がパラメータとして送られてくるが、これは画面のHTMLを書き換えれば誰でも別の数字を送ってしまう。そこで「取得した予約の持ち主 (`r.getUserId()`) が、いまログイン中の本人 (`userId`) と一致するか」を必ず確認してからキャンセルしている。これを忘れると「他人の予約を勝手にキャンセルできてしまう」という重大な不具合になる。

読み方のポイント

- `execute()` の中身は「`action` が `doCancel` かどうかで処理を分けて、最後はどちらも `loadReservations()` を呼んで同じJSPに forward する」という、Day2の `LoginLogic` より単純な2分岐。`showConfirm` / `doCancel` という名前から「表示系」「実行系」と読み分ける癖をここでもつけておくと、Day6の `AdminReserveLogic` が読みやすくなる。
- `resDAO.cancel(reserveId)` は物理的に行を消すのではなく、「キャンセル済みフラグを立てる」処理 (`ReservationsDAO` はDay3で確認済み)。そのため、キャンセル後も `findByUserId` で取得される一覧には表示されない (`ReservationsDAO` 側で取消済みを除外している)。

3-5-2. webapp/WEB-INF/jsp/reserveConfirm.jsp (予約確認・キャンセル画面)

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.*, entity.Reservation, entity.Seat" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>予約確認 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/header.jspf" %>

<div class="container">
  <div class="card">
    <h2>予約確認・キャンセル</h2>

    <!-- キャンセル完了メッセージ -->
    <% if (request.getAttribute("cancelMessage") != null) { %>
    <div class="alert alert-success" style="font-size:15px; font-weight:600;">
      &#10003; <%= request.getAttribute("cancelMessage") %>
    </div>
    <% } %>

    <%
      List<Reservation> reservations = (List<Reservation>) request.getAttribute("reservations");
      Map<Integer, Seat> seatMap = (Map<Integer, Seat>) request.getAttribute("seatMap");
      String ctx = request.getContextPath();
      String menuAction = Boolean.TRUE.equals(session.getAttribute("isAdmin")) ? "adminMenu" : "menu";
    %>

    <% if (reservations == null || reservations.isEmpty()) { %>
    <p style="color:#757575;">有効な予約はありません。</p>
    <% } else { %>
    <table class="table">
      <thead>
        <tr>
          <th>予約ID</th><th>予約日</th><th>行先</th><th>座席</th><th>メモ</th><th>操作</th>
        </tr>
      </thead>
      <tbody>
        <% for (Reservation r : reservations) {
          Seat seat = r.getSeatId() > 0 && seatMap != null ? seatMap.get(r.getSeatId()) : null;
          String seatLabel = seat != null
            ? seat.getBaseName() + " " + seat.getFName() + " " + seat.getSeatName()
            : "-";
        %>
        <tr>
          <td><%= r.getReserveId() %></td>
          <td><%= r.getReserveDate() %></td>
          <td><span class="badge badge-green"><%= r.getGyosaki() %></span></td>
          <td><%= seatLabel %></td>
          <td><%= r.getMemo() != null ? r.getMemo() : "" %></td>
          <td>
            <!-- キャンセルボタン：POSTで送信（論理削除） -->
            <button type="submit" form="cancelForm<%= r.getReserveId() %>" class="btn btn-danger btn-sm">
              キャンセル
            </button>
          </td>
        </tr>
      <% } %>
    </tbody>
    </table>

    <!-- キャンセル用フォーム（テーブル外に配置） -->
    <% for (Reservation r : reservations) { %>
    <form id="cancelForm<%= r.getReserveId() %>" action="<%= ctx %>/ControlServlet" method="post" style="display:none;">
      <input type="hidden" name="action" value="doCancel">
      <input type="hidden" name="reserveId" value="<%= r.getReserveId() %>">
    </form>
    <% } %>
  </div>
</div>
```

```

        <% } %>
    <% } %>

    <div style="margin-top:20px;">
        <a href="<%= ctx %>/ControlServlet?action=<%= menuAction %>" class="btn btn-secondary">&larr; メニューへ戻る</a>
    </div>
</div>
</div>
</body>
</html>

```

ここで新しく出てくるもの

- `<button type="submit" form="cancelForm<%= r.getReserveId() %>" class="btn btn-danger btn-sm"> ...` ■ 新しい構文：HTML5の** form 属性**。通常、ボタンは自分を囲んでいる `<form>` タグに対して送信するが、`form="cancelFormXX"` と書くことで、「テーブルの外（後ろ）に置いてある別の `<form id="cancelFormXX">` に対して送信する」ことができる。
- なぜテーブルの中にボタンだけ置いて、フォーム本体を外に出しているのか？ ... `<table>` の `<tr>` の中に `<form>` を直接書くと、HTML的に行儀が悪くレイアウトが崩れることがあるため。「ボタンは見た目の位置に、フォームの実体はテーブルの後ろにまとめて並べる」という工夫。
`<% for (Reservation r : reservations) { %> <form id="cancelFormXX">...</form> <% } %>` で、予約の件数分のフォームが用意される。
- `String seatLabel = seat != null ? seat.getBaseName() + " " + seat.getFName() + " " + seat.getSeatName() : "-";` ... 三項演算子で「座席が無い予約（在宅・出張）の場合は全角ダッシュ - を表示する」という、Day1から繰り返し出てくる「null の場合の表示」パターン。

読み方のポイント

- `Map<Integer, Seat> seatMap` は `ReserveConfirmLogic` で作られたキャッシュそのもの。JSP側では `seatMap.get(r.getSeatId())` で「この予約の座席情報」を取り出すだけで、DBに問い合わせる必要は無い。「JavaのコードでMapを作る→JSPでMapから取り出す」というセットで理解する。
- 1人のユーザーが複数件の予約を持っている場合、`<tr>` が複数行表示され、それぞれに対応する `cancelFormXX` が用意される。「行ごとに別々のキャンセルボタン・フォームが必要」という要件を、`id` に `reserveld` を埋め込むことで実現している。

3-5-3. model/NameSearchLogic.java (氏名検索)

```
package model;

import dao.AccountsDAO;
import dao.LocationsDAO;
import dao.ReservationsDAO;
import dao.SeatsDAO;
import entity.Account;
import entity.Reservation;
import entity.Seat;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.time.LocalDate;
import java.util.*;

/**
 * 氏名検索（入力されたキーワードを含む氏名のアカウントを探し、各人の所属部署と
 * 本日の出社状況・座席を一覧表示する）。
 *
 * keyword パラメータが無ければ検索フォームだけを表示し、
 * あれば AccountsDAO.findByName() で部分一致検索した結果に
 * 部署名・本日の行先・座席情報を付け加えて JSP に渡す。
 */
public class NameSearchLogic implements Logic {

    @Override
    public String execute(HttpServletRequest req, HttpServletResponse res) throws Exception {
        String keyword = req.getParameter("keyword");

        if (keyword != null) {
            req.setAttribute("keyword", keyword);

            if (keyword.trim().isEmpty()) {
                // 何も入力せずに検索すると全員がヒットしてしまい一覧の意味がないため、
                // 入力を促すメッセージを出して検索は行わない
                req.setAttribute("errorMsg", "氏名を入力してください。");
            } else {
                // 検索実行
                AccountsDAO accDAO = new AccountsDAO();
                LocationsDAO locDAO = new LocationsDAO();
                ReservationsDAO resDAO = new ReservationsDAO();
                SeatsDAO seatDAO = new SeatsDAO();
                String today = LocalDate.now().toString();
                Map<Integer, String> todayUsage = resDAO.getSeatUsageForDate(today);

                List<Account> results = accDAO.findByName(keyword.trim());

                // 検索結果に各ユーザーの本日のステータスと予約座席を付与
                List<Map<String, Object>> rows = new ArrayList<>();
                for (Account a : results) {
                    Map<String, Object> row = new HashMap<>();
                    row.put("account", a);
                    row.put("bName", locDAO.getBNameById(a.getBId()));

                    List<Reservation> todayRes = resDAO.findTodayByUserId(a.getUserId());
                    String todayStatus = "未登録";
                    Seat seat = null;
                    if (!todayRes.isEmpty()) {
                        Reservation r = todayRes.get(0);
                        todayStatus = r.getGyosaki();
                        if (r.getSeatId() > 0) {
                            seat = seatDAO.findById(r.getSeatId());
                        }
                    }
                    row.put("todayStatus", todayStatus);
                    row.put("seat", seat);
                    rows.add(row);
                }
                req.setAttribute("results", rows);
            }
        }
    }
}
```

```
    }  
    return "/WEB-INF/jsp/nameSearch.jsp";  
  }  
}
```

ここで新しく出てくるもの

- `keyword.trim().isEmpty()` … `String.trim()` : 文字列の前後の空白を取り除く。「スペースだけ入力して検索ボタンを押す」というケースも「未入力」として扱うための処理。
- `dao.AccountsDAO.findByName(keyword)` … `WHERE name LIKE ?` に `"%" + keyword + "%"` を渡して呼び出している（実装はDay2で読んだ `AccountsDAO` の中にある）。用語集 → **LIKE** / **%** (**ワイルドカード**)。SQLの **LIKE** は「指定した文字を含むかどうか」で検索する命令で、**%** は「任意の文字列（0文字でもよい）」を表す記号。`"%田中%"` なら「どこかに“田中”を含む」全ての氏名にマッチする。
- このメソッドは `Map<Integer, String> todayUsage` を取得しているが、実際には使われていない（コードの中で計算だけして使っていない変数）。誤りではなく、結果に影響は無いが、「変数を作ったけど使い忘れている箇所もある」という、実際のコードにはよくある状態の例として知っておくとよい。

読み方のポイント

- `List<Map<String, Object>> rows` を作るループは、Day1の `LocationRegistLogic` やDay4の `ReserveLogic` の `items` 作成と全く同じ「**1件ごとに複数のDAOから情報を集めて1つのMapにまとめる**」パターンの3回目の登場。`row.put("account", a)` で元のオブジェクトをそのまま持たせつつ、`row.put("bName", ...)` のように「JSPで表示しやすい形に変換した値」も一緒に詰める、という考え方に慣れてきたはず。
 - `if (!todayRes.isEmpty()) { ... }` で「本日の予約が1件もない＝未登録」をデフォルト値 **"未登録"** として扱っているのは、Day1の `menu.jsp` 周りや、Day4で見た「行先が決まっていない人をどう表示するか」という設計と同じ考え方。
-

3-5-4. webapp/WEB-INF/jsp/nameSearch.jsp (氏名検索画面)

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.*, entity.Account, entity.Seat" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>氏名検索 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/header.jspf" %>

<%
String menuAction = Boolean.TRUE.equals(session.getAttribute("isAdmin")) ? "adminMenu" : "menu";
%>

<div class="container">
<div class="card">
<h2>氏名検索</h2>

<% if (request.getAttribute("errorMsg") != null) { %>
<div class="alert alert-danger"><%= request.getAttribute("errorMsg") %></div>
<% } %>

<form action="<%= request.getContextPath() %>/ControlServlet" method="get">
<input type="hidden" name="action" value="doNameSearch">
<div style="display:flex; gap:10px; align-items:flex-end;">
<div class="form-group" style="flex:1; margin:0;">
<label>氏名 (部分一致) </label>
<input type="text" name="keyword"
value="<%= request.getAttribute("keyword") != null ? request.getAttribute("keyword") : "" %>"
placeholder="例: 田中" autofocus>
</div>
<button type="submit" class="btn btn-primary">
検索
</button>
</div>
</form>

<% List<Map<String, Object>> results = (List<Map<String, Object>>) request.getAttribute("results"); %>
<% if (results != null) { %>
<div style="margin-top:20px;">
<p style="font-size:13px; color:#757575; margin-bottom:8px;"><%= results.size() %> 件見つかりました</p>
<table class="table">
<thead>
<tr><th>ユーザーID</th><th>氏名</th><th>部門</th><th>本日の行先</th><th>本日の予約座席</th></tr>
</thead>
<tbody>
<% for (Map<String, Object> row : results) {
Account a = (Account) row.get("account");
Seat seat = (Seat) row.get("seat");
%>
<tr>
<td><%= a.getUserId() %></td>
<td><%= a.getName() %></td>
<td><%= row.get("bName") %></td>
<td><span class="badge badge-green"><%= row.get("todayStatus") %></span></td>
<td>
<% if (seat != null) { %>
<%= seat.getBaseName() %> <%= seat.getFName() %> <%= seat.getAreaName() %> <%= seat.getSeatName() %>
<% } else { %>
<span style="color:var(--text-muted);">-</span>
<% } %>
</td>
</tr>
<% } %>
</tbody>
</table>
</div>
</div>
```



```

<% } %>

<div style="margin-top:20px;">
  <a href="<%= request.getContextPath() %>/ControlServlet?action=<%= menuAction %>" class="btn btn-secondary">&larr; メニューへ戻る</a>
</div>
</div>
</div>
</body>
</html>

```

ここで新しく出てくるもの

- `<form action="..." method="get">` …  ここで初めて**`method="get"` の検索フォーム**が登場する。これまでの登録系フォーム（Day3の行先登録、Day4の予約）は `method="post"` だったが、検索条件はURLに残ってもよい（むしろブックマークできた方が便利な）情報なので `get` を使っている。送信すると `?action=doNameSearch&keyword=田中` のようなURLになる。
- `autofocus` …  HTML属性。ページが開いたときに、その入力欄に自動でカーソル（フォーカス）が当たるようにする。検索画面でキーボードからすぐ入力を始められるようにする工夫。
- `<% if (results != null) { %>` … `results` は「検索を実行したかどうか」のフラグも兼ねている。`NameSearchLogic` 側で `keyword` が `null`（初回表示）のときは `results` を `setAttribute` しないため、ここで `null` のままになり、結果テーブルが表示されない。「変数が `null` かどうかで“まだ何もしていない”を表す」という、 解説書でも触れた考え方の実例。

読み方のポイント

- `value="<%= request.getAttribute("keyword") != null ? request.getAttribute("keyword") : "" %>"` … 検索後にこの画面が再表示されたとき、入力欄に直前の検索キーワードが残るようにしている。これも「入力値を消さない」というUI上の細かい配慮で、Day3~4でも何度か見た書き方。
- このJSPの構造は「フォーム（検索条件） → 結果テーブル（あれば） → 戻るリンク」という3段構成で、Day5・6で出てくる検索系画面（`departmentSearch.jsp` ・ `seatStatus.jsp`）もほぼ同じ構成になっている。

3-5-5. model/DepartmentSearchLogic.java (部門検索)

```
package model;

import dao.AccountsDAO;
import dao.LocationsDAO;
import dao.ReservationsDAO;
import dao.SeatsDAO;
import entity.Account;
import entity.Location;
import entity.Reservation;
import entity.Seat;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.util.*;

/**
 * 部門検索（指定した部署・社員IDに合致するメンバー一覧と、各メンバーの本日の出社状況を表示する）。
 *
 * 流れ：
 * 1) 画面表示時：部署の一覧（プルダウン用）を取得して渡す
 * 2) 部署（bId）・社員ID（userIdKeyword）のどちらか、または両方が指定されたとき：
 *    条件に合うメンバーを取得し、1人ずつ「本日の予約」を調べて
 *    「氏名」「本日の行先（出社/在宅/出張/未登録）」「座席（出社の場合）」をまとめてJSPに渡す
 *
 * 複数のDAO（Accounts → Reservations → Seats）を組み合わせて1画面分のデータを作る、
 * Logicクラスの典型的な役割が分かりやすい例になっている。
 */
public class DepartmentSearchLogic implements Logic {

    @Override
    public String execute(HttpServletRequest req, HttpServletResponse res) throws Exception {
        LocationsDAO locDAO = new LocationsDAO();
        List<Location> departments = locDAO.findAll();
        req.setAttribute("departments", departments); // 検索フォームの部署プルダウン用

        String bId = req.getParameter("bId");
        String userIdKeyword = req.getParameter("userIdKeyword");
        boolean hasBId = bId != null && !bId.isEmpty();
        boolean hasUserIdKeyword = userIdKeyword != null && !userIdKeyword.isEmpty();

        if (hasBId || hasUserIdKeyword) {
            AccountsDAO accDAO = new AccountsDAO();
            ReservationsDAO resDAO = new ReservationsDAO();
            SeatsDAO seatDAO = new SeatsDAO();

            // 条件に合うメンバー全員分を、1人ずつ「本日の状況」付きで組み立てる
            List<Account> members = accDAO.search(bId, userIdKeyword);
            List<Map<String, Object>> rows = new ArrayList<>();
            for (Account a : members) {
                Map<String, Object> row = new HashMap<>();
                row.put("account", a);

                // 本日の予約を取得し、行先（出社/在宅/出張）と座席（出社の場合のみ）を判定する
                List<Reservation> todayRes = resDAO.findTodayByUserId(a.getUserId());
                String todayStatus = "未登録";
                Seat seat = null;
                if (!todayRes.isEmpty()) {
                    Reservation r = todayRes.get(0);
                    todayStatus = r.getGyosaki();
                    if (r.getSeatId() > 0) {
                        seat = seatDAO.findById(r.getSeatId());
                    }
                }
                row.put("todayStatus", todayStatus);
                row.put("seat", seat);
                rows.add(row);
            }

            // 検索条件に応じた結果見出し（部署名・社員IDのどちらかを条件に使ったか分かるように）
            String resultLabel;

```

```

        if (hasBId && hasUserIdKeyword) {
            resultLabel = "「" + locDAO.getBNameById(bId) + "」かつ社員ID「" + userIdKeyword + "」に一致する社員";
        } else if (hasBId) {
            resultLabel = "「" + locDAO.getBNameById(bId) + "」のメンバー";
        } else {
            resultLabel = "社員ID「" + userIdKeyword + "」に一致する社員";
        }

        req.setAttribute("selectedBId", bId);
        req.setAttribute("selectedUserIdKeyword", userIdKeyword);
        req.setAttribute("resultLabel", resultLabel);
        req.setAttribute("results", rows);
    } else if (bId != null || userIdKeyword != null) {
        // フォームが送信された (=画面初回表示ではない) のに、部門・社員IDのどちらも
        // 指定されていない場合は、検索条件が無いまま全員を表示してしまわないように
        // 入力を促すメッセージを出す
        req.setAttribute("errorMsg", "部門または社員IDを入力してください。");
    }
    return "/WEB-INF/jsp/departmentSearch.jsp";
}
}
}

```

ここで新しく出てくるもの

- `dao.AccountsDAO.search(bId, userIdKeyword)` … 内部では `StringBuilder` を使って、`bId` が指定されていれば `AND b_id = ?`、`userIdKeyword` が指定されていれば `AND CAST(user_id AS CHAR) LIKE ?` をSQLに追加していく（`AccountsDAO` はDay2で読んだクラス）。
用語集 → **CAST(列 AS CHAR)**。`user_id` はDB上は数値 (int) の列だが、「社員IDの一部を入力して検索したい（部分一致）」という要件のために、いったん文字列 (CHAR) に変換してから `LIKE` で部分一致検索している。「数値の列を“文字として”扱いたいときに `CAST` を使う」という考え方を覚えておく。
- `boolean hasBId = bId != null && !bId.isEmpty();` と `boolean hasUserIdKeyword = ...` … 2つの条件をそれぞれ `boolean` 変数にしてから、`hasBId || hasUserIdKeyword` ・ `hasBId && hasUserIdKeyword` のように組み合わせて使っている。条件式をそのまま `if` に書くと長く読みにくくなるため、**意味のある名前の変数に一度入れてから使う**という書き方。Day4の `ReserveLogic` にも似た書き方（`boolean baseChosen` 等）があったので、見覚えがあるはず。
- `String resultLabel = ... ;` の `if / else if / else` で文字列を組み立てる部分 … 「検索条件によって見出し文が変わる」という、画面の説明文を動的に作るパターン。`+` による文字列連結（Day1から繰り返し登場）の応用。

読み方のポイント

- `execute()` 全体は「①プルダウン用の部署一覧を必ず作る → ②検索条件があれば結果を作る → ③検索条件が中途半端ならエラーメッセージ」という3段構成。`if (hasBId || hasUserIdKeyword) {...} else if (bId != null || userIdKeyword != null) {...}` の2段目の `else if` は、「検索ボタンを押したのに何も入力しなかった」場合だけを拾うための条件で、「画面の初回表示（`bId` も `userIdKeyword` も両方 `null`）」とは区別されている。この区別の付け方は `NameSearchLogic` の `keyword != null` の使い方とほぼ同じ発想。
- `List<Map<String, Object>> rows` を作る部分は `NameSearchLogic` とほぼ同じコード（4回目の「DAOをまとめて1行分のデータを作る」パターン）。違いは「検索対象が氏名キーワードか、部署+社員IDか」だけで、「本日の状況を調べて `row` に詰める」部分は全く同じ。**同じパターンが複数のLogicで繰り返し出てくる** = 座・Naviの「型」を覚えれば、新しい検索機能を追加するときも同じ型で書ける、ということが実感できる。

3-5-6. webapp/WEB-INF/jsp/departmentSearch.jsp (部門検索画面)

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.*, entity.Account, entity.Location, entity.Seat" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>部門検索 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/header.jspf" %>

<%
String menuAction = Boolean.TRUE.equals(session.getAttribute("isAdmin")) ? "adminMenu" : "menu";
%>

<div class="container">
<div class="card">
<h2>部門検索</h2>

<% if (request.getAttribute("errorMsg") != null) { %>
<div class="alert alert-danger"><%= request.getAttribute("errorMsg") %></div>
<% } %>

<%
List<Location> departments = (List<Location>) request.getAttribute("departments");
String selectedBId = (String) request.getAttribute("selectedBId");
String selectedUserIdKeyword = (String) request.getAttribute("selectedUserIdKeyword");
String resultLabel = (String) request.getAttribute("resultLabel");
%>

<form action="<%= request.getContextPath() %>/ControlServlet" method="get">
<input type="hidden" name="action" value="doDepartmentSearch">
<div style="display:flex; gap:10px; align-items:flex-end; flex-wrap:wrap;">
<div class="form-group" style="flex:1; min-width:180px; margin:0;">
<label>部門で絞り込み（任意）</label>
<select name="bId">
<option value=""-- 指定なし --</option>
<% if (departments != null) for (Location dept : departments) { %>
<option value="<%= dept.getBId() %>" <%= (selectedBId != null && selectedBId.equals(dept.getBId())) ? "selected" :
"" %>>
<%= dept.getBName() %>
</option>
<% } %>
</select>
</div>
<div class="form-group" style="flex:1; min-width:160px; margin:0;">
<label>社員IDで絞り込み（任意）</label>
<input type="text" name="userIdKeyword" placeholder="例: 1001"
value="<%= selectedUserIdKeyword != null ? selectedUserIdKeyword : "" %>">
</div>
<button type="submit" class="btn btn-primary">
検索
</button>
</div>
<p style="font-size:12px; color:#757575; margin-top:6px;">
部門・社員IDのどちらか一方、または両方を指定して検索できます。
</p>
</form>

<% List<Map<String, Object>> results = (List<Map<String, Object>>) request.getAttribute("results"); %>
<% if (results != null) { %>
<div style="margin-top:20px;">
<p style="font-size:13px; color:#757575; margin-bottom:8px;">
<%= resultLabel %> : <%= results.size() %> 名
</p>
<table class="table">
<thead>
<tr><th>ユーザーID</th><th>氏名</th><th>本日の行先</th><th>本日の予約座席</th></tr>
```

```

</thead>
<tbody>
  <% for (Map<String, Object> row : results) {
    Account a = (Account) row.get("account");
    Seat seat = (Seat) row.get("seat");
    %>
    <tr>
      <td><%= a.getUserId() %></td>
      <td><%= a.getName() %></td>
      <td><span class="badge badge-green"><%= row.get("todayStatus") %></span></td>
      <td>
        <% if (seat != null) { %>
          <%= seat.getBaseName() %> <%= seat.getFName() %> <%= seat.getAreaName() %> <%= seat.getSeatName() %>
        <% } else { %>
          <span style="color:var(--text-muted);"></span>
        <% } %>
      </td>
    </tr>
  <% } %>
</tbody>
</table>
</div>
<% } %>

<div style="margin-top:20px;">
  <a href="<%= request.getContextPath() %>/ControlServlet?action=<%= menuAction %>" class="btn btn-secondary">&larr; メニューへ戻る</a>
</div>
</div>
</div>
</body>
</html>

```

ここで新しく出てくるもの

- `<select name="bId"> <option value="">--- 指定なし ---</option> <% for (Location dept : departments) { %> <option value="<%= dept.getBId() %>" ...>...</option> <% } %> </select>` … `<select>` タグによる**プルダウン**。Day3の `locationRegist.jsp` 等でも似たプルダウンはあったが、ここでは先頭に `<option value="">--- 指定なし ---</option>` という「何も選んでいない」状態専用の選択枝を用意している点新しい。これにより「部署を指定しない検索」がプルダウンのUIとして自然に表現できる。
- `<%= (selectedBId != null && selectedBId.equals(dept.getBId())) ? "selected" : "" %>` … Day4の `reserve.jsp` の日付プルダウンで見た「選択中の値に `selected` を付ける」パターンの再登場。検索後にこの画面が再表示されたとき、直前に選んだ部署がプルダウンに残るようにしている。
- 全体として、`nameSearch.jsp` との違いは「検索条件が1つ（氏名）か、2つ（部署＋社員ID）か」「結果テーブルの列がやや違う（部門検索には部門列が無い＝検索条件として部門を選んでいるため、結果に重複して出す必要が薄いから）」程度で、画面の骨格はほぼ同じ。

読み方のポイント

- `nameSearch.jsp` を理解した後にこのファイルを読むと、「フォーム部分が `<input>` から `<select>` に変わった」「結果テーブルの列が少し違う」という**差分**として読めるはず。1から読むのではなく「前に読んだファイルとの違いはどこか」という視点を持つと、Day5以降は読むスピードが上がっていく。
- `resultLabel`（`DepartmentSearchLogic` で組み立てた文字列）は、検索結果の見出しとして `<%= resultLabel %> : <%= results.size() %> 名` の形でそのまま表示される。「Javaで文字列を組み立てて、JSPでそのまま出す」というのは、Day1からずっと繰り返されているMVCの基本形そのもの。

Day5のまとめ・①解説書との関連

- `ReserveConfirmLogic.doCancel()` の「`r.getUserId().equals(userId)` で自分の予約かどうかを確認する」セキュリティチェックは、①解説書「8. セキュリティ上の注意点」と対応する。**他人のデータを操作できないようにする**という考え方は、Day6で読む `AdminReserveLogic`（管理者が代わりに予約する機能）と対比すると、「管理者だからこそ複数人のデータを操作できる」という違いがより明確になる。
- `NameSearchLogic`・`DepartmentSearchLogic` の「`List<Map<String, Object>> rows` を1人ずつ組み立てる」パターンは、Day1・Day4に続いて3回目・4回目の登場。①解説書「3. 主な機能一覧」の「検索機能」の項目はこの2つのLogicが対応している。
- `AccountsDAO.search()` で出てきた `CAST(user_id AS CHAR) LIKE ?` は、Day7で `MastersDAO`／座席マスタのSQLを読むときに「`WHERE` に複数条件を組み合わせる」考え方の比較対象になる。「番号を文字列として検索する」という発想は、座席番号などでも応用できる。

Day6：メニュー画面／管理者機能（座席利用状況・代理予約）

Day6では、一般ユーザー・管理者それぞれの「ホーム画面」である `menu.jsp` / `adminMenu.jsp` と、管理者専用の2機能「座席利用状況確認」「代理予約」を読む。特に代理予約（`AdminReserveLogic` + `adminReserve.jsp`）は座・Naviの中で最もJavaScriptの量が多い画面で、「サーバー側（Java）でできること」と「ブラウザ側（JavaScript）でできること」の役割分担を学ぶ良い例になっている。

3-6-1. webapp/WEB-INF/jsp/menu.jsp (一般メニュー)

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.List, entity.Reservation, dao.SeatsDAO, entity.Seat" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>メニュー - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/header.jspf" %>

<div class="container">

    <%
    // 行先登録などリダイレクト後に表示するメッセージはセッションに積まれているため、
    // 表示したら一度だけ出るように消しておく（フラッシュメッセージ）
    Object successMsg = session.getAttribute("successMsg");
    if (successMsg != null) session.removeAttribute("successMsg");
    %>
    <% if (successMsg != null) { %>
    <div class="alert alert-success"><%= successMsg %></div>
    <% } %>

    <!-- 本日の予約 -->
    <div class="card">
        <h2>本日の予約状況</h2>
        <%
        List<Reservation> todayRes = (List<Reservation>) session.getAttribute("todayReservations");
        if (todayRes == null) todayRes = new java.util.ArrayList<>();
        SeatsDAO seatsDAO = new SeatsDAO();
        %>
        <% if (todayRes.isEmpty()) { %>
        <p style="color:#757575;">本日の予約はまだありません。</p>
        <% } else { %>
        <table class="table">
            <thead>
                <tr><th>日付</th><th>行先</th><th>座席</th><th>メモ</th></tr>
            </thead>
            <tbody>
                <% for (Reservation r : todayRes) {
                Seat seat = (r.getSeatId() > 0) ? seatsDAO.findById(r.getSeatId()) : null;
                String seatLabel = seat != null
                    ? seat.getBaseName() + " " + seat.getFName() + " " + seat.getSeatName()
                    : "-";
                %>
                <tr>
                    <td><%= r.getReserveDate() %></td>
                    <td><span class="badge badge-green"><%= r.getGyosaki() %></span></td>
                    <td><%= seatLabel %></td>
                    <td><%= r.getMemo() != null ? r.getMemo() : "" %></td>
                </tr>
                <% } %>
            </tbody>
        </table>
        <% } %>
    </div>

    <!-- 同じ部署の出勤状況 -->
    <%
    @SuppressWarnings("unchecked")
    List<java.util.Map<String, Object>> deptSeatList =
        (List<java.util.Map<String, Object>>) request.getAttribute("deptSeatList");
    %>
    <% if (request.getAttribute("deptMemberCount") != null) { %>
    <div class="card">
        <div style="display:flex; justify-content:space-between; align-items:center;">
            <h2 style="margin:0;">同じ部署の本日の出勤状況</h2>
            <button type="button" id="deptSeatToggleBtn" class="btn btn-secondary" style="font-size:13px; padding:6px 14px;">
```

```

        onclick="toggleDeptSeatList()">表示/非表示</button>
    </div>
    <div class="summary-row" style="margin-top:10px;">
        <span class="badge badge-green">本日出社:<%= request.getAttribute("deptOfficeCount") %> 名</span>
        <span class="badge badge-gray">部署人数:<%= request.getAttribute("deptMemberCount") %> 名</span>
    </div>
    <div id="deptSeatListBody">
    <% if (deptSeatList != null && !deptSeatList.isEmpty()) { %>
    <table class="table" style="margin-top:12px;">
        <thead><tr><th>氏名</th><th>本日の状況</th><th>座席</th></tr></thead>
        <tbody>
            <% for (java.util.Map<String, Object> row : deptSeatList) {
                Seat seat = (Seat) row.get("seat");
                String seatLabel = seat != null
                    ? seat.getBaseName() + " " + seat.getFName() + " " + seat.getAreaName() + " " + seat.getSeatName()
                    : "-";

            %>
            <tr>
                <td><%= row.get("name") %></td>
                <td><span class="badge badge-green"><%= row.get("todayStatus") %></span></td>
                <td><%= seatLabel %></td>
            </tr>
            <% } %>
        </tbody>
    </table>
    <% } %>
    </div>
</div>
<script>
// 表示/非表示の状態はブラウザに記憶させ、次回アクセス時と同じ状態で開く
(function() {
    var KEY = 'zaNaviDeptSeatListHidden';
    var body = document.getElementById('deptSeatListBody');
    if (localStorage.getItem(KEY) === 'true') body.style.display = 'none';
})();
function toggleDeptSeatList() {
    var KEY = 'zaNaviDeptSeatListHidden';
    var body = document.getElementById('deptSeatListBody');
    var hidden = body.style.display === 'none';
    body.style.display = hidden ? '' : 'none';
    localStorage.setItem(KEY, String(!hidden));
}
</script>
<% } %>

<!-- 機能メニュー -->
<div class="card">
    <h2>機能メニュー</h2>
    <div class="menu-grid">
        <a href="<%= request.getContextPath() %>/ControlServlet?action=locationRegist" class="menu-btn">
            行先登録
        </a>
        <a href="<%= request.getContextPath() %>/ControlServlet?action=reserveConfirm" class="menu-btn">
            予約確認
        </a>
        <a href="<%= request.getContextPath() %>/ControlServlet?action=nameSearch" class="menu-btn">
            氏名検索
        </a>
        <a href="<%= request.getContextPath() %>/ControlServlet?action=departmentSearch" class="menu-btn">
            部門検索
        </a>
    </div>
</div>

</div>
</body>
</html>

```

ここで新しく出てくるもの

- Object successMsg = session.getAttribute("successMsg"); if (successMsg != null) session.removeAttribute("successMsg"); ... ■ 新しい考え方：フラッシュメッセージ。「行先登録が完了しました」のようなメッセージを、登録処理から menu にリダイレクトした後の画面で1

回だけ表示したい場合、`request` 属性では画面遷移（リダイレクト）の間に消えてしまう。そこで一時的に `session` に入れておき、メニュー画面で**読み取ったらすぐ消す**ことで「次にこの画面を表示したときには出ない」ようにしている。

- `localStorage.getItem(KEY)` / `localStorage.setItem(KEY, ...)` … ■ ブラウザの `localStorage`。`sessionStorage`（Day4で既出、タブを閉じると消える）と違い、**ブラウザを閉じても消えない**保存領域。「同じ部署の出社状況を表示/非表示にした状態」をブラウザに覚えさせておき、次回アクセス時も同じ表示状態にする。
- `<div class="menu-grid"> <a href="...?action=XXX" class="menu-btn"> <img ...> ラベル </a> ... </div>` … 機能メニューのアイコンリンク一覧。`action=` の値（`locationRegist`、`reserveConfirm` 等）は、これまでのDayで読んできた `ControlServlet` の `switch` の各 `case` とすべて対応している。「このアイコンを押すとどのLogicが呼ばれるか」を **action 名で逆引き**できるようになっていれば、Day1～5の内容が身についている証拠。

読み方のポイント

- `menu.jsp` は「本日の予約」「同じ部署の出社状況（管理者が部署のメンバーである場合のみ表示）」「機能メニュー」の3つの `<div class="card">` からなる、シンプルな一覧画面。`deptSeatList` は `ControlServlet.showMenu()` で作られる属性で、次の3-6-2で詳しく確認する。
- `<%@ include file="common/header.jspf" %>` で表示される共通ヘッダー（Day2で既読）には、ログイン中ユーザーの氏名やログアウトボタンが含まれている。`menu.jsp` 自体にはログイン情報の表示コードが無いことに注目すると、「共通部分は `.jspf` に切り出す」という設計がもう一度確認できる。

3-6-2. `control/ControlServlet.java` の `showMenu()` （メニュー表示の中身）

`ControlServlet` 本体は既にDay1で全体構造を確認済みなので、ここでは `menu` / `adminMenu` 両方のアクションから呼ばれる `showMenu()` メソッドだけを取り上げる。`isAdmin` という1つの `boolean` 引数によって、一般メニューと管理者メニューの両方を1つのメソッドで作っている点に注目する。

```
// ----- メニュー共通 -----

private String showMenu(HttpServletRequest req, boolean isAdmin) {
    HttpSession session = req.getSession(false);
    String userId = (String) session.getAttribute("userId");

    AccountsDAO accDAO = new AccountsDAO();
    ReservationsDAO resDAO = new ReservationsDAO();

    List<Reservation> todayRes = resDAO.findTodayByUserId(userId);
    session.setAttribute("todayReservations", todayRes);
    req.setAttribute("todayReservations", todayRes);

    // 同じ部署の本日の出社人数・座席一覧（管理者メニューだけに表示する情報なので、管理者のときだけ集計する）
    Account me = accDAO.findById(userId);
    if (isAdmin && me != null) {
        SeatsDAO seatDAO = new SeatsDAO();
        List<Account> deptMembers = accDAO.findByBId(me.getBId());

        List<Map<String, Object>> deptSeatList = new ArrayList<>();
        long officeCount = 0;
        for (Account a : deptMembers) {
            List<Reservation> mRes = resDAO.findTodayByUserId(a.getUserId());

            // 本日の状況（未登録の場合は「未登録」のままにする）
            String todayStatus = "未登録";
            Seat seat = null;
            if (!mRes.isEmpty()) {
                Reservation r = mRes.get(0);
                todayStatus = r.getGyosaki();
                if ("出社".equals(todayStatus)) {
                    officeCount++;
                    seat = r.getSeatId() > 0 ? seatDAO.findById(r.getSeatId()) : null;
                }
            }

            Map<String, Object> row = new HashMap<>();
            row.put("name", a.getName());
            row.put("todayStatus", todayStatus);
            row.put("seat", seat);
            deptSeatList.add(row);
        }
        req.setAttribute("deptMemberCount", deptMembers.size());
        req.setAttribute("deptOfficeCount", officeCount);
        req.setAttribute("deptSeatList", deptSeatList);
    }

    // 管理者向け：今日から2週間分の予約状況サマリ（人数だけでなく、誰がどの席も見られるように）
    if (isAdmin) {
        SeatsDAO seatDAO = new SeatsDAO();
        dao.LocationsDAO locDAO2 = new dao.LocationsDAO();

        // 部署一覧は最初に1回だけ取得して bId→bName のマップを作っておく
        // （1人ずつ部署名を問い合わせると、14日分×人数のDB接続が発生し重くなるため）
        Map<String, String> deptNameMap = new HashMap<>();
        for (entity.Location loc : locDAO2.findAll()) {
            deptNameMap.put(loc.getBId(), loc.getBName());
        }

        List<Map<String, Object>> twoWeekSummary = new ArrayList<>();
        for (int i = 0; i < 14; i++) {
            String date = LocalDate.now().plusDays(i).toString();
            List<Reservation> dayRes = resDAO.findByDate(date);
            long officeNum = dayRes.stream().filter(r -> "出社".equals(r.getGyosaki())).count();
            long seatNum = dayRes.stream().filter(r -> r.getSeatId() > 0).count();

            // 出社・座席が確定している人の「氏名+部署+座席」明細（折りたたみ表示用）
            List<Map<String, Object>> people = new ArrayList<>();
            for (Reservation r : dayRes) {
                if (!"出社".equals(r.getGyosaki()) || r.getSeatId() <= 0) continue;
                Account acc = accDAO.findById(r.getUserId());
            }
        }
    }
}
```

```

        Seat seat = seatDAO.findById(r.getSeatId());
        Map<String, Object> p = new HashMap<>();
        p.put("name", acc != null ? acc.getName() : r.getUserId());
        p.put("bName", acc != null ? deptNameMap.getOrDefault(acc.getBId(), "不明") : "");
        p.put("seat", seat);
        people.add(p);
    }

    Map<String, Object> row = new HashMap<>();
    row.put("date", date);
    row.put("officeCount", officeNum);
    row.put("seatCount", seatNum);
    row.put("people", people);
    twoWeekSummary.add(row);
}
req.setAttribute("twoWeekSummary", twoWeekSummary);
}

return isAdmin ? "/WEB-INF/jsp/adminMenu.jsp" : "/WEB-INF/jsp/menu.jsp";
}

```

ここで新しく出てくるもの

- `private String showMenu(HttpServletRequest req, boolean isAdmin) ...` `** boolean` の引数で処理を分岐する**設計。 `dispatch()` の中で `case "menu": return showMenu(req, false); case "adminMenu": return showMenu(req, true);` と呼び分けることで、「共通する処理（本日の予約取得）はまとめて書き、管理者だけの追加処理は `if (isAdmin)` で囲む」という1メソッド2画面構成になっている。最後の `return isAdmin ? "/WEB-INF/jsp/adminMenu.jsp" : "/WEB-INF/jsp/menu.jsp";` で、forward先のJSPも切り分けている。
- `Map<String, String> deptNameMap = new HashMap<>(); for (entity.Location loc : locDAO2.findAll()) { deptNameMap.put(loc.getBId(), loc.getBName()); } ...` Day5の `seatMap` キャッシュと同じ考え方だが、こちらは「最初に全件を取得してMapに変換しておく」という**事前一括取得型**のキャッシュ。コメントにある通り、「14日分×人数のDB接続」を避けるための工夫。 `!containsKey()` 方式（必要になった時点で取りに行く）と、 `findAll()` 方式（最初に全部取っておく）の2種類のキャッシュの作り方があることに気づけるとよい。
- `deptNameMap.getOrDefault(acc.getBId(), "不明") ...` `Map.getOrDefault(キー, デフォルト値)`。 `get()` だと該当が無いと `null` が返るが、 `getOrDefault()` は「無ければこの値を返す」をその場で指定できる。 `acc.getBId()` が `deptNameMap` に登録されていない部署IDだった場合に `"不明"` という文字列を返す。

読み方のポイント

- `twoWeekSummary` の二重ループ（外側：14日分の日付、内側：その日の予約一覧）は、計算量としては「14日 × （その日の予約件数）」になる。「外側のループの中で、毎回 `findAll()` のような重い処理を呼ばない」よう、 `deptNameMap` や `seatDAO` を**ループの外で1回だけ用意している**点が、コメントで説明されている設計意図。
- `req.setAttribute("twoWeekSummary", twoWeekSummary)` で渡された値は、3-6-4の `adminMenu.jsp` で `<details>` タグを使った折りたたみ表示として使われる。「Javaで2週間分のデータを1つのListにまとめて、JSP側はそれをループで表示するだけ」という、これまでのDayと同じ役割分担。

3-6-3. model/AdminReserveLogic.java (代理予約)

```
package model;

import dao.AccountsDAO;
import dao.LocationsDAO;
import dao.ReservationsDAO;
import dao.SeatsDAO;
import entity.Account;
import entity.Location;
import entity.Reservation;
import entity.Seat;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.time.LocalDate;
import java.util.*;

public class AdminReserveLogic implements Logic {

    @Override
    public String execute(HttpServletRequest req, HttpServletResponse res) throws Exception {
        String action = req.getParameter("action");

        if ("doAdminReserve".equals(action)) {
            return doReserve(req);
        }
        return showForm(req);
    }

    private String showForm(HttpServletRequest req) {
        SeatsDAO seatsDAO = new SeatsDAO();
        AccountsDAO accDAO = new AccountsDAO();
        LocationsDAO locDAO = new LocationsDAO();

        // 部署一覧は最初に1回だけ取得して bId→bName のマップを作っておく
        // (社員1人ごとに部署名をDB問い合わせすると、社員数×2回のDB接続が発生し、
        // ページ表示が遅くなったり、接続過多でフロア一覧などの取得が失敗することがあったため)
        List<Location> departments = locDAO.findAll();
        Map<String, String> deptNameMap = new HashMap<>();
        List<String> deptNames = new ArrayList<>();
        for (Location loc : departments) {
            deptNameMap.put(loc.getBId(), loc.getBName());
            deptNames.add(loc.getBName());
        }

        // パラメータ引き継ぎ (段階選択用)
        String base = req.getParameter("base");
        String floor = req.getParameter("floor");
        String area = req.getParameter("area");
        String date = req.getParameter("date");
        String selectedDate = (date != null && !date.isEmpty()) ? date : LocalDate.now().toString();
        req.setAttribute("selectedDate", selectedDate);

        // 「〇〇部署の〇〇」形式で検索・表示できるように、社員一覧に部署名を付与
        // すでにその日の予定 (出社・在宅・出張のいずれか) が入っている人は、
        // 代理予約しても結局エラーになるため、検索結果の時点で除外しておく
        //
        // 社員1人ごとに findActiveByUserAndDate() でDB問い合わせすると
        // 社員数分のDB接続が毎回発生し、拠点・フロア・エリアを切り替えるたびに
        // ページ全体が重くなっていたため、その日の予約済みuserIdを1回のクエリで
        // まとめて取得し、Setで判定する。
        ReservationsDAO resDAO = new ReservationsDAO();
        Set<String> reservedUserIds = new HashSet<>();
        for (Reservation r : resDAO.findByDate(selectedDate)) {
            reservedUserIds.add(r.getUserId());
        }

        // 社員一覧は1回だけ取得し、userId→Account のマップも作っておく
        // (後段の「使用中座席の予約者名表示」で1件ずつ findById() すると
        // その日の予約数だけDB接続が発生してしまうため、ここで使い回す)
```

```

List<Account> allAccounts = accDAO.findAll();
Map<String, Account> accountMap = new HashMap<>();
for (Account a : allAccounts) accountMap.put(a.getUserId(), a);

List<Map<String, Object>> accountRows = new ArrayList<>();
for (Account a : allAccounts) {
    if (reservedUserIds.contains(a.getUserId())) continue;
    String bName = deptNameMap.getOrDefault(a.getBId(), "不明");
    Map<String, Object> row = new HashMap<>();
    row.put("userId", a.getUserId());
    row.put("name", a.getName());
    row.put("bName", bName);
    row.put("label", bName + "の" + a.getName());
    accountRows.add(row);
}
req.setAttribute("accountRows", accountRows);
req.setAttribute("deptNames", deptNames);
req.setAttribute("bases", seatsDAO.getBases());

// 「対象社員を選択してください」等のエラーで画面に戻ってきたときや、
// 拠点・フロア・エリアを切り替えてページが再読み込みされたときも、
// 入力済みの検索条件（部署・氏名）・選択中の対象社員・モードが
// 消えてしまわないように引き継ぐ（reloadWith()がこれらもPOSTで送ってくる）
req.setAttribute("searchDeptIndividual", req.getParameter("searchDeptIndividual"));
req.setAttribute("searchNameIndividual", req.getParameter("searchNameIndividual"));
req.setAttribute("searchUserIdIndividual", req.getParameter("searchUserIdIndividual"));
req.setAttribute("searchDeptMeeting", req.getParameter("searchDeptMeeting"));
req.setAttribute("searchNameMeeting", req.getParameter("searchNameMeeting"));
req.setAttribute("searchUserIdMeeting", req.getParameter("searchUserIdMeeting"));
req.setAttribute("selMode", req.getParameter("mode"));
req.setAttribute("selTargetUserId", req.getParameter("targetUserId"));
req.setAttribute("selTargetUserIds", req.getParameterValues("targetUserIds"));

if (base != null && !base.isEmpty()) {
    req.setAttribute("selBase", base);
    req.setAttribute("floors", seatsDAO.getFloors(base));
}
if (floor != null && !floor.isEmpty() && base != null) {
    req.setAttribute("selFloor", floor);
    req.setAttribute("areas", seatsDAO.getAreas(base, floor));
}
if (area != null && !area.isEmpty() && base != null && floor != null) {
    req.setAttribute("selArea", area);

    // 指定日の座席状況
    Map<Integer, String> usage = resDAO.getSeatUsageForDate(selectedDate);
    List<Seat> seats = seatsDAO.findByArea(base, floor, area);

    // 使用中の座席にカーソルを合わせたときに、予約者の氏名・部署名が
    // ひと目で分かるように（座席ボタンには userId しか入っていないため変換する）
    Map<Integer, String> seatUserNameMap = new HashMap<>();
    Map<Integer, String> seatUserDeptMap = new HashMap<>();
    for (Map.Entry<Integer, String> e : usage.entrySet()) {
        Account acc = accountMap.get(e.getValue());
        seatUserNameMap.put(e.getKey(), acc != null ? acc.getName() : e.getValue());
        seatUserDeptMap.put(e.getKey(), acc != null ? deptNameMap.getOrDefault(acc.getBId(), "不明") : "");
    }

    req.setAttribute("areaSeats", seats);
    req.setAttribute("seatUsage", usage);
    req.setAttribute("seatUserNameMap", seatUserNameMap);
    req.setAttribute("seatUserDeptMap", seatUserDeptMap);
}

// 日付プルダウン（予約できるのは今日から2週間先まで）
List<String> dates = new ArrayList<>();
LocalDate d = LocalDate.now();
for (int i = 0; i < 15; i++) dates.add(d.plusDays(i).toString());
req.setAttribute("dates", dates);

return "/WEB-INF/jsp/adminReserve.jsp";
}

```

```

private String doReserve(HttpServletRequest req) {
    HttpSession session = req.getSession(false);
    String adminId = (String) session.getAttribute("userId");
    ReservationsDAO resDAO = new ReservationsDAO();
    SeatsDAO seatsDAO = new SeatsDAO();

    String mode = req.getParameter("mode"); // individual / meeting
    String date = req.getParameter("date");
    String memo = req.getParameter("memo");
    String[] seatIds = req.getParameterValues("seatIds"); // 複数選択

    if (date == null || date.isEmpty()) date = LocalDate.now().toString();
    if (seatIds == null || seatIds.length == 0) {
        req.setAttribute("errorMsg", "座席を選択してください。");
        return showForm(req);
    }

    Map<Integer, String> usage = resDAO.getSeatUsageForDate(date);
    List<String> created = new ArrayList<>();

    if ("individual".equals(mode)) {
        // 個人の代理予約 (1名・1席)
        if (seatIds.length > 1) {
            req.setAttribute("errorMsg", "個人の代理予約では座席は1席のみ選択してください。");
            return showForm(req);
        }
        String targetUserId = req.getParameter("targetUserId");
        if (targetUserId == null || targetUserId.isEmpty()) {
            req.setAttribute("errorMsg", "対象社員を選択してください。");
            return showForm(req);
        }
        int seatId = Integer.parseInt(seatIds[0]);
        if (usage.containsKey(seatId)) {
            req.setAttribute("errorMsg", "選択した座席はすでに予約済みです。");
            return showForm(req);
        }
        Reservation r = makeReservation(adminId, targetUserId, date, seatId, memo);
        resDAO.add(r);
        created.add(targetUserId + " → " + getSeatLabel(seatsDAO.findById(seatId)));
    } else {
        // 会議用 (複数名×複数席)
        String[] targetUserIds = req.getParameterValues("targetUserIds");
        if (targetUserIds == null || targetUserIds.length == 0) {
            req.setAttribute("errorMsg", "対象社員を選択してください。");
            return showForm(req);
        }
        if (targetUserIds.length != seatIds.length) {
            req.setAttribute("errorMsg",
                "参加者数 (" + targetUserIds.length + "名) と選択座席数 (" + seatIds.length + "席) が一致しません。");
            return showForm(req);
        }
        for (int i = 0; i < targetUserIds.length; i++) {
            int seatId = Integer.parseInt(seatIds[i]);
            if (usage.containsKey(seatId)) continue; // すでに使用中はスキップ
            Reservation r = makeReservation(adminId, targetUserIds[i], date, seatId, memo);
            resDAO.add(r);
            created.add(targetUserIds[i] + " → " + getSeatLabel(seatsDAO.findById(seatId)));
        }
    }

    req.setAttribute("createdList", created);
    req.setAttribute("reserveDate", date);
    return "/WEB-INF/jsp/reserveOK.jsp";
}

private Reservation makeReservation(String registerId, String userId, String date, int seatId, String memo) {
    Reservation r = new Reservation();
    r.setRegisterId(registerId); // 予約した管理者
    r.setUserId(userId); // 実際に座る人
    r.setReserveDate(date);
}

```

```

        r.setGyosaki("出社");
        r.setSeatId(seatId);
        r.setMemo(memo != null && !memo.isEmpty() ? memo : null);
        r.setCancelFlag(0);
        return r;
    }

    private String getSeatLabel(Seat s) {
        if (s == null) return "不明";
        return s.getBaseName() + " " + s.getFName() + " " + s.getAreaName() + " " + s.getSeatName();
    }
}

```

ここで新しく出てくるもの

- `Set<String> reservedUserIds = new HashSet<>(); for (Reservation r : resDAO.findByDate(selectedDate)) { reservedUserIds.add(r.getUserId()); } ...` ■ 用語集 → **Set** / **HashSet**。 **List** と同じく複数の値を持てるが、**Set** は「同じ値を2つ持てない」「順番を保証しない」コレクション。ここでは「その日すでに予定が入っている **userId** の集合」を作っており、`reservedUserIds.contains(a.getUserId())` で「この人はもう予定がある」を高速に判定できる。**List** の `contains()` も使えるが、件数が多くなるほど **Set** の方が高速。
- `r.setRegisterId(registerId); r.setUserId(userId); ...` ■ **registerId**（予約した人＝管理者）と **userId**（実際にその席に座る人）が**別人になる**のは、座・Naviの中でこの代理予約だけ。Day1で **Reservation** エンティティを読んだときに「**registerId** と **userId** という似た2つのフィールドがある」のを覚えているはず。通常の予約（Day4の **ReserveLogic**）では両方に同じ **userId** が入っていたが、ここで初めて「登録した人 ≠ 利用する人」のケースが出てくる。
- `mode` による `if ("individual".equals(mode)) { ... } else { ... }` の大きな分岐…「個人の代理予約（1名・1席）」と「会議用（複数名×複数席）」で、入力チェックの内容が異なる。会議用では `for (int i = 0; i < targetUserIds.length; i++)` で「`targetUserIds[i]` の人が `seatIds[i]` の席に座る」という**配列のインデックスを対応させる**書き方をしている。`targetUserIds` と `seatIds` は、画面側で「何番目を選んで参加者か」と「何番目を選んだ座席か」の順序が対応するように作られている（3-6-5のJavaScriptで詳しく見る）。

読み方のポイント

- `showForm()` はDay4の `ReserveLogic.showReserve()` と同じ「拠点→フロア→エリア→座席」という段階選択の考え方を踏襲しているが、**セッションに保存せず、毎回パラメータとして送る**点異なる（`reloadWith()` がPOSTで毎回 `base / floor / area / date` を送ってくる）。これは、管理者が複数の社員・複数の拠点を切り替えながら何度も画面を操作するため、「ブラウザの戻る/進む」よりも「常に最新の選択状態をフォームごと送る」方が確実、という設計判断と考えられる。
- `doReserve()` の最後は必ず `reserveOK.jsp`（Day4で確認した「`createdList` が渡されるとそちらを表示する」分岐）にforwardされる。Day4で「`createdList` はDay6で戻ってくる」と書いた箇所がここに対応する。

3-6-4. webapp/WEB-INF/jsp/adminMenu.jsp (管理者メニュー)

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.List, java.util.Map, entity.Reservation, dao.SeatsDAO, entity.Seat" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>管理者メニュー - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/headerAdmin.jspf" %>

<div class="container">

    <%
    // 行先登録などリダイレクト後に表示するメッセージはセッションに積まれているため、
    // 表示したら一度だけ出るように消しておく（フラッシュメッセージ）
    Object successMsg = session.getAttribute("successMsg");
    if (successMsg != null) session.removeAttribute("successMsg");
    %>
    <% if (successMsg != null) { %>
    <div class="alert alert-success"><%= successMsg %></div>
    <% } %>

    <!-- 本日の予約 -->
    <div class="card">
        <h2>本日の予約状況（管理者ビュー）</h2>
        <%
        List<Reservation> todayRes = (List<Reservation>) session.getAttribute("todayReservations");
        if (todayRes == null) todayRes = new java.util.ArrayList<>();
        SeatsDAO seatsDAO = new SeatsDAO();
        %>
        <% if (todayRes.isEmpty()) { %>
        <p style="color:#757575;">本日の予約はありません。</p>
        <% } else { %>
        <table class="table">
            <thead><tr><th>日付</th><th>行先</th><th>座席</th><th>メモ</th></tr></thead>
            <tbody>
            <% for (Reservation r : todayRes) {
                Seat seat = r.getSeatId() > 0 ? seatsDAO.findById(r.getSeatId()) : null;
                String seatLabel = seat != null ? seat.getBaseName()+" "+seat.getFName()+" "+seat.getSeatName() : "-";
                %>
                <tr>
                    <td><%= r.getReserveDate() %></td>
                    <td><span class="badge badge-green"><%= r.getGyosaki() %></span></td>
                    <td><%= seatLabel %></td>
                    <td><%= r.getMemo() != null ? r.getMemo() : "" %></td>
                </tr>
            <% } %>
            </tbody>
        </table>
        <% } %>
    </div>

    <!-- 同じ部署の出社状況 -->
    <%
    @SuppressWarnings("unchecked")
    List<Map<String, Object>> deptSeatList =
        (List<Map<String, Object>>) request.getAttribute("deptSeatList");
    %>
    <% if (request.getAttribute("deptMemberCount") != null) { %>
    <div class="card">
        <div style="display:flex; justify-content:space-between; align-items:center;">
            <h2 style="margin:0;">同じ部署の本日の出社状況</h2>
            <button type="button" id="deptSeatToggleBtn" class="btn btn-secondary" style="font-size:13px; padding:6px 14px;"
                onclick="toggleDeptSeatList()">▶ 表示する</button>
        </div>
        <p id="deptSeatHint" style="color:#757575; font-size:13px; margin:8px 0 0 0;">
            「表示する」を押すと、同じ部署のメンバーが本日どこにいるか（出社人数・座席など）を確認できます。
        </p>
    </div>
    <% } %>
    </div>
</div>
```



```

</p>
<div id="deptSeatListBody" style="display:none; margin-top:10px;">
  <div class="summary-row">
    <span class="badge badge-green">本日出勤 : <%= request.getAttribute("deptOfficeCount") %> 名</span>
    <span class="badge badge-gray">部署人数 : <%= request.getAttribute("deptMemberCount") %> 名</span>
  </div>
  <% if (deptSeatList != null && !deptSeatList.isEmpty()) { %>
  <table class="table" style="margin-top:10px;">
    <thead><tr><th>氏名</th><th>本日の状況</th><th>拠点</th><th>座席</th></tr></thead>
    <tbody>
      <% for (Map<String, Object> row : deptSeatList) {
        Seat seat = (Seat) row.get("seat");
        String status = (String) row.get("todayStatus");
        String baseLabel = seat != null ? seat.getBaseName() : "-";
        String seatLabel = seat != null
          ? seat.getFName() + " " + seat.getAreaName() + " " + seat.getSeatName()
          : ("出社".equals(status) ? "座席未定" : "-");
        String statusBadge = "出社".equals(status) ? "badge-green"
          : "在宅".equals(status) ? "badge-blue"
          : "出張".equals(status) ? "badge-orange"
          : "badge-gray";

        %>
        <tr>
          <td><%= row.get("name") %></td>
          <td><span class="badge <%= statusBadge %>"><%= status %></span></td>
          <td><%= baseLabel %></td>
          <td><%= seatLabel %></td>
        </tr>
      <% } %>
    </tbody>
  </table>
  <% } %>
</div>
</div>
<script>
  // 管理者メニューを開いた直後は隠しておき、必要なときだけボタンで開けるようにする
  function toggleDeptSeatList() {
    var body = document.getElementById('deptSeatListBody');
    var hint = document.getElementById('deptSeatHint');
    var btn = document.getElementById('deptSeatToggleBtn');
    var hidden = body.style.display === 'none';
    body.style.display = hidden ? '' : 'none';
    if (hint) hint.style.display = hidden ? 'none' : '';
    btn.textContent = hidden ? '▼ 隠す' : '▶ 表示する';
  }
</script>
<% } %>

<!-- 2週間分の予約状況（管理者のみ）：行をクリックすると氏名・座席の詳細が開く -->
<%
  List<Map<String, Object>> twoWeekSummary = (List<Map<String, Object>>) request.getAttribute("twoWeekSummary");
  %>
  <% if (twoWeekSummary != null) { %>
  <div class="card">
    <h2>今後2週間の予約状況</h2>
    <p style="font-size:12px; color:#757575; margin-bottom:8px;">日付をクリックすると、その日に出社する人の氏名と座席が確認できます。
  </p>
    <% for (Map<String, Object> row : twoWeekSummary) {
      @SuppressWarnings("unchecked")
      List<Map<String, Object>> people = (List<Map<String, Object>>) row.get("people");
    %>
    <details class="day-summary">
      <summary>
        <span class="day-date"><%= row.get("date") %></span>
        <span class="badge badge-green"><%= row.get("officeCount") %> 名出勤</span>
        <span class="badge badge-gray"><%= row.get("seatCount") %> 席予約済み</span>
        <span class="caret">&#9662;</span>
      </summary>
      <div class="day-detail">
        <% if (people == null || people.isEmpty()) { %>
        <p style="color:#9E9E9E; font-size:13px; margin:0;">座席が確定している予約はまだありません。</p>
        <% } else { %>

```

```

<table class="table" style="margin-top:4px;">
  <thead><tr><th>氏名</th><th>部署</th><th>拠点</th><th>座席</th></tr></thead>
  <tbody>
    <% for (Map<String, Object> p : people) {
      Seat seat = (Seat) p.get("seat");
      String baseLabel = seat != null ? seat.getBaseName() : "-";
      String seatLabel = seat != null
        ? seat.getFName() + " " + seat.getAreaName() + " " + seat.getSeatName()
        : "-";
    %>
    <tr>
      <td><%= p.get("name") %></td>
      <td><%= p.get("bName") %></td>
      <td><%= baseLabel %></td>
      <td><%= seatLabel %></td>
    </tr>
    <% } %>
  </tbody>
</table>
<% } %>
</div>
</details>
<% } %>
</div>
<% } %>

<!-- 一般機能 -->
<div class="card">
  <h2>一般機能</h2>
  <div class="menu-grid">
    <a href="<%= request.getContextPath() %>/ControlServlet?action=locationRegist" class="menu-btn">
      行先登録
    </a>
    <a href="<%= request.getContextPath() %>/ControlServlet?action=reserveConfirm" class="menu-btn">
      予約確認
    </a>
    <a href="<%= request.getContextPath() %>/ControlServlet?action=nameSearch" class="menu-btn">
      氏名検索
    </a>
    <a href="<%= request.getContextPath() %>/ControlServlet?action=departmentSearch" class="menu-btn">
      部門検索
    </a>
  </div>
</div>

<!-- 管理機能 -->
<div class="card">
  <h2>管理機能</h2>
  <div class="menu-grid">
    <a href="<%= request.getContextPath() %>/ControlServlet?action=seatStatus" class="menu-btn">
      座席利用状況
    </a>
    <a href="<%= request.getContextPath() %>/ControlServlet?action=adminReserve" class="menu-btn">
      代理予約
    </a>
    <a href="<%= request.getContextPath() %>/ControlServlet?action=masterUpdate" class="menu-btn">
      マスタ更新
    </a>
  </div>
</div>

</div>
</body>
</html>

```

ここで新しく出てくるもの

- <details class="day-summary"> <summary> ... </summary> <div class="day-detail"> ... </div> </details> ...
 ■ 新しいHTMLタグ：<details> / <summary>。JavaScriptを書かなくても「クリックすると開閉する」UIをブラウザ標準機能だけで作れるタグ。<summary>の中身が常に見えるタイトル部分、<details>の残りの中身（<div class="day-detail">）が開いたときだけ見える部分になる。twoWeekSummaryの14日分、それぞれが1つの<details>になっている。

- `String seatLabel = seat != null ? ... : ("出社".equals(status) ? "座席未定" : "-");` … 三項演算子のネスト（入れ子）。「座席情報が無い」場合に、さらに「行先が“出社”なのに座席が無い（＝座席未定のまま）」のか「在宅・出張で座席自体が不要」なのかで表示を分けている。一見複雑だが、`A ? B : (C ? D : E)` の形は「`A` が真なら `B`、そうでなければ（`C` が真なら `D`、そうでなければ `E`）」と外側から読むと理解しやすい。
- `String statusBadge = "出社".equals(status) ? "badge-green" : "在宅".equals(status) ? "badge-blue" : "出張".equals(status) ? "badge-orange" : "badge-gray";` … 三項演算子を**4つ連続でつなげる**ことで、「行先の文字列によってバッジの色（CSSクラス名）を変える」という、簡易的な `switch` の代わりのような書き方。読みにくく感じる場合は、`出社→green`、`在宅→blue`、`出張→orange`、`それ以外（未登録）→gray`、と対応表のように読むとよい。

読み方のポイント

- `adminMenu.jsp` は `menu.jsp` に「同じ部署の出社状況（開閉式）」「2週間分の予約状況サマリ」「管理機能カード」を追加した構成。`menu.jsp` を先に読んでいれば、共通部分（本日の予約・一般機能カード）は同じコードであることがすぐ分かる。
- `deptSeatList` を使う部分は `menu.jsp` にも存在するが、`adminMenu.jsp` では**初期状態で非表示**（`style="display:none;"`）になっており、ボタンを押すと表示される。これは、管理者メニューは情報量が多くなりがちなため、「最初は折りたたんでおき、必要なときだけ開く」というUI上の工夫。`menu.jsp` の方は表示/非表示を覚える `localStorage`、`adminMenu.jsp` の方は開いた状態を覚えない、という違いも、画面の使い方（毎日見るか、たまに見るか）の違いを反映していると考えられる。

3-6-5. `webapp/WEB-INF/jsp/seatStatus.jsp` + `control/ControlServlet.java` の `showSeatStatus()`

座席利用状況確認画面。「拠点 → フロア → エリア → 日付」の順に絞り込み、選んだ場所・日付の座席マップと、使用中座席の一覧を表示する。管理者メニューからのみ入れる。

```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.*, entity.Seat, dao.SeatsDAO" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>座席利用状況 - 座・Navi</title>
<link rel="stylesheet" href="%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/headerAdmin.jspf" %>

<div class="container">
    <div class="card">
        <h2>座席利用状況確認</h2>

        <%
            List<String> bases = (List<String>) request.getAttribute("bases");
            List<String> floors = (List<String>) request.getAttribute("floors");
            List<String> areas = (List<String>) request.getAttribute("areas");
            String filterBase = (String) request.getAttribute("filterBase");
            String filterFloor = (String) request.getAttribute("filterFloor");
            String filterArea = (String) request.getAttribute("filterArea");
            String filterDate = (String) request.getAttribute("filterDate");
            LinkedHashMap<String, String> dateOptions = (LinkedHashMap<String, String>) request.getAttribute("dateOptions");
            String ctx = request.getContextPath();

            String myDeptName = (String) request.getAttribute("myDeptName");
            @SuppressWarnings("unchecked")
            List<Map<String, Object>> deptTodayList = (List<Map<String, Object>>) request.getAttribute("deptTodayList");
        %>

        <!-- 同じ部署の本日の出社者一覧（自分の部署のメンバーが今日どこにいるかをひと目で確認） -->
        <% if (myDeptName != null) { %>
        <div style="background:#F5F5F5; border-radius:8px; padding:12px 16px; margin-bottom:16px;">
            <strong style="font-size:13px; color:#424242;"><%= myDeptName %>の本日の出社者</strong>
            <% if (deptTodayList == null || deptTodayList.isEmpty()) { %>
            <p style="color:#9E9E9E; font-size:13px; margin:6px 0 0 0;">本日「出社」の予定があるメンバーはいません。</p>
            <% } else { %>
            <div class="summary-row" style="margin-top:8px; gap:8px;">
                <% for (Map<String, Object> row : deptTodayList) {
                    Seat dSeat = (Seat) row.get("seat");
                    String dSeatLabel = dSeat != null
                        ? dSeat.getBaseName() + " " + dSeat.getFName() + " " + dSeat.getAreaName() + " " + dSeat.getSeatName()
                        : "座席未定";

                    %>
                    <span class="badge badge-gray"><%= row.get("name") %> <span style="color:#00695C;">(<%= dSeatLabel %>)</span></span>
                    <% } %>
                </div>
                <% } %>
            </div>
        <% } %>

        <p style="font-size:12px; color:#757575; margin-bottom:10px;">
            「〇〇オフィスの〇〇エリア」のように、拠点 → フロア → エリアの順に具体的な場所を選んで確認します。
        </p>

        <!-- 絞り込みフォーム（拠点 → フロア → エリアの順に選択。前の項目を選ぶまで次は選べない） -->
        <form action="%= ctx %>/ControlServlet" method="get">
            <input type="hidden" name="action" value="doSeatStatus">
            <div class="filter-row">
                <div class="form-group">
                    <label>① 拠点</label>
                    <select name="filterBase" onchange="this.form.submit()">
                        <option value="">-- 拠点を選択 --</option>
                        <% if (bases != null) for (String b : bases) { %>
                        <option value="%= b %>" <%= b.equals(filterBase) ? "selected" : "" %>><%= b %></option>
                        <% } %>
                    </select>
                </div>
                <div class="form-group">

```

```

<label>② フロア</label>
<select name="filterFloor" onchange="this.form.submit()" <%= (floors == null) ? "disabled" : "" %>>
  <option value="">-- フロアを選択 --</option>
  <% if (floors != null) for (String f : floors) { %>
    <option value="<%= f %>" <%= f.equals(filterFloor) ? "selected" : "" %>><%= f %></option>
  <% } %>
</select>
</div>
<div class="form-group">
  <label>③ エリア</label>
  <select name="filterArea" onchange="this.form.submit()" <%= (areas == null) ? "disabled" : "" %>>
    <option value="">-- エリアを選択 --</option>
    <% if (areas != null) for (String a : areas) { %>
      <option value="<%= a %>" <%= a.equals(filterArea) ? "selected" : "" %>><%= a %></option>
    <% } %>
  </select>
</div>
<div class="form-group">
  <label>④ 日付 (2週間前～2週間後まで確認できます) </label>
  <select name="filterDate" onchange="this.form.submit()">
    <% if (dateOptions != null) for (Map.Entry<String, String> e : dateOptions.entrySet()) { %>
      <option value="<%= e.getKey() %>" <%= e.getKey().equals(filterDate) ? "selected" : "" %>><%= e.getValue() %></option>
    <% } %>
  </select>
</div>
</div>
</form>

<!-- 拠点・フロアごとのフロアマップ画像（選ぶと該当の画像に切り替わる） -->
<%
  String officeImage = (filterBase != null && !filterBase.isEmpty())
    ? new SeatsDAO().getOfficeImage(filterBase, filterFloor) : null;
%>
<% if (officeImage != null) { %>
<div style="margin:16px 0;">
  のフロアマップ"
    style="max-width:100%; border:1px solid #ECEFF1; border-radius:8px;">
</div>
<% } %>

<%
  List<Seat> filteredSeats = (List<Seat>) request.getAttribute("filteredSeats");
  Map<Integer, String> seatUsageMap = (Map<Integer, String>) request.getAttribute("seatUsageMap");
  Map<Integer, String> seatUserNameMap = (Map<Integer, String>) request.getAttribute("seatUserNameMap");
  Map<Integer, String> seatUserDeptMap = (Map<Integer, String>) request.getAttribute("seatUserDeptMap");
  Object usedObj = request.getAttribute("usedCount");
  Object freeObj = request.getAttribute("freeCount");
  long usedCount = usedObj instanceof Long ? (Long)usedObj : (usedObj != null ? ((Integer)usedObj).longValue() : 0);
  long freeCount = freeObj instanceof Long ? (Long)freeObj : (freeObj != null ? ((Integer)freeObj).longValue() : 0);
%>

<% if (filterArea != null && !filterArea.isEmpty()) { %>
<!-- 結果見出し：常に「〇〇オフィスの〇〇 エリア」のように具体的な場所を一意に表示する -->
<h3 style="margin:18px 0 10px; color:#00695C;">
  <%= filterBase %>の<%= filterFloor %> <%= filterArea %>
</h3>

<!-- サマリ -->
<div class="summary-row" style="margin-bottom:16px;">
  <span class="badge badge-gray">対象 <%= filteredSeats.size() %> 席</span>
  <span class="badge badge-red">使用中 <%= usedCount %> 席</span>
  <span class="badge badge-green">空き <%= freeCount %> 席</span>
</div>

<!-- 凡例 -->
<div class="summary-row" style="font-size:12px;">
  <span class="badge badge-green">■ 空き</span>
  <span class="badge badge-red">■ 使用中</span>
</div>

<!--

```

座席マップ：席のボタンは76x54pxと小さく「氏名（部署名）」を入れると見切れてしまうため、ここでは座席名と状態だけを示すミニマップに留め、カーソルを合わせると title 属性で「部署名 氏名」をツールチップ表示する（誰がどこにいるかの全体像をひと目で把握する用途）。詳しい内訳は、すぐ下の「使用中の座席一覧」で省略なしに確認できる。

```
-->
<div class="seat-grid">
  <% for (Seat s : filteredSeats) {
    boolean occupied = seatUsageMap != null && seatUsageMap.containsKey(s.getSeatId());
    String uname = (seatUserNameMap != null && seatUserNameMap.containsKey(s.getSeatId()))
      ? seatUserNameMap.get(s.getSeatId()) : "";
    String udept = (seatUserDeptMap != null && seatUserDeptMap.containsKey(s.getSeatId()))
      ? seatUserDeptMap.get(s.getSeatId()) : "";
    String hoverInfo = occupied ? ((udept != null && !udept.isEmpty()) ? (udept + " " + uname) : uname) : "空き";
  %>
    <div class="seat-btn <%= occupied ? "occupied" : "available" %>" style="cursor:default;" title="<%= hoverInfo %>">
      <span class="seat-id"><%= s.getSeatName() %></span>
      <span class="seat-user"><%= occupied ? "使用中" : "空き" %></span>
    </div>
  <% } %>
</div>

<!-- 使用中の座席一覧：誰がどの座席を使っているかを、省略せず一覧で確認できるようにする -->
<% if (usedCount > 0) { %>
<h4 style="margin:22px 0 8px; color:#424242; font-size:14px;">使用中の座席一覧（<%= usedCount %> 件）</h4>
<table class="table" style="margin-bottom:8px;">
  <thead>
    <tr><th style="width:140px;">座席</th><th>部署</th><th>氏名</th></tr>
  </thead>
  <tbody>
    <% for (Seat s : filteredSeats) {
      if (seatUsageMap == null || !seatUsageMap.containsKey(s.getSeatId())) continue;
      String uname = (seatUserNameMap != null && seatUserNameMap.containsKey(s.getSeatId()))
        ? seatUserNameMap.get(s.getSeatId()) : "";
      String udept = (seatUserDeptMap != null && seatUserDeptMap.containsKey(s.getSeatId()))
        ? seatUserDeptMap.get(s.getSeatId()) : "";
    %>
    <tr>
      <td><span class="badge badge-red" style="white-space:nowrap;"><%= s.getSeatName() %></span></td>
      <td><%= udept %></td>
      <td><%= uname %></td>
    </tr>
    <% } %>
  </tbody>
</table>
<% } %>
<% } else { %>
<!-- 拠点・フロア・エリアがすべて選ばれるまでは、案内メッセージのみ表示する -->
<p style="color:#9E9E9E; padding:24px 0; text-align:center;">
  拠点・フロア・エリアを順番に選択すると、その場所の座席利用状況が表示されます。
</p>
<% } %>

<div style="margin-top:24px;">
  <a href="<%= ctx %>/ControlServlet?action=adminMenu" class="btn btn-secondary">&larr; 管理メニューへ戻る</a>
</div>
</div>
</div>
</body>
</html>
```

このJSPを描画するために、ControlServlet 側で値を準備しているのが showSeatStatus() メソッド。

```
// ----- 座席利用状況 -----

private String showSeatStatus(HttpServletRequest req) {
    SeatsDAO seatsDAO = new SeatsDAO();
    ReservationsDAO resDAO = new ReservationsDAO();
    AccountsDAO accDAO = new AccountsDAO();
    dao.LocationsDAO locDAO = new dao.LocationsDAO();

    String filterBase = req.getParameter("filterBase");
    String filterFloor = req.getParameter("filterFloor");
    String filterArea = req.getParameter("filterArea");
    String filterDate = req.getParameter("filterDate");
    if (filterDate == null || filterDate.isEmpty()) filterDate = LocalDate.now().toString();

    // 座席利用状況の確認は「今日を基準に2週間前～2週間後」までに限定する
    // (カレンダーで自由に選べしまうと際限なく過去・未来を確認できてしまうため)
    LocalDate today = LocalDate.now();
    LocalDate minDate = today.minusDays(14);
    LocalDate maxDate = today.plusDays(14);
    LocalDate parsedDate;
    try {
        parsedDate = LocalDate.parse(filterDate);
    } catch (Exception e) {
        parsedDate = today;
    }
    if (parsedDate.isBefore(minDate)) parsedDate = minDate;
    if (parsedDate.isAfter(maxDate)) parsedDate = maxDate;
    filterDate = parsedDate.toString();
    req.setAttribute("minDate", minDate.toString());
    req.setAttribute("maxDate", maxDate.toString());

    // 日付はカレンダーUIだと押しづらいとの声があったため、他の項目と同じ
    // セレクトボックス形式(2週間前～2週間後を一覧)で選べるようにする
    String[] youbi = {"日", "月", "火", "水", "木", "金", "土"};
    LinkedHashMap<String, String> dateOptions = new LinkedHashMap<>();
    for (LocalDate d = minDate; !d.isAfter(maxDate); d = d.plusDays(1)) {
        String label = d.getMonthValue() + "/" + d.getDayOfMonth()
            + " (" + youbi[d.getDayOfWeek().getValue() % 7] + ") "
            + (d.equals(today) ? "※本日" : "");
        dateOptions.put(d.toString(), label);
    }
    req.setAttribute("dateOptions", dateOptions);

    // ドロップダウン用リスト (「すべて」は用意せず、拠点→フロア→エリアの順に選んでもらう)
    req.setAttribute("bases", seatsDAO.getBases());
    boolean baseChosen = filterBase != null && !filterBase.isEmpty();
    boolean floorChosen = baseChosen && filterFloor != null && !filterFloor.isEmpty();
    boolean areaChosen = floorChosen && filterArea != null && !filterArea.isEmpty();

    if (baseChosen) req.setAttribute("floors", seatsDAO.getFloors(filterBase));
    if (floorChosen) req.setAttribute("areas", seatsDAO.getAreas(filterBase, filterFloor));

    // 拠点・フロア・エリアがすべて決まったときだけ座席を表示する
    // (「〇〇オフィスの〇〇エリア」のように、常に具体的な場所が一意に決まる状態にするため)
    List<Seat> filteredSeats = areaChosen
        ? seatsDAO.filterSeats(filterBase, filterFloor, filterArea)
        : new ArrayList<>();

    // 指定日の利用状況
    Map<Integer, String> seatUsageMap = resDAO.getSeatUsageForDate(filterDate);

    // 使用中ユーザーの「氏名」「部署名」マップ (座席マップ上で氏名の横に部署名も出すため)
    Map<Integer, String> seatUserNameMap = new HashMap<>();
    Map<Integer, String> seatUserDeptMap = new HashMap<>();
    for (Map.Entry<Integer, String> e : seatUsageMap.entrySet()) {
        entity.Account acc = accDAO.findById(e.getValue());
        seatUserNameMap.put(e.getKey(), acc != null ? acc.getName() : e.getValue());
        seatUserDeptMap.put(e.getKey(), acc != null ? locDAO.getBNameById(acc.getBId()) : "");
    }

    long usedCount = filteredSeats.stream()

```

```

        .filter(s -> seatUsageMap.containsKey(s.getSeatId()))).count());
long freeCount = filteredSeats.size() - usedCount;

// ログイン中の管理者と同じ部署で、本日「出社」する人の一覧（氏名・座席）
// ※座席利用状況の確認画面で「自分の部署のメンバーが今日どこに座っているか」をすぐ把握できるようにするため
HttpSession statusSession = req.getSession(false);
String myUserId = (String) statusSession.getAttribute("userId");
Account me = accDAO.findById(myUserId);
List<Map<String, Object>> deptTodayList = new ArrayList<>();
if (me != null) {
    for (Account a : accDAO.findById(me.getBId())) {
        List<Reservation> mRes = resDAO.findTodayByUserId(a.getUserId());
        if (mRes.isEmpty() || !"出社".equals(mRes.get(0).getGyosaki())) continue;
        Reservation r = mRes.get(0);
        Seat seat = r.getSeatId() > 0 ? seatsDAO.findById(r.getSeatId()) : null;
        Map<String, Object> row = new HashMap<>();
        row.put("name", a.getName());
        row.put("seat", seat);
        deptTodayList.add(row);
    }
}

req.setAttribute("filteredSeats", filteredSeats);
req.setAttribute("seatUsageMap", seatUsageMap);
req.setAttribute("seatUserNameMap", seatUserNameMap);
req.setAttribute("seatUserDeptMap", seatUserDeptMap);
req.setAttribute("usedCount", usedCount);
req.setAttribute("freeCount", freeCount);
req.setAttribute("filterBase", filterBase);
req.setAttribute("filterFloor", filterFloor);
req.setAttribute("filterArea", filterArea);
req.setAttribute("filterDate", filterDate);
req.setAttribute("myDeptName", me != null ? locDAO.getBNameByBId(me.getBId()) : null);
req.setAttribute("deptTodayList", deptTodayList);

return "/WEB-INF/jsp/seatStatus.jsp";
}

```

ここで新しく出てくるもの

- `LinkedHashMap<String, String> dateOptions = new LinkedHashMap<>();` … ■ 新しい型: **LinkedHashMap**。これまで出てきた `HashMap` は「順番を保証しない」マップだが、`LinkedHashMap` は**「追加した順番」を覚えている**マップ。ここでは「2週間前 → 今日 → 2週間後」の順に1日ずつ `put` しているので、JSP側で `for (Map.Entry<...> e : dateOptions.entrySet())` とループするとそのまま日付順のセレクトボックスになる。「中身を検索する」目的なら `HashMap`、「中身を順番に取り出したい」目的なら `LinkedHashMap`、という使い分けの基準になる。
- `String[] youbi = {"日", "月", "火", "水", "木", "金", "土"};` … ■ 新しい書き方: **配列（はいれつ）リテラル**。`List` と違って、要素数が固定で変わらない値（曜日の名前など）をまとめて持つときに使う。`youbi[d.getDayOfWeek().getValue() % 7]` で曜日を取り出している。`getDayOfWeek().getValue()` は月曜=1〜日曜=7を返すため、`% 7`（7で割った余り）で日曜だけ `7→0` に変換し、配列の添字（0〜6）に合わせている。
- `for (LocalDate d = minDate; !d.isAfter(maxDate); d = d.plusDays(1))` … `LocalDate` を1日ずつ進めながらループする書き方。`d.plusDays(1)` で「1日後の日付」を新しく作り、`!d.isAfter(maxDate)`（`d` が `maxDate` より後ではない = `d <= maxDate`）の間繰り返す。`isBefore()` / `isAfter()` はどちらも「自分より後／前か」を `boolean` で返すメソッドで、日付の前後比較によく使う。
- `if (parsedDate.isBefore(minDate)) parsedDate = minDate; if (parsedDate.isAfter(maxDate)) parsedDate = maxDate;` … 値を許される範囲（`minDate` ～ `maxDate`）に収める「クランプ（clamp）」処理。URLパラメータで範囲外の日付を指定されても、自動的に範囲内の端の値に補正される。
- `<select name="filterFloor" onchange="this.form.submit()" <%= (floors == null) ? "disabled" : "" %>>` … `disabled` 属性を条件式で出し分けることで、「拠点を選ぶ前はフロアのプルダウンを操作不能にする」という段階的のUIを実現している。`onchange="this.form.submit()"` は、選択肢を変えた瞬間にそのフォームを自動送信する書き方（Day4の `reserve.jsp` でも同じ仕組みが使われている）。

読み方のポイント

- `seatStatus.jsp` の「拠点→フロア→エリア」の段階的な絞り込みは、Day4の `reserve.jsp`（一般ユーザー向けの座席予約フロー）と発想が同じ。違いは、`reserve.jsp` がセッションに状態を保存して複数ページに分けていたのに対し、`seatStatus.jsp` は1ページの中で `<select onchange="this.form.submit()">` を使い、URLパラメータ（`filterBase` / `filterFloor` / `filterArea` / `filterDate`）だけで状態を表現している点。どちらも「全部選が終わるまでは結果を表示しない」という設計は共通している。
- `showSeatStatus()` の中の「使用中ユーザーの氏名・部署名マップを作る」部分（`seatUserNameMap` / `seatUserDeptMap`）は、Day4の `ReserveLogic` の `seat` ステージで出てきたコードとほぼ同じ。同じ目的のコードが複数箇所に存在している = 「共通化できそうな箇所」の例

として、チームで議論する余地がある。

3-6-6. webapp/WEB-INF/jsp/adminReserve.jsp

代理予約画面。AdminReserveLogic（3-6-3）が用意したデータをもとに、「① 予約タイプ選択 → ② 対象者検索・追加 → ③ 日付と場所を選択 → ④ 座席を選択」の4ステップを1ページにまとめている。JavaScriptの量が多すぎるファイルで、ページ全体を再読み込みすることなく「対象者の検索・追加・削除」や「座席の選択」をその場で行えるようにしている。

```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.*, entity.Seat" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>代理予約 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
<style>
.mode-card {
    flex: 1; border: 2px solid #ECEFF1; border-radius: 8px; padding: 14px;
    text-align: center; cursor: pointer; transition: border-color .15s, background-color .15s;
}
.mode-card.active { border-color: #00897B; background-color: #E0F2F1; }
.mode-card h4 { margin: 4px 0; color: #424242; }
.mode-card p { margin: 0; font-size: 12px; color: #757575; }
.step-section { margin-top: 24px; padding-top: 20px; border-top: 1px solid #ECEFF1; }
.step-section h3 { color: #00695C; margin-bottom: 10px; }
</style>
</head>
<body>
<%@ include file="common/headerAdmin.jspf" %>

<div class="container">
    <div class="card">
        <h2>代理予約</h2>
        <p style="font-size:12px; color:#757575;">
            個人の代理予約、または会議で複数人がまとめて出社する場合の代理予約を行います。
        </p>

        <%
            @SuppressWarnings("unchecked")
            List<Map<String, Object>> accountRows = (List<Map<String, Object>>) request.getAttribute("accountRows");
            List<String> bases = (List<String>) request.getAttribute("bases");
            List<String> floors = (List<String>) request.getAttribute("floors");
            List<String> areas = (List<String>) request.getAttribute("areas");
            List<String> dates = (List<String>) request.getAttribute("dates");
            List<Seat> areaSeats = (List<Seat>) request.getAttribute("areaSeats");
            Map<Integer, String> seatUsage = (Map<Integer, String>) request.getAttribute("seatUsage");
            Map<Integer, String> seatUserNameMap = (Map<Integer, String>) request.getAttribute("seatUserNameMap");
            Map<Integer, String> seatUserDeptMap = (Map<Integer, String>) request.getAttribute("seatUserDeptMap");
            String selBase = (String) request.getAttribute("selBase");
            String selFloor = (String) request.getAttribute("selFloor");
            String selArea = (String) request.getAttribute("selArea");
            String selDate = (String) request.getAttribute("selDate");
            String ctx = request.getContextPath();
        %>

        <!-- ① 予約タイプを選択 -->
        <div class="step-section" style="margin-top:8px; padding-top:0; border-top:none;">
            <h3>① 予約タイプを選択</h3>
            <div style="display:flex; gap:12px;">
                <div class="mode-card" id="modeCardIndividual" onclick="setMode('individual')">
                    <h4>個人の代理予約</h4>
                    <p>1人分の出社予約を代理で登録します。</p>
                </div>
                <div class="mode-card" id="modeCardMeeting" onclick="setMode('meeting')">
                    <h4>会議でまとめて予約</h4>
                    <p>複数人を選び、まとめて座席を割り当てます。</p>
                </div>
            </div>
        </div>
    </div>

    <form id="reserveForm" action="<%= ctx %>/ControlServlet" method="post" onsubmit="return confirmAdminReserve();">
        <input type="hidden" name="action" value="doAdminReserve">
        <input type="hidden" name="mode" id="modeInput" value="">

        <%
            String searchDeptIndividual = (String) request.getAttribute("searchDeptIndividual");
            String searchNameIndividual = (String) request.getAttribute("searchNameIndividual");
            String searchIdIndividual = (String) request.getAttribute("searchIdIndividual");

```

```

String searchDeptMeeting = (String) request.getAttribute("searchDeptMeeting");
String searchNameMeeting = (String) request.getAttribute("searchNameMeeting");
String searchIdMeeting = (String) request.getAttribute("searchIdMeeting");
String selMode = (String) request.getAttribute("selMode");
String selTargetUserId = (String) request.getAttribute("selTargetUserId");
@SuppressWarnings("unchecked")
List<String> selTargetUserIds = (List<String>) request.getAttribute("selTargetUserIds");
%>

<!-- ② 対象社員を検索して追加（個人） -->
<div class="step-section" id="sectionIndividual">
  <h3>② 対象社員を検索して追加</h3>
  <div class="filter-row">
    <div class="form-group">
      <label>部署で検索</label>
      <input type="text" id="searchDeptIndividual" value="%= searchDeptIndividual != null ? searchDeptIndividual : "" %
>" placeholder="部署名の一部">
    </div>
    <div class="form-group">
      <label>氏名で検索</label>
      <input type="text" id="searchNameIndividual" value="%= searchNameIndividual != null ? searchNameIndividual : "" %
>" placeholder="氏名の一部">
    </div>
    <div class="form-group">
      <label>社員番号で検索</label>
      <input type="text" id="searchIdIndividual" value="%= searchIdIndividual != null ? searchIdIndividual : "" %>" plac
eholder="社員番号の一部">
    </div>
    <div class="form-group" style="align-self: flex-end;">
      <button type="button" class="btn btn-secondary" onclick="searchAccounts('individual')">検索</button>
    </div>
  </div>
  <div id="searchResultIndividual" style="margin-top: 8px;"></div>
  <div id="selectedIndividual" style="margin-top: 12px;"></div>
  <div id="hiddenIndividual"></div>
</div>

<!-- ② 参加者を検索して追加（複数可・会議モード） -->
<div class="step-section" id="sectionMeeting" style="display: none;">
  <h3>② 参加者を検索して追加（複数可）</h3>
  <div class="filter-row">
    <div class="form-group">
      <label>部署で検索</label>
      <input type="text" id="searchDeptMeeting" value="%= searchDeptMeeting != null ? searchDeptMeeting : "" %>" placeho
lder="部署名の一部">
    </div>
    <div class="form-group">
      <label>氏名で検索</label>
      <input type="text" id="searchNameMeeting" value="%= searchNameMeeting != null ? searchNameMeeting : "" %>" placeho
lder="氏名の一部">
    </div>
    <div class="form-group">
      <label>社員番号で検索</label>
      <input type="text" id="searchIdMeeting" value="%= searchIdMeeting != null ? searchIdMeeting : "" %>" placeholder
="社員番号の一部">
    </div>
    <div class="form-group" style="align-self: flex-end;">
      <button type="button" class="btn btn-secondary" onclick="searchAccounts('meeting')">検索</button>
    </div>
  </div>
  <div id="searchResultMeeting" style="margin-top: 8px;"></div>
  <div id="selectedMeeting" style="margin-top: 12px;"></div>
  <div id="hiddenMeeting"></div>
</div>

<!-- ③ 日付と場所を選択 -->
<div class="step-section">
  <h3>③ 日付と場所を選択</h3>
  <div class="filter-row">
    <div class="form-group">
      <label>日付</label>
      <select onchange="reloadWith('date', this.value)">

```

```

        <% if (dates != null) for (String d : dates) { %>
        <option value="<%= d %>" <%= d.equals(selDate) ? "selected" : "" %>><%= d %></option>
        <% } %>
    </select>
</div>
<div class="form-group">
    <label>拠点</label>
    <select onchange="reloadWith('base', this.value)">
        <option value="">-- 拠点を選択 --</option>
        <% if (bases != null) for (String b : bases) { %>
        <option value="<%= b %>" <%= b.equals(selBase) ? "selected" : "" %>><%= b %></option>
        <% } %>
    </select>
</div>
<div class="form-group">
    <label>フロア</label>
    <select onchange="reloadWith('floor', this.value)" <%= (floors == null) ? "disabled" : "" %>>
        <option value="">-- フロアを選択 --</option>
        <% if (floors != null) for (String f : floors) { %>
        <option value="<%= f %>" <%= f.equals(selFloor) ? "selected" : "" %>><%= f %></option>
        <% } %>
    </select>
</div>
<div class="form-group">
    <label>エリア</label>
    <select onchange="reloadWith('area', this.value)" <%= (areas == null) ? "disabled" : "" %>>
        <option value="">-- エリアを選択 --</option>
        <% if (areas != null) for (String a : areas) { %>
        <option value="<%= a %>" <%= a.equals(selArea) ? "selected" : "" %>><%= a %></option>
        <% } %>
    </select>
</div>
</div>

<%
String dispImage = null;
String dispLabel = null;
if (selBase != null && selFloor != null && selArea != null) {
    dispImage = new dao.SeatsDAO().getExpectedAreaImage(selBase, selFloor, selArea);
    dispLabel = selBase + "の" + selFloor + " " + selArea;
} else if (selBase != null && selFloor != null) {
    dispImage = new dao.SeatsDAO().getOfficeImage(selBase, selFloor);
    dispLabel = selBase + "の" + selFloor;
}
%>
<% if (dispImage != null) { %>
<div style="margin:12px 0;">
    <p style="font-size:12px; color:#757575; margin-bottom:4px;"><%= dispLabel %>のマップ</p>
    "
        style="max-width:100%; border:1px solid #ECEFF1; border-radius:8px;"
        onerror="this.src='<%= ctx %>/images/no_image.png'">
</div>
<% } %>
</div>

<!-- ④ 座席を選択 -->
<% if (selArea != null && areaSeats != null) { %>
<div class="step-section">
    <h3>④ 座席を選択</h3>
    <div class="seat-grid">
        <% for (Seat s : areaSeats) {
            boolean occupied = seatUsage != null && seatUsage.containsKey(s.getSeatId());
            String uname = (seatUserNameMap != null && seatUserNameMap.containsKey(s.getSeatId()))
                ? seatUserNameMap.get(s.getSeatId()) : "";
            String udept = (seatUserDeptMap != null && seatUserDeptMap.containsKey(s.getSeatId()))
                ? seatUserDeptMap.get(s.getSeatId()) : "";
            String hoverInfo = occupied ? ((udept != null && !udept.isEmpty()) ? (udept + " " + uname) : uname) : "";
        %>
        <% if (occupied) { %>
        <div class="seat-btn occupied" style="cursor:default;" title="<%= hoverInfo %>">
            <span class="seat-id"><%= s.getSeatName() %></span>
            <span class="seat-user">使用中</span>

```

```

    </div>
    <% } else { %>
    <label class="seat-btn available" for="seat<%= s.getSeatId() %>">
      <input type="checkbox" name="seatIds" id="seat<%= s.getSeatId() %>" value="<%= s.getSeatId() %>"
        style="display:none;" onchange="toggleSeat(this)">
      <span class="seat-id"><%= s.getSeatName() %></span>
      <span class="seat-user">空き</span>
    </label>
    <% } %>
  <% } %>
</div>

<div class="form-group" style="margin-top:16px;">
  <label>メモ（任意）</label>
  <textarea name="memo" rows="2" style="width:100%;"></textarea>
</div>

  <button type="submit" class="btn btn-primary" style="margin-top:12px;">この内容で予約する</button>
</div>
<% } %>
</form>
</div>
</div>

<script>
// ― サーバー側で持っている社員一覧・座席一覧をJSの配列／オブジェクトに変換 ―
// ページを再読み込みせずに検索・絞り込みを行うため、最初に全データをJS側へ渡しておく
var ACCOUNTS = [
  <% if (accountRows != null) for (Map<String, Object> row : accountRows) { %>
  { userId: "<%= row.get("userId") %>", name: "<%= row.get("name") %>", bName: "<%= row.get("bName") %>" },
  <% } %>
];

var SEATS = {
  <% if (areaSeats != null) for (Seat s : areaSeats) { %>
  "<%= s.getSeatId() %>": "<%= s.getSeatName() %>",
  <% } %>
};

var selectedIndividual = null; // { userId, name, bName }
var selectedMeeting = []; // [{ userId, name, bName }, ...]

function findAccount(userId) {
  for (var i = 0; i < ACCOUNTS.length; i++) {
    if (ACCOUNTS[i].userId === userId) return ACCOUNTS[i];
  }
  return null;
}

// ページ再読み込み後も選択状態を復元する
(function restoreSelected() {
  <%
  boolean validIndividual = selTargetUserId != null && selTargetUserId.matches("\\d+");
  %>
  <% if (validIndividual) { %>
  selectedIndividual = findAccount("<%= selTargetUserId %>");
  <% } %>

  <% if (selTargetUserIds != null) for (String uid : selTargetUserIds) {
    if (uid != null && uid.matches("\\d+")) { %>
  (function(){ var a = findAccount("<%= uid %>"); if (a) selectedMeeting.push(a); })();
  <% }
  } %>
})();

function searchAccounts(mode) {
  var dept = document.getElementById("searchDept" + capitalize(mode)).value.trim();
  var name = document.getElementById("searchName" + capitalize(mode)).value.trim();
  var id = document.getElementById("searchId" + capitalize(mode)).value.trim();

  var result = ACCOUNTS.filter(function(a) {
    return (dept === "" || a.bName.indexOf(dept) !== -1)

```

```

    && (name === "" || a.name.indexOf(name) !== -1)
    && (id === "" || a.userId.indexOf(id) !== -1);
});

var container = document.getElementById("searchResult" + capitalize(mode));
container.innerHTML = "";

if (result.length === 0) {
    container.innerHTML = "<p style='color:#9E9E9E; font-size:13px;'>該当する社員が見つかりません。</p>";
    return;
}

var select = document.createElement("select");
result.forEach(function(a) {
    var opt = document.createElement("option");
    opt.value = a.userId;
    opt.textContent = a.name + " (" + a.bName + "/" + a.userId + ") ";
    select.appendChild(opt);
});

var addBtn = document.createElement("button");
addBtn.type = "button";
addBtn.className = "btn btn-secondary";
addBtn.textContent = "追加";
addBtn.style.marginLeft = "8px";
addBtn.onclick = function() {
    var userId = select.value;
    var account = findAccount(userId);
    if (account) addPerson(mode, account);
};

container.appendChild(select);
container.appendChild(addBtn);
}

function capitalize(s) {
    return s.charAt(0).toUpperCase() + s.slice(1);
}

function addPerson(mode, account) {
    if (mode === "individual") {
        selectedIndividual = account;
    } else {
        if (!selectedMeeting.some(function(a) { return a.userId === account.userId; })) {
            selectedMeeting.push(account);
        }
    }
    renderSelected(mode);
}

function removePerson(mode, userId) {
    if (mode === "individual") {
        selectedIndividual = null;
    } else {
        selectedMeeting = selectedMeeting.filter(function(a) { return a.userId !== userId; });
        // 参加者が減って選択中の座席数が参加者数を超えたら、超えた分の選択を解除する
        var checked = document.querySelectorAll('input[name="seatIds"]:checked');
        for (var i = checked.length - 1; i >= 0; i--) {
            if (i >= selectedMeeting.length) {
                checked[i].checked = false;
                toggleSeat(checked[i]);
            }
        }
    }
    renderSelected(mode);
}

function renderSelected(mode) {
    var people = mode === "individual" ? (selectedIndividual ? [selectedIndividual] : []) : selectedMeeting;
    var container = document.getElementById(mode === "individual" ? "selectedIndividual" : "selectedMeeting");
    var hidden = document.getElementById(mode === "individual" ? "hiddenIndividual" : "hiddenMeeting");

```

```

container.innerHTML = "";
hidden.innerHTML = "";

if (people.length === 0) {
    container.innerHTML = "<p style='color:#9E9E9E; font-size:13px;'>対象者が選択されていません。</p>";
    return;
}

var table = document.createElement("table");
table.className = "table";
var thead = document.createElement("thead");
thead.innerHTML = "<tr><th>氏名</th><th>部署</th><th>社員番号</th><th></th></tr>";
table.appendChild(thead);

var tbody = document.createElement("tbody");
people.forEach(function(a) {
    var tr = document.createElement("tr");
    var tdName = document.createElement("td"); tdName.textContent = a.name;
    var tdDept = document.createElement("td"); tdDept.textContent = a.bName;
    var tdId = document.createElement("td"); tdId.textContent = a.userId;
    var tdRemove = document.createElement("td");
    var removeLink = document.createElement("a");
    removeLink.href = "javascript:void(0)";
    removeLink.textContent = "取り消す";
    removeLink.onclick = function() { removePerson(mode, a.userId); };
    tdRemove.appendChild(removeLink);
    tr.appendChild(tdName); tr.appendChild(tdDept); tr.appendChild(tdId); tr.appendChild(tdRemove);
    tbody.appendChild(tr);

    var input = document.createElement("input");
    input.type = "hidden";
    input.name = mode === "individual" ? "targetUserId" : "targetUserIds";
    input.value = a.userId;
    hidden.appendChild(input);
});
table.appendChild(tbody);
container.appendChild(table);
}

function setMode(mode) {
    document.getElementById("modeInput").value = mode;
    var isIndividual = mode === "individual";
    document.getElementById("sectionIndividual").style.display = isIndividual ? "" : "none";
    document.getElementById("sectionMeeting").style.display = isIndividual ? "none" : "";
    document.getElementById("modeCardIndividual").classList.toggle("active", isIndividual);
    document.getElementById("modeCardMeeting").classList.toggle("active", !isIndividual);
}

// 初期表示・前回の選択状態の復元
setMode("<%= selMode != null ? selMode : 'individual' %>");
if (selectedIndividual) renderSelected("individual");
if (selectedMeeting.length > 0) renderSelected("meeting");

<% if (searchDeptIndividual != null || searchNameIndividual != null || searchIdIndividual != null) { %>
searchAccounts("individual");
<% } %>
<% if (searchDeptMeeting != null || searchNameMeeting != null || searchIdMeeting != null) { %>
searchAccounts("meeting");
<% } %>

function toggleSeat(cb) {
    var mode = document.getElementById("modeInput").value;
    var checked = document.querySelectorAll('input[name="seatIds"]:checked');

    if (mode === "individual") {
        // 個人モードは1席のみ選択可能。他のチェックを外す
        if (cb.checked) {
            checked.forEach(function(other) {
                if (other !== cb) {
                    other.checked = false;
                    other.closest("label").classList.remove("selected");
                    other.closest("label").classList.add("available");
                }
            });
        }
    }
}

```

```

    }
  });
}
} else {
  // 会議モードは参加者数を超える座席は選べない
  if (cb.checked && checked.length > selectedMeeting.length) {
    cb.checked = false;
    alert("選択できる座席数は参加者数 (" + selectedMeeting.length + "人) までです。");
    return;
  }
}

var label = cb.closest("label");
if (cb.checked) {
  label.classList.remove("available");
  label.classList.add("selected");
} else {
  label.classList.remove("selected");
  label.classList.add("available");
}
}
}

// ― スクロール位置の保持 ―
// select の onchange でページを再読み込みすると一覧の先頭に戻ってしまうため、
// 送信前のスクロール位置を保存し、再読み込み後に復元する
var ADMIN_RESERVE_SCROLL_KEY = "adminReserveScroll";

window.addEventListener("beforeunload", function() {
  sessionStorage.setItem(ADMIN_RESERVE_SCROLL_KEY, window.scrollY);
});

window.addEventListener("load", function() {
  var y = sessionStorage.getItem(ADMIN_RESERVE_SCROLL_KEY);
  if (y !== null) {
    window.scrollTo(0, parseInt(y, 10));
    sessionStorage.removeItem(ADMIN_RESERVE_SCROLL_KEY);
  }
});

function confirmAdminReserve() {
  var mode = document.getElementById("modeInput").value;
  var people = mode === "individual" ? (selectedIndividual ? [selectedIndividual] : []) : selectedMeeting;
  if (people.length === 0) {
    alert("対象者が選択されていません。");
    return false;
  }
  var checked = document.querySelectorAll('input[name="seatIds"]:checked');
  if (checked.length === 0) {
    alert("座席が選択されていません。");
    return false;
  }

  var names = people.map(function(a) { return a.name; }).join(", ");
  var seatNames = Array.prototype.map.call(checked, function(cb) { return SEATS[cb.value] || cb.value; }).join(", ");
  return confirm(names + " を、座席「" + seatNames + "」で予約します。よろしいですか?");
}

function reloadWith(paramName, value) {
  var form = document.createElement("form");
  form.method = "post";
  form.action = "<%= ctx %>/ControlServlet";

  function addField(name, value) {
    var input = document.createElement("input");
    input.type = "hidden";
    input.name = name;
    input.value = value;
    form.appendChild(input);
  }

  addField("action", "adminReserve");
  addField("mode", document.getElementById("modeInput").value || "<%= selMode != null ? selMode : 'individual' %>");
}

```



```

addField("date", paramName === "date" ? value : "<%= selDate != null ? selDate : "" %>");
addField("base", paramName === "base" ? value : "<%= selBase != null ? selBase : "" %>");
addField("floor", paramName === "floor" ? value : "<%= selFloor != null ? selFloor : "" %>");
addField("area", paramName === "area" ? value : "<%= selArea != null ? selArea : "" %>");

// 拠点を変えたらフロア・エリアの選択はリセットする
if (paramName === "base") {
    addField("floor", "");
    addField("area", "");
}
if (paramName === "floor") {
    addField("area", "");
}

// 検索条件・選択中の対象者を維持したまま再読み込みする
addField("searchDeptIndividual", document.getElementById("searchDeptIndividual") ? document.getElementById("searchDeptIndividual").value : "");
addField("searchNameIndividual", document.getElementById("searchNameIndividual") ? document.getElementById("searchNameIndividual").value : "");
addField("searchIdIndividual", document.getElementById("searchIdIndividual") ? document.getElementById("searchIdIndividual").value : "");
addField("searchDeptMeeting", document.getElementById("searchDeptMeeting") ? document.getElementById("searchDeptMeeting").value : "");
addField("searchNameMeeting", document.getElementById("searchNameMeeting") ? document.getElementById("searchNameMeeting").value : "");
addField("searchIdMeeting", document.getElementById("searchIdMeeting") ? document.getElementById("searchIdMeeting").value : "");

if (selectedIndividual) addField("targetUserId", selectedIndividual.userId);
selectedMeeting.forEach(function(a) { addField("targetUserIds", a.userId); });

document.body.appendChild(form);
form.submit();
}
</script>
</body>
</html>

```

ここで新しく出てくるもの

- `var ACCOUNTS = [ ... ]; var SEATS = { ... };` … JSPのループでJavaScriptの配列・オブジェクトの**中身をその場書き出ししている**書き方。サーバー側が持っている「全社員一覧」「座席名一覧」を、画面が表示された時点でまろごとJavaScript側に渡してしまう。これにより、検索ボタンを押したときにサーバーへ問い合わせる必要がなく、ブラウザの中だけで `Array.filter()` を使った検索ができる（通信が発生しないので速い）。一方で、社員数が非常に多くなると最初の読み込みが重くなるという欠点もある。
- `boolean validIndividual = selTargetUserId != null && selTargetUserId.matches("\\d+");` … ■ セキュリティ上重要な書き方：**String.matches(正規表現) による値の検証**。 `selTargetUserId` はURLパラメータ経由でユーザーが書き換えられる値だが、これをそのまま `<script>` 内に埋め込むと、悪意のある文字列（例： `" ; alert(1); //` のようなJavaScriptのコード片）を埋め込まれてページが乗っ取られる危険がある（クロスサイトスクリプティング/XSS）。 `\\d+`（1文字以上の数字のみ）に一致するかを確認してから埋め込むことで、「数字以外の値は無視する」という安全策になっている。 `matches()` は文字列が正規表現のパターンに**完全一致**するかを `true` / `false` で返すメソッド。
- `document.createElement("select") / appendChild() / .onclick = function() {...}` … ■ JavaScriptの基本：DOM（Document Object Model）操作。HTMLの要素をJavaScriptのコードから動的に作成（`createElement`）し、画面に追加（`appendChild`）し、クリック時の動作（`onclick`）を設定する一連の流れ。 `searchAccounts()` や `renderSelected()` は、検索結果や選択済みの対象者一覧を、この方法で毎回作り直している。
- `ACCOUNTS.filter(function(a) { return ...; })` … JavaScriptの `Array.filter()`。Javaの `stream().filter()`（Day3で初出）とほぼ同じ考え方で、「条件に合う要素だけを残した新しい配列」を作る。 `.some()`（1つでも条件に合う要素があるか）、 `.map()`（各要素を変換した新しい配列を作る）、 `.forEach()`（各要素に対して処理を行う）もJavaScriptの配列メソッドで、それぞれJavaの `anyMatch()`・`map()`・`forEach()` に対応する考え方。
- `<input type="checkbox" ... style="display:none;" onchange="toggleSeat(this)"> + <label class="seat-btn available" for="seat...">` … ■ よく使われるUIパターン：**チェックボックスを見えなくして、ラベルをボタンのように見せる**。 `<input type="checkbox">` を `display:none` で隠し、 `<label for="...">` でそのチェックボックスと結びつけることで、ラベル部分をクリックするとチェックボックスがON/OFFされる。見た目は座席ボタンだが、実際には複数選択可能なチェックボックス群になっている。
- `function reloadWith(paramName, value) { var form = document.createElement("form"); ... form.submit(); }` … JavaScriptで**フォーム要素そのものを動的に作成して送信する**書き方。 `<select onchange="...">` で選択肢を変えるたびに、現在の選択状態（モード・検索条件・選択中の対象者など）をすべて `hidden` 項目として詰め込んだ `<form>` を組み立て、 `POST` で送信し直している。 `GET` ではなく `POST` を使っている

のは、「日本語の拠点名・フロア名・エリア名」をURLの一部（クエリ文字列）に含めると、サーバーの設定によって文字が壊れる場合があるため。`POST` のボディに含めることで、文字エンコーディングの問題を避けている。

- `sessionStorage.setItem(...)` / `sessionStorage.getItem(...)` / `window.addEventListener("beforeunload", ...)` / `window.addEventListener("load", ...)` … Day2で出てきた `localStorage` との対比。`sessionStorage` は**タブを閉じると消える**一時的な保存場所。`beforeunload`（ページを離れる直前）にスクロール位置を保存し、`load`（ページが読み込まれた直後）に復元することで、`<select>` を変更してページが再読み込みされても、見ていた場所付近に戻るようにしている。

読み方のポイント

- このファイルは「JSPでサーバーのデータをHTMLとJSへ出力する部分」と「ブラウザの中だけで動くJavaScriptのロジック」が混在しているため、読むときは**2つの言語が同じファイルに書かれている**ことを意識する。`<% ... %>` で囲まれている部分（サーバー側でJavaのコードが実行され、その結果の文字列がHTML/JSとして出力される）と、それ以外の部分（ブラウザでJavaScriptとして実行される）を分けて読むとよい。
- 全体の流れは、3-6-3で読んだ `AdminReserveLogic` の `showAdminReserve()` が用意した `request` 属性（`accountRows`、`bases` / `floors` / `areas`、`areaSeats`、`seatUsage` など）を、このJSPが「画面の初期状態」として描画し、そこから先の「対象者の検索・追加・削除」「座席の選択」はJavaScriptだけで完結する、という構成。`doAdminReserve` が呼ばれるのは、最終的に「この内容で予約する」ボタンを押したときの1回だけ。
- `selectedIndividual` / `selectedMeeting` の復元（`restoreSelected()`）は、3-6-3で見た `selTargetUserId` / `selTargetUserIds`（`AdminReserveLogic` が登録失敗時に再表示用としてセットする値）と対応している。「① モードを選ぶ → ② 対象者を選ぶ → ③ 日付・場所を選ぶ → ④ 座席を選ぶ」のうち、後の方のステップ（③④）で選び直しても、②で選んだ対象者が消えない仕組みが、この復元処理によって支えられている。

Day6のまとめ・①解説書との関連

- Day6では、管理者専用の3画面（`menu.jsp` を拡張した `adminMenu.jsp`、`seatStatus.jsp`、`adminReserve.jsp`）と、それを支える `ControlServlet` の `showMenu()` / `showSeatStatus()`、そして `AdminReserveLogic` を読んだ。①解説書の「管理者機能（座席利用状況の確認／代理予約／マスタ更新）」の章を読むときは、この3画面のうち「マスタ更新」を除いた2つがここで対応する（マスタ更新はDay7）。
 - `AdminReserveLogic` で初めて登場した `registerId`（登録した人）と `userId`（予約の本人）の区別は、①解説書で「誰が予約したか」を管理者が把握できる、という説明の裏付けになっているコード。一般ユーザーの予約（Day4の `ReserveLogic`）では `registerId == userId`（自分で自分の予約を登録する）だが、代理予約では `registerId`（管理者）と `userId`（対象者）が異なる、という違いを押さえておくと、Day7で予約データの一覧を見るときにも役立つ。
 - `seatStatus.jsp` の `LinkedHashMap` を使った日付プルダウン（2週間前～2週間後）は、Day4の `reserve.jsp` の日付プルダウン（今日～2週間後）と似ているが、`seatStatus.jsp` は**過去の日付も選べる**という違いがある。これは「過去の利用状況を振り返って確認したい」という管理者向けの用途を反映している。
 - `adminReserve.jsp` のJavaScript量の多さに圧倒された場合は、まず `AdminReserveLogic.java`（Javaのロジック）の方を先に理解し、「`request` にどんな名前のデータが、どんな型（`List` / `Map` / `String`）で入っているか」を一覧にしてから `adminReserve.jsp` を読むと、JavaScriptが「そのデータをどう画面に出しているか」という変換作業として読めるようになる。
-

Day7：マスタ更新・データベース定義

最終日は、座席マスタ（`seats` テーブル）をCRUD（追加・更新・削除）する「マスタ更新」画面と、これまで何度も登場した `seats` / `users` / `departments` / `reservations` の4テーブルの定義そのものを読む。Day1〜6で読んできた「Javaのコードが何をしているか」が、最終的に「データベースのどのテーブル・どの列に対応しているか」を確認する1日。

3-7-1. model/MasterUpdateLogic.java

```
package model;

import dao.MastersDAO;
import dao.SeatsDAO;
import entity.Seat;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.util.List;
import java.util.stream.Collectors;

public class MasterUpdateLogic implements Logic {

    @Override
    public String execute(HttpServletRequest req, HttpServletResponse res) throws Exception {
        String action = req.getParameter("action");
        MastersDAO dao = new MastersDAO();

        switch (action) {
            case "masterUpdate":
                // 一覧表示
                setListAttributes(req, dao);
                return "/WEB-INF/jsp/masterUpdate.jsp";

            case "masterEdit": {
                // 更新フォームを表示 (対象行を editSeat にセット)
                int id = Integer.parseInt(req.getParameter("seatId"));
                Seat editSeat = dao.findById(id);
                req.setAttribute("editSeat", editSeat);
                setListAttributes(req, dao);
                return "/WEB-INF/jsp/masterUpdate.jsp";
            }

            case "doMasterAdd": {
                // 新規追加
                Seat s = buildSeat(req, 0);
                if (dao.exists(s.getBaseName(), s.getFName(), s.getAreaName(), s.getSeatName(), -1)) {
                    req.setAttribute("errorMsg",
                        "同じ座席 (" + s.getBaseName() + " " + s.getFName() + " " + s.getAreaName() + " " + s.getSeatName() +
                        ") はすでに存在します。");
                    setListAttributes(req, dao);
                    return "/WEB-INF/jsp/masterUpdate.jsp";
                }
                dao.add(s);
                req.setAttribute("message", "座席「" + s.getSeatName() + "」を追加しました。");
                setListAttributes(req, dao);
                return "/WEB-INF/jsp/masterUpdate.jsp";
            }

            case "doMasterUpdate": {
                // 更新
                int id = Integer.parseInt(req.getParameter("seatId"));
                Seat s = buildSeat(req, id);
                if (dao.exists(s.getBaseName(), s.getFName(), s.getAreaName(), s.getSeatName(), id)) {
                    req.setAttribute("errorMsg",
                        "同じ座席 (" + s.getBaseName() + " " + s.getFName() + " " + s.getAreaName() + " " + s.getSeatName() +
                        ") はすでに存在します。");
                    req.setAttribute("editSeat", dao.findById(id));
                    setListAttributes(req, dao);
                    return "/WEB-INF/jsp/masterUpdate.jsp";
                }
                dao.update(s);
                req.setAttribute("message", "座席ID " + id + " を更新しました。");
                setListAttributes(req, dao);
                return "/WEB-INF/jsp/masterUpdate.jsp";
            }

            case "doMasterDelete": {
                // 削除
                int id = Integer.parseInt(req.getParameter("seatId"));
            }
        }
    }
}
```

```

        dao.delete(id);
        req.setAttribute("message", "座席ID " + id + " を削除しました。");
        setListAttributes(req, dao);
        return "/WEB-INF/jsp/masterUpdate.jsp";
    }

    default:
        setListAttributes(req, dao);
        return "/WEB-INF/jsp/masterUpdate.jsp";
    }
}

private Seat buildSeat(HttpServletRequest req, int seatId) {
    Seat s = new Seat();
    s.setSeatId(seatId);

    // 「既存オフィスから選択」または「新しいオフィス名を入力」のどちらかが baseName に入る。
    // 新規オフィス入力欄 (newBaseName) に値があれば、そちらを優先する。
    // ※オフィスという名前のマスタ行は無く、座席を1件追加した時点でそのオフィスが新しく登場する仕組み。
    String newBase = req.getParameter("newBaseName");
    String baseName = (newBase != null && !newBase.trim().isEmpty())
        ? newBase.trim() : req.getParameter("baseName");

    s.setBaseName(baseName);
    s.setFName(req.getParameter("fName"));
    s.setAreaName(req.getParameter("areaName"));
    s.setSeatName(req.getParameter("seatName"));
    return s;
}

/**
 * 一覧表示・絞り込み・フロアマップ画像表示に必要な属性をまとめてセットする。
 * 「㉑ オフィスを選んで見る」「㉒ フロアを選んで見る」ための
 * bases / selectedBase / floors / selectedFloor / officeImage と、
 * 選んだオフィス・フロアだけに絞り込んだ座席一覧 seats をリクエストに積む。
 *
 * フロアまで選ぶと、そのフロア専用のマップ（「拠点名_フロア名」）があれば優先表示される
 * （無ければ拠点単位の画像にフォールバック）→ 新しいオフィスを追加したときも、
 * フロアを選んだ時点でそのフロアのマップ画像に切り替わる。
 */
private void setListAttributes(HttpServletRequest req, MastersDAO dao) {
    SeatsDAO seatsDAO = new SeatsDAO();
    String selectedBase = req.getParameter("filterBase");
    String selectedFloor = req.getParameter("filterFloor");
    boolean hasBase = selectedBase != null && !selectedBase.isEmpty();
    boolean hasFloor = selectedFloor != null && !selectedFloor.isEmpty();

    List<Seat> all = dao.getAll();
    List<Seat> seats = all;
    if (hasBase) seats = seats.stream().filter(s -> selectedBase.equals(s.getBaseName())).collect(Collectors.toList());
    if (hasFloor) seats = seats.stream().filter(s -> selectedFloor.equals(s.getFName())).collect(Collectors.toList());

    req.setAttribute("seats", seats);
    req.setAttribute("bases", seatsDAO.getBases());
    req.setAttribute("selectedBase", selectedBase);
    req.setAttribute("selectedFloor", selectedFloor);
    req.setAttribute("floors", hasBase ? seatsDAO.getFloors(selectedBase) : null);
    req.setAttribute("officeImage", hasBase ? seatsDAO.getExpectedFloorImage(selectedBase, selectedFloor) : null);
}
}

```

ここで新しく出てくるもの

- `case "masterEdit": { ... }` のように、複数の `case` が中括弧 `{ }` で囲まれている … `switch` 文の中で、各 `case` ごとにローカル変数（`id`、`editSeat` など）を使いたい場合、`{ }` で囲んで**そのcaseだけのスコープ（変数の有効範囲）**を作る書き方。`{ }` が無いと、別の `case` で同じ名前の変数を定義したときに「変数が重複している」というコンパイルエラーになる。
- `dao.exists(s.getBaseName(), s.getFName(), s.getAreaName(), s.getSeatName(), -1)` … `doMasterAdd`（新規追加）では、まだ `seatId` が存在しないため、除外IDとして `-1`（存在しない値）を渡している。`doMasterUpdate`（更新）では、自分自身の行は重複チェックの対象から除外する必要があるため、更新対象の `id` を渡している。「この処理だけの特別な意味を持つ値」を関数の引数として渡す例。

- `buildSeat()` の中の「`newBaseName` に値があればそちらを優先する」処理は、Day7-3 (`masterUpdate.jsp`) で見る「+ 新しいオフィスを追加する」という選択肢と対応している。

読み方のポイント

- `MasterUpdateLogic` は、Day3の `LocationRegistLogic` や、Day6の `AdminReserveLogic` と同じ「1つのLogicクラスが `action` の値によって複数の処理（一覧表示・編集フォーム表示・追加・更新・削除）を切り分ける」という構成。`switch` 文の各 `case` が「画面のどの操作に対応しているか」を意識しながら読むと、複雑に見えても実際は「表示系（`masterUpdate` / `masterEdit`）」と「更新系（`doMaster*`）」の2グループに分かれていることが分かる。
 - 更新系の3つの `case`（`doMasterAdd` / `doMasterUpdate` / `doMasterDelete`）は、いずれも処理の最後で `setListAttributes(req, dao)` を呼んで一覧を再表示している。これは、Day3の `LocationRegistLogic` で見た「処理が終わったら一覧画面に戻す」パターンと同じ。
-

```

package dao;

import entity.Seat;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

/**
 * 座席マスタCRUD用DAO（JDBC版）
 * SeatsDAO と同じ seats テーブルを操作する。
 */
public class MastersDAO {

    private Seat map(ResultSet rs) throws SQLException {
        return new Seat(
            rs.getInt("seat_id"),
            rs.getString("base_name"),
            rs.getString("f_name"),
            rs.getString("area_name"),
            rs.getString("seat_name")
        );
    }

    public List<Seat> getAll() {
        String sql = "SELECT seat_id, base_name, area_name, f_name, seat_name FROM seats "
            + "ORDER BY base_name, f_name, area_name, seat_name";
        List<Seat> list = new ArrayList<>();
        try (Connection conn = DBUtil.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {
            while (rs.next()) list.add(map(rs));
        } catch (SQLException e) {
            throw new RuntimeException("座席マスタ一覧の取得に失敗しました", e);
        }
        return list;
    }

    /** 指定の拠点・フロア・エリア・席名と同じ座席が既に存在するか（excludeSeatIdの席は除く） */
    public boolean exists(String baseName, String fName, String areaName, String seatName, int excludeSeatId) {
        String sql = "SELECT 1 FROM seats WHERE base_name=? AND f_name=? AND area_name=? AND seat_name=? AND seat_id<>?";
        try (Connection conn = DBUtil.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, baseName);
            ps.setString(2, fName);
            ps.setString(3, areaName);
            ps.setString(4, seatName);
            ps.setInt(5, excludeSeatId);
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next();
            }
        } catch (SQLException e) {
            throw new RuntimeException("座席の重複チェックに失敗しました", e);
        }
    }

    public Seat findById(int seatId) {
        String sql = "SELECT seat_id, base_name, area_name, f_name, seat_name FROM seats WHERE seat_id = ?";
        try (Connection conn = DBUtil.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, seatId);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) return map(rs);
            }
        } catch (SQLException e) {
            throw new RuntimeException("座席の取得に失敗しました", e);
        }
        return null;
    }
}

```

```

public void add(Seat seat) {
    // seat_id は AUTO_INCREMENT ではないため、現在の最大値+1を採番する
    String selectMaxSql = "SELECT IFNULL(MAX(seat_id), 0) + 1 AS next_id FROM seats";
    String insertSql = "INSERT INTO seats (seat_id, base_name, area_name, f_name, seat_name) VALUES (?, ?, ?, ?, ?)";
    try (Connection conn = DBUtil.getConnection()) {
        int nextId;
        try (PreparedStatement ps = conn.prepareStatement(selectMaxSql);
            ResultSet rs = ps.executeQuery()) {
            rs.next();
            nextId = rs.getInt("next_id");
        }
        try (PreparedStatement ps = conn.prepareStatement(insertSql)) {
            ps.setInt(1, nextId);
            ps.setString(2, seat.getBaseName());
            ps.setString(3, seat.getAreaName());
            ps.setString(4, seat.getFName());
            ps.setString(5, seat.getSeatName());
            ps.executeUpdate();
        }
        seat.setSeatId(nextId);
    } catch (SQLException e) {
        throw new RuntimeException("座席の追加に失敗しました", e);
    }
}

public boolean update(Seat updated) {
    String sql = "UPDATE seats SET base_name=?, area_name=?, f_name=?, seat_name=? WHERE seat_id=?";
    try (Connection conn = DBUtil.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, updated.getBaseName());
        ps.setString(2, updated.getAreaName());
        ps.setString(3, updated.getFName());
        ps.setString(4, updated.getSeatName());
        ps.setInt(5, updated.getSeatId());
        return ps.executeUpdate() > 0;
    } catch (SQLException e) {
        throw new RuntimeException("座席の更新に失敗しました", e);
    }
}

public boolean delete(int seatId) {
    String sql = "DELETE FROM seats WHERE seat_id = ?";
    try (Connection conn = DBUtil.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, seatId);
        return ps.executeUpdate() > 0;
    } catch (SQLException e) {
        throw new RuntimeException("座席の削除に失敗しました", e);
    }
}
}

```

ここで新しく出てくるもの

- try (Connection conn = DBUtil.getConnection()) { ... } の中で、さらに try (PreparedStatement ps = ...) { ... } を2回続けて書いている (add() メソッド) … **** try のネスト (入れ子)**。外側の try は「DB接続全体」を、内側の各 try は「1回ごとのSQL実行」を表す。1つの接続の中で「①現在の最大 seat_id を調べる」→「②その値+1で INSERT する」という2回のSQL**を順番に実行している。
PreparedStatement は1つのSQLにつき1つ必要なので、SQLが2つあればtryも2つ必要になる。
- String selectMaxSql = "SELECT IFNULL(MAX(seat_id), 0) + 1 AS next_id FROM seats"; … SQLの MAX() (最大値) と IFNULL(値, 代替値) (値がNULLなら代替値を使う) の組み合わせ。座席が1件も無い状態 (MAX(seat_id) が NULL になる) でも 0 + 1 = 1 から始められるようにしている。AUTO_INCREMENT を使わず、手動で「現在の最大値+1」を採番している点に注意 (複数人が同時に追加すると番号が重複する可能性があるが、この座席数・利用人数の規模では問題にならないと判断されている)。
- "SELECT 1 FROM seats WHERE ... " の exists() メソッド … 実際のデータの値は使わず「該当する行が存在するかどうか」だけを知りたいときによく使われる書き方。SELECT * のように全列を取得する必要がないので、SELECT 1 (固定の値 1 だけを返す) にすることで、データベースの負荷を少し減らせる。rs.next() が true を返せば「1行以上ヒットした=存在する」、false なら「存在しない」。

読み方のポイント

- MastersDAO は、SeatsDAO (Day4) と同じ seats テーブルを操作している。SeatsDAO は「予約のために座席を検索・絞り込みする(読み取り専用)」役割、MastersDAO は「座席そのものを追加・更新・削除する」役割、と役割分担されている。同じテーブルに対して目的の異なる2

つのDAOが存在する、という構成は、チームで設計を議論するときの良い例になる。

- `add()` の「最大値+1で採番する」処理は、Day1の `Reservation` エンティティで見た `reserve_id`（こちらは `AUTO_INCREMENT` で自動採番）との違いを意識すると理解しやすい。`reservations` テーブルは行が増え続けるため `AUTO_INCREMENT` が向いているが、`seats` テーブルは「削除されることもある／物理的な座席数に対応する番号」のため、手動採番になっている。

3-7-3. `webapp/WEB-INF/jsp/masterUpdate.jsp`

座席マスタの一覧表示・絞り込み・新規追加・更新・削除をすべて1ページで行う管理画面。

```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ page import="java.util.List, entity.Seat" %>
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>マスタ更新 - 座・Navi</title>
<link rel="stylesheet" href="<%= request.getContextPath() %>/css/style.css">
</head>
<body>
<%@ include file="common/headerAdmin.jspf" %>

<div class="container">
  <div class="card">
    <h2>座席マスタ更新</h2>

    <!-- 完了メッセージ -->
    <% if (request.getAttribute("message") != null) { %>
    <div class="alert alert-success">&#10003; <%= request.getAttribute("message") %></div>
    <% } %>

    <!-- エラーメッセージ（重複座席など） -->
    <% if (request.getAttribute("errorMsg") != null) { %>
    <div class="alert alert-danger"><%= request.getAttribute("errorMsg") %></div>
    <% } %>

    <%
    List<Seat> seats    = (List<Seat>) request.getAttribute("seats");
    List<String> bases  = (List<String>) request.getAttribute("bases");
    Seat editSeat      = (Seat) request.getAttribute("editSeat");
    String selectedBase = (String) request.getAttribute("selectedBase");
    String selectedFloor = (String) request.getAttribute("selectedFloor");
    List<String> floors  = (List<String>) request.getAttribute("floors");
    String officeImage  = (String) request.getAttribute("officeImage");
    String ctx = request.getContextPath();
    %>

    <p style="font-size:12px; color:#757575; margin-bottom:10px;">
      ① オフィス、② フロアを選ぶと、そのフロアのマップ画像とその条件に絞り込んだ座席だけを表示できます（見やすさ・確認用の絞り込みです）。
    </p>

    <!-- ① オフィス・② フロアを選んで表示・絞り込み -->
    <form action="<%= ctx %>/ControlServlet" method="get" style="margin-bottom:14px;">
      <input type="hidden" name="action" value="masterUpdate">
      <div class="filter-row">
        <div class="form-group">
          <label>① オフィスを選択</label>
          <select name="filterBase" onchange="submitMasterFilterForm(this)">
            <option value="">-- すべてのオフィスを表示 --</option>
            <% if (bases != null) for (String b : bases) { %>
            <option value="<%= b %>" <%= b.equals(selectedBase) ? "selected" : "" %>><%= b %></option>
            <% } %>
          </select>
        </div>
        <div class="form-group">
          <label>② フロアを選択</label>
          <select name="filterFloor" onchange="submitMasterFilterForm(this)" <%= (floors == null) ? "disabled" : "" %>>
            <option value="">-- すべてのフロアを表示 --</option>
            <% if (floors != null) for (String f : floors) { %>
            <option value="<%= f %>" <%= f.equals(selectedFloor) ? "selected" : "" %>><%= f %></option>
            <% } %>
          </select>
        </div>
      </div>
    </form>

    <!-- 選んだオフィス・フロアのマップ画像（画像を見ながら座席を確認・追加できるように。
      フロアまで選ぶとそのフロア専用の画像に切り替わる＝新しいオフィスを追加したときも
      フロアを選んだ時点でそのフロアのマップに切り替わる） -->
    <% if (officeImage != null) { %>
    <div style="margin-bottom:16px;">

```

```

<%= selectedFloor != null ? selectedFloor : "" %>のフロアマップ"
    style="max-width:100%; border:1px solid #ECEFF1; border-radius:8px; display:block;"
    onerror="this.style.display='none'; this.nextElementSibling.style.display='flex';">
<div class="map-placeholder" style="display:none;">
    <p class="ph-title">フロアマップ画像が未設定です</p>
    <code><%= officeImage %></code>
    <p class="ph-hint">このファイルを images/ フォルダに配置すると自動で表示されます</p>
</div>
</div>
<% } %>

<!-- 座席一覧（①で選んだオフィスのみ表示。未選択ならすべて表示） -->
<h3 style="margin-bottom:8px; color:var(--teal-dark);">
    <%
        String listTitle;
        if (selectedBase != null && !selectedBase.isEmpty()) {
            listTitle = selectedBase + ((selectedFloor != null && !selectedFloor.isEmpty()) ? selectedFloor : "") + "の座席一覧";
        } else {
            listTitle = "座席一覧（全オフィス）";
        }
    %>
    <%= listTitle %>
    <span style="font-size:12px; color:#9E9E9E; font-weight:400;">（<%= seats != null ? seats.size() : 0 %> 席）</span>
</h3>
<!--
    件数が増えてもページ全体が縦に伸び続けないよう、一覧をカード内の
    スクロール領域（最大高さ固定 + ヘッダー追従）に収める。
    「①オフィスを選択」で絞り込めば、その拠点だけの短い一覧として確認できる。
-->
<div style="max-height:480px; overflow-y:auto; border:1px solid #ECEFF1; border-radius:8px; margin-bottom:24px;">
<table class="table" style="margin-bottom:0;">
    <thead>
        <tr>
            <th style="position:sticky; top:0; background:#fff; z-index:1;">ID</th>
            <th style="position:sticky; top:0; background:#fff; z-index:1;">拠点</th>
            <th style="position:sticky; top:0; background:#fff; z-index:1;">フロア</th>
            <th style="position:sticky; top:0; background:#fff; z-index:1;">エリア</th>
            <th style="position:sticky; top:0; background:#fff; z-index:1;">席名</th>
            <th style="position:sticky; top:0; background:#fff; z-index:1; width:140px;">操作</th>
        </tr>
    </thead>
    <tbody>
        <%
            String rowBase = null;
            if (seats != null) for (Seat s : seats) {
                boolean newBase = !s.getBaseName().equals(rowBase);
                rowBase = s.getBaseName();
                if (newBase && (selectedBase == null || selectedBase.isEmpty())) {
                    %>
                    <!-- 全オフィス表示時は拠点の切れ目が見出し行を挟む -->
                    <tr style="background:#F5F5F5;">
                        <td colspan="6" style="font-weight:700; color:var(--teal-dark);"><%= rowBase %></td>
                    </tr>
                    <% } %>
                    <tr>
                        <td><%= s.getSeatId() %></td>
                        <td><%= s.getBaseName() %></td>
                        <td><%= s.getFName() %></td>
                        <td><%= s.getAreaName() %></td>
                        <td><%= s.getSeatName() %></td>
                        <td>
                            <!-- 更新リンク → 編集フォームにIDを渡す -->
                            <a href="<%= ctx %>/ControlServlet?action=masterEdit&seatId=<%= s.getSeatId() %>&filterBase=<%= selectedBase != null ? selectedBase : "" %>&filterFloor=<%= selectedFloor != null ? selectedFloor : "" %>"
                                class="btn btn-primary btn-sm">更新</a>
                            <!-- 削除ボタン（POSTフォームを外部に配置し form= で紐づけ） -->
                            <button type="submit" form="delForm<%= s.getSeatId() %>" class="btn btn-danger btn-sm"
                                onclick="return confirm('座席ID <%= s.getSeatId() %> を削除しますか?')">削除</button>
                        </td>
                    </tr>
                </td>
            }
        </tbody>
    <!-- 削除用フォーム（テーブル外） -->

```

```

        <% } %>
    </tbody>
</table>
</div>

<!-- 削除フォーム（テーブル外に配置） -->
<% if (seats != null) for (Seat s : seats) { %>
<form id="delForm"<%= s.getSeatId() %>" action="<%= ctx %>/ControlServlet" method="post" style="display:none;">
    <input type="hidden" name="action" value="doMasterDelete">
    <input type="hidden" name="seatId" value="<%= s.getSeatId() %>">
</form>
<% } %>

<!-- 更新フォーム（masterEditで対象座席が設定された場合に表示） -->
<% if (editSeat != null) { %>
<div class="card" style="background-color:#E3F2FD; border:2px solid #42A5F5; margin-bottom:20px;">
    <h2 style="color:#1565C0;">座席 ID <%= editSeat.getSeatId() %> を更新</h2>
    <form action="<%= ctx %>/ControlServlet" method="post">
        <input type="hidden" name="action" value="doMasterUpdate">
        <input type="hidden" name="seatId" value="<%= editSeat.getSeatId() %>">
        <div class="filter-row">
            <div class="form-group">
                <label>拠点</label>
                <input type="text" name="baseName" value="<%= editSeat.getBaseName() %>" required>
            </div>
            <div class="form-group">
                <label>フロア</label>
                <input type="text" name="fName" value="<%= editSeat.getFName() %>" required>
            </div>
            <div class="form-group">
                <label>エリア</label>
                <input type="text" name="areaName" value="<%= editSeat.getAreaName() %>" required>
            </div>
            <div class="form-group">
                <label>席名</label>
                <input type="text" name="seatName" value="<%= editSeat.getSeatName() %>" required>
            </div>
            <div>
                <button type="submit" class="btn btn-primary">更新する</button>
                <a href="<%= ctx %>/ControlServlet?action=masterUpdate&filterBase=<%= selectedBase != null ? selectedBase : "" %>&filterFloor=<%= selectedFloor != null ? selectedFloor : "" %>" class="btn btn-secondary">キャンセル</a>
            </div>
        </div>
    </form>
</div>
<% } %>

<!-- 新規追加フォーム -->
<div class="card" style="background-color:#F9FBE7; border:2px solid #AFB42B;">
    <h2 style="color:#558B2F;">② 座席を追加する</h2>
    <p style="font-size:12px; color:#757575; margin-bottom:10px;">
        既存のオフィスに座席を増やすときは「拠点」で一覧から選び、まだ無いオフィスを増やしたいときは
        「+ 新しいオフィスを追加する」を選んでオフィス名を入力してください。
        ※オフィスという独立したマスタ行はなく、座席を1件追加した時点でそのオフィス名が新しく登場する仕組みです（③参照）。
    </p>
    <form action="<%= ctx %>/ControlServlet" method="post" id="addSeatForm">
        <input type="hidden" name="action" value="doMasterAdd">
        <div class="filter-row">
            <div class="form-group">
                <label>拠点</label>
                <select name="baseName" id="baseSelect" onchange="onBaseSelectChange()">
                    <% if (bases != null) for (String b : bases) { %>
                        <option value="<%= b %>" <%= b.equals(selectedBase) ? "selected" : "" %>><%= b %></option>
                    <% } %>
                    <option value="__NEW__">+ 新しいオフィスを追加する</option>
                </select>
            </div>
            <div class="form-group" id="newBaseGroup" style="display:none;">
                <label>新しいオフィス名</label>
                <input type="text" name="newBaseName" id="newBaseName" placeholder="例: 渋谷オフィス">
            </div>
            <div class="form-group">

```

```

        <label>フロア</label>
        <input type="text" name="fName" placeholder="例: 3F" required>
    </div>
    <div class="form-group">
        <label>エリア</label>
        <input type="text" name="areaName" placeholder="例: Cエリア" required>
    </div>
    <div class="form-group">
        <label>席名</label>
        <input type="text" name="seatName" placeholder="例: C-01" required>
    </div>
    <div>
        <button type="submit" class="btn btn-green">追加する</button>
    </div>
</div>
</div>
</form>
<p style="font-size:11px; color:#9E9E9E; margin-top:10px;">
    ③ 新しいオフィス（オフィスレイアウト画像）を増やすときは、
    <code>images</code> フォルダに画像ファイルを追加して、
    <code>SeatsDAO</code> の <code>OFFICE_IMAGES</code> に「拠点名」（またはフロアごとに画像が違う場合は
    「拠点名_フロア名」、例: 「芝浦_1F」）のキーで登録してください。この画面・座席利用状況・代理予約・座席予約の
    各画面に画像が表示されるようになります（手順は <code>SeatsDAO.java</code> 冒頭のコメントを参照）。
</p>
</div>

<script>
    // 「拠点」で「+ 新しいオフィスを追加する」を選んだときだけ、
    // 新しいオフィス名の入力欄を表示／必須にする（vanilla JS、フレームワーク不使用）
    function onBaseSelectChange() {
        var sel = document.getElementById('baseSelect');
        var grp = document.getElementById('newBaseGroup');
        var isNew = (sel.value === '__NEW__');
        grp.style.display = isNew ? '' : 'none';
        document.getElementById('newBaseName').required = isNew;
    }
    onBaseSelectChange();
</script>

<div style="margin-top:10px;">
    <a href="%< ctx %>/ControlServlet?action=adminMenu" class="btn btn-secondary">&larr; 管理メニューへ戻る</a>
</div>
</div>
</div>

<script>
    // 更新・追加・削除・絞り込みのたびにページ上部へ戻されてしまうのを防ぐため、
    // 送信前のスクロール位置を覚えておき、再読み込み後に同じ位置へ戻す。
    //
    // つまづきポイント1: <select onchange="this.form.submit()"> のように
    // フォームの submit() メソッドを直接呼び出すと "submit" イベントは発火しない
    // （addEventListener('submit', ...) では拾えない）。そのため絞り込み用セレクトは
    // 保存してから送信する関数を直接 onchange から呼び形にしている。
    // つまづきポイント2: ページ読み込み直後（スクリプト実行時点）に scrollToすると、
    // 画像（フロアマップ）の読み込みで後からレイアウトが変わり、
    // 結局トップへ戻ってしまう。window の load イベント（画像読み込み完了後）まで待つ。
    var MASTER_SCROLL_KEY = 'masterUpdateScrollY';

    function submitMasterFilterForm(sel) {
        sessionStorage.setItem(MASTER_SCROLL_KEY, String(window.scrollY));
        sel.form.submit();
    }

    document.querySelectorAll('form').forEach(function(f) {
        f.addEventListener('submit', function() {
            sessionStorage.setItem(MASTER_SCROLL_KEY, String(window.scrollY));
        });
    });

    (function() {
        var saved = sessionStorage.getItem(MASTER_SCROLL_KEY);
        if (saved === null) return;
        sessionStorage.removeItem(MASTER_SCROLL_KEY);
    })();

```

```

window.addEventListener('load', function() {
    window.scrollTo(0, parseInt(saved, 10));
});
})();
</script>
</body>
</html>

```

ここで新しく出てくるもの

- `String rowBase = null; if (seats != null) for (Seat s : seats) { boolean newBase = !s.getBaseName().equals(rowBase); rowBase = s.getBaseName(); if (newBase && ...) { ... } }` … `**「直前の行と値が変わったかどうか」を覚えておくループ**`。 `seats` 一覧は `base_name`, `f_name`, `area_name`, `seat_name` の順に並んでいるため、ループの中で「1つ前の行の `baseName`」を `rowBase` という変数に保存しておき、今の行の `baseName` と比べることで「拠点が切り替わった瞬間」だけ見出し行を挟むことができる。最初のループでは `rowBase` が `null` なので、必ず `newBase` が `true` になり、最初の拠点の見出しも表示される。一覧表示で「グループの境目」を表現する定番の書き方。
- `<button type="submit" form="delForm<%= s.getSeatId() %>" ...>削除</button>` … `HTML5` の `form` 属性。通常、ボタンは自分が入っている `<form>` タグに対して送信されるが、`form="delForm1"` のように他の場所にある `<form>` の `id` を指定することで、「テーブルの行の中にあるボタンを押すと、テーブルの外にある別のフォームが送信される」という構成にできる。`<table>` の中に直接 `<form>` を書くと `HTML` として正しく動かない場合があるため、削除用の小さな `<form>` をテーブルの外（149～154行目）にまとめて配置し、`form` 属性で結びつけている。
- `document.querySelectorAll('form').forEach(function(f) { f.addEventListener('submit', ...) })` … `querySelectorAll()` は条件に合う要素をすべて取得するメソッド（Day3の `querySelector` は最初の1つだけ取得）。ページ内のすべての `<form>` に対して同じイベントリスナーを一括で設定している。

読み方のポイント

- このJSPは「①②絞り込み」「一覧表示（更新・削除ボタン付き）」「更新フォーム（条件付き表示）」「新規追加フォーム」の4つのブロックが縦に並んでいる構成。3-7-1の `MasterUpdateLogic` の各 `case` が、それぞれのブロックに対応するデータをセットしているかを見比べると、「Javaが用意したデータ」と「JSPがそれをどう表示するか」の対応がとりやすい。
 - `masterUpdate / masterEdit` → ①②絞り込み・一覧・（`masterEdit` のときは）更新フォーム
 - `doMasterAdd` → 新規追加フォームの送信先
 - `doMasterUpdate` → 更新フォームの送信先
 - `doMasterDelete` → 削除フォーム（テーブル外）の送信先
- 「拠点を選んで一覧の見出しを切り替える」「フロアマップ画像を表示する」という処理は、Day1で見た `MasterUpdateLogic.setListAttributes()` のコメントに書かれている設計意図（フロア専用画像があれば優先、無ければ拠点単位にフォールバック）が、画像表示部分（70～82行目）にそのまま反映されている。コードを読むときに「コメントに書かれている意図」と「実際の表示処理」を両方確認すると理解が深まる。

3-7-4. データベース定義（`docs/db/za_navi_schema.sql`）

最後に、これまで何度も登場した4つのテーブルの定義を確認する。これはJavaのファイルではないが、`entity` パッケージの各クラス（Day1）や、各DAOのSQL文（Day2～7）が「結局どんな表に対して読み書きしているか」を理解するための土台になる。

```

-- ----- 部署マスタ -----
CREATE TABLE departments (
    b_id VARCHAR(10) PRIMARY KEY,
    b_name VARCHAR(100) NOT NULL
);

-- ----- アカウント -----
CREATE TABLE users (
    user_id INT(10) PRIMARY KEY,
    password VARCHAR(20) NOT NULL,
    b_id VARCHAR(10) NOT NULL,
    name VARCHAR(30) NOT NULL,
    admin TINYINT(1) NOT NULL DEFAULT 0,
    CONSTRAINT fk_users_departments
        FOREIGN KEY (b_id) REFERENCES departments (b_id)
);

-- ----- 座席マスタ -----
CREATE TABLE seats (
    seat_id INT(3) PRIMARY KEY,
    base_name VARCHAR(50) NOT NULL,
    area_name VARCHAR(10) NOT NULL,
    f_name VARCHAR(10) NOT NULL,
    seat_name VARCHAR(20) NOT NULL
);

-- ----- 予約・行先登録 -----
CREATE TABLE reservations (
    reserve_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT(10) NOT NULL,
    register_id INT(10) NOT NULL,
    reserve_date DATE NOT NULL,
    place VARCHAR(30) NOT NULL,
    seat_id INT(3) NOT NULL,
    memo VARCHAR(200) DEFAULT NULL,
    cancel_flag TINYINT(1) NOT NULL DEFAULT 0,
    -- 「有効な (cancel_flag=0) の予約だけを対象に重複チェックしたい」を表現するための生成列。
    -- 対象外のケースではNULLにし、MySQLのUNIQUEインデックスはNULL同士を重複と見なさない性質を利用する。
    -- seat_active_key: 席を伴わない予約 (在宅・出張=seat_id=0) やキャンセル済みは対象外 → 同日同席の二重予約だけを防ぐ
    -- user_active_key: キャンセル済みは対象外 → 1人が同じ日に有効な予約を複数持てないようにする
    seat_active_key INT(3) GENERATED ALWAYS AS
        (CASE WHEN cancel_flag = 0 AND seat_id > 0 THEN seat_id END) STORED,
    user_active_key INT(10) GENERATED ALWAYS AS
        (CASE WHEN cancel_flag = 0 THEN user_id END) STORED,
    CONSTRAINT fk_reservations_users
        FOREIGN KEY (user_id) REFERENCES users (user_id),
    CONSTRAINT fk_reservations_register_users
        FOREIGN KEY (register_id) REFERENCES users (user_id),
    CONSTRAINT uq_reservations_seat_active
        UNIQUE (reserve_date, seat_active_key),
    CONSTRAINT uq_reservations_user_active
        UNIQUE (reserve_date, user_active_key)
);

```

ここで新しく出てくるもの

- PRIMARY KEY** (主キー) … その表の中で**1行を一意に識別する列**。 `departments.b_id`、 `users.user_id`、 `seats.seat_id`、 `reservations.reserve_id` がそれぞれの表の主キー。Day1の各エンティティクラス (`Location` / `Account` / `Seat` / `Reservation`) の「IDにあたるフィールド」と1対1で対応している。
- FOREIGN KEY (b_id) REFERENCES departments (b_id)** (外部キー) … ある表の列の値が、**別の表の主キーに存在する値でなければならない**という制約。 `users.b_id` は必ず `departments.b_id` に存在する値でなければならない、という関係を表す。 `AccountsDAO` や `LocationsDAO` で「 `b_id` から `b_name` を引く」処理 (Day2・Day5で出てきた `LocationsDAO.getBNameById()` など) が成立するのは、この外部キー制約によって「存在しない部署IDを持つユーザー」が作られないことが保証されているため。
- AUTO_INCREMENT** (自動連番) … `reservations.reserve_id` に付いている。新しい行を `INSERT` するたびに、データベースが自動的に「今までの最大値+1」を割り振る。3-7-2で見た `MastersDAO.add()` の「手動で最大値+1を計算する」処理と対比すると、 `AUTO_INCREMENT` が「データベースに任せる」方式、 `MastersDAO` 方式が「アプリ側で計算する」方式という違いがよく分かる。
- GENERATED ALWAYS AS (...) STORED** (生成列) … 他の列の値から**自動的に計算されて保存される列**。 `seat_active_key` は「キャンセルされておらず、かつ座席を使う予約 (`seat_id > 0`) のときだけ `seat_id` の値、それ以外は `NULL` 」という列。MySQLの `UNIQUE` 制約は「 `NULL` 同士

は重複とみなさない」という性質があるため、この生成列に対して `UNIQUE` 制約をかけることで「キャンセル済みや在宅・出張の予約はチェック対象外、有効な出社予約だけが『同じ日・同じ席は1件まで』という制約の対象になる」という、複雑な業務ルールをデータベース側で保証している。Day3の `ReserveLogic.doReserve()` で見た「先にチェックしてからINSERTする」処理（コメントで `uq_reservations_user_active` に言及）は、この制約と対応関係にある。

読み方のポイント

- この4テーブルの関係を図にすると、`departments` ← `users` ← `reservations` → `seats` という形になる（`reservations` が `users` と `seats` の両方を参照する、いわゆる「中間テーブル」のような位置づけ）。Day1〜7で読んできた各DAOのSQLは、すべてこの4テーブルのいずれか（または複数の組み合わせ）に対する `SELECT` / `INSERT` / `UPDATE` / `DELETE` である。
- `seats` テーブルには `reservations` への外部キーが**無い**（コメントにも明記されている）。これは、`reservations.seat_id` が「席を使わない予約（在宅・出張）」を表す `0` という値を許容する設計のため。「外部キー制約が無い列がある」のは設計ミスではなく、意図的な選択であることがコメントから読み取れる。こうした「コメントに書かれている設計判断の理由」は、コード自体からは読み取れない情報なので、読み飛ばさずに確認する価値がある。

Day7のまとめ・①解説書との関連

- Day7では、座席マスタを編集する `masterUpdate.jsp` / `MasterUpdateLogic` / `MastersDAO` と、これまでのDAOがすべて操作してきた4つのテーブルの定義を確認した。①解説書の「管理者機能」の章にある「マスタ更新」の説明は、ここで読んだ `MasterUpdateLogic` の5つの `case` （一覧・編集表示・追加・更新・削除）に対応する。
 - `MastersDAO.add()` の「最大値+1で採番する」処理と、`reservations.reserve_id` の `AUTO_INCREMENT` は、同じ「新しい行にIDを割り振る」という目的に対する**2つの異なる方法**。どちらが優れているというものではなく、「テーブルの性質（行が増減するか、増え続けるだけか）によって適切な方法を選ぶ」という考え方の例として、チームでの議論のきっかけにするとよい。
 - `reservations` テーブルの生成列・`UNIQUE` 制約は、Day3の `ReserveLogic` ・ `AdminReserveLogic` で書かれていた「事前にチェックしてからINSERTする」コードの**裏付け**になっている。「アプリ側のコードでチェックしているから、データベース側の制約は無くてもよいのでは？」という疑問が浮かんだ場合は、このSQLのコメントにある「制約違反の生エラーがそのまま『システムエラー』になってしまう」という記述を思い出すとよい。アプリ側のチェックは「分かりやすいエラーメッセージを出すため」、データベース側の制約は「アプリのチェックをすり抜けた場合の最終防御」として、両方が役割を持っている。
-

全体の振り返り・索引

ファイル一覧（読んだ順）

Day	ファイル	種類	主な役割
1	model/Logic.java	Java（インターフェース）	すべてのLogicクラスが実装する共通の型
1	entity/Account.java	Java（エンティティ）	users テーブルの1行を表す
1	entity/Seat.java	Java（エンティティ）	seats テーブルの1行を表す
1	entity/Location.java	Java（エンティティ）	departments テーブルの1行を表す
1	entity/Reservation.java	Java（エンティティ）	reservations テーブルの1行を表す
1	dao/DBUtil.java	Java（DAO共通）	データベース接続をまとめて作成する
2	webapp/.../login.jsp	JSP	ログイン画面
2	model/LoginLogic.java	Java（Logic）	ログイン処理
2	dao/AccountsDAO.java	Java（DAO）	users テーブルの検索
2	webapp/.../loginNG.jsp	JSP	ログイン失敗画面
2	model/LogoutLogic.java	Java（Logic）	ログアウト処理
2	webapp/.../logout.jsp	JSP	ログアウト完了画面
2	webapp/.../common/header.jspf	JSPF	一般ユーザー向け共通ヘッダー
2	webapp/.../common/headerAdmin.jspf	JSPF	管理者向け共通ヘッダー
3	webapp/.../locationRegist.jsp	JSP	行先登録画面
3	model/LocationRegistLogic.java	Java（Logic）	行先登録処理（出社・在宅・出張）
3	webapp/.../businessTrip.jsp	JSP	出張登録画面
3	webapp/.../remoteWork.jsp	JSP	在宅勤務登録画面
3	dao/LocationsDAO.java	Java（DAO）	departments テーブルの検索
3	dao/ReservationsDAO.java	Java（DAO）	reservations テーブルのCRUD
4	dao/SeatsDAO.java	Java（DAO）	seats テーブルの検索・画像名解決
4	model/ReserveLogic.java	Java（Logic）	座席予約フロー（拠点→フロア→エリア→座席）
4	webapp/.../reserve.jsp	JSP	座席予約画面
4	webapp/.../reserveOK.jsp	JSP	予約完了画面
4	webapp/.../reserveNG.jsp	JSP	予約失敗画面
5	model/ReserveConfirmLogic.java	Java（Logic）	予約確認・キャンセル処理
5	webapp/.../reserveConfirm.jsp	JSP	予約確認・キャンセル画面
5	model/NameSearchLogic.java	Java（Logic）	氏名検索処理
5	webapp/.../nameSearch.jsp	JSP	氏名検索画面
5	model/DepartmentSearchLogic.java	Java（Logic）	部門検索処理
5	webapp/.../departmentSearch.jsp	JSP	部門検索画面
6	webapp/.../menu.jsp	JSP	一般メニュー画面
6	control/ControlServlet.java（showMenu()）	Java（サーブレット）	メニュー表示の共通処理

Day	ファイル	種類	主な役割
6	model/AdminReserveLogic.java	Java (Logic)	代理予約処理 (個人・会議)
6	webapp/.../adminMenu.jsp	JSP	管理者メニュー画面
6	webapp/.../seatStatus.jsp	JSP	座席利用状況確認画面
6	control/ControlServlet.java (showSeatStatus())	Java (サーブレット)	座席利用状況の準備処理
6	webapp/.../adminReserve.jsp	JSP	代理予約画面
7	model/MasterUpdateLogic.java	Java (Logic)	座席マスタCRUD処理
7	dao/MastersDAO.java	Java (DAO)	座席マスタのCRUD用DAO
7	webapp/.../masterUpdate.jsp	JSP	マスタ更新画面
7	docs/db/za_navi_schema.sql	SQL	データベース定義 (4テーブル)
-	control/ControlServlet.java (全体)	Java (サーブレット)	フロントコントローラ (全アクションの振り分け)

※ control/ControlServlet.java は、Day2 (セッションチェック・全体のdispatch) とDay6 (showMenu() / showSeatStatus()) で分けて読んだが、表では便宜上1行にまとめている。

用語・概念の索引 (初出のDay)

用語・概念	初出
インターフェース (implements)	Day1
エンティティクラス (getter/setter)	Day1
try-with-resources	Day1
PreparedStatement /SQLインジェクション対策	Day1
HttpSession / session.setAttribute()	Day2
request.setAttribute() と <%= %>	Day2
<%@ include file="..." %> (JSPF)	Day2
List / ArrayList	Day2
Map / HashMap	Day3
stream().filter()	Day3
三項演算子 (? :)	Day3
LocalDate	Day4
Collectors.toList()	Day4
Set / HashSet	Day6
LinkedHashMap	Day6
localStorage / sessionStorage	Day2／Day6
<details> / <summary>	Day6
JavaScriptの Array.filter() / map() / forEach()	Day6
DOM操作 (createElement / appendChild)	Day6
String.matches() による値の検証 (XSS対策)	Day6
try のネスト	Day7
PRIMARY KEY / FOREIGN KEY / AUTO_INCREMENT /生成列	Day7

おわりに

この資料は、座・Naviの全40ファイル（Java 21ファイル + JSP/JSPF 19ファイル）を、初めてこのコードを読む人でも1週間（Day1〜7）で読み進められるように構成したもの。各Dayの最後にある「①解説書との関連」を見比べながら読むと、「①プログラム解説書」で説明されている機能・画面遷移が、実際にどのコードによって実現されているかが分かるようになっている。

チームで読み進める際は、

1. まず①解説書で「この機能が何のためにあるか」を確認する
2. その機能に対応するDayをこの②解説書で読み、「ここで新しく出てくるもの」で初出の概念を確認する
3. 「読み方のポイント」で、他のDayとのつながりを確認する

という順番がおすすめ。分からないことがあれば、用語・概念の索引から「初出のDay」に戻って確認するか、チーム内で質問してほしい。